Cuaderno de Isabelle

Las teorías del laboratorio de lógica del Aula de Filosofía

15 teorías · 298 resultados demostrados · 23-10-2026

Isabelle/HOL es un programa que comprueba demostraciones paso a paso: si acepta una prueba, no tiene fallos lógicos. Aquí están, tal como se escribieron, todas las teorías que sostienen la mesa «Mundos posibles» del Rincón de lógica: cada premisa es una `definition`, y cada afirmación que la mesa da por demostrada es un `theorem` o un `lemma` con su prueba. Los comentarios van entre `(*` y `*)`. Todas se comprobaron con Isabelle2025-2 sin ningún error.

Teoría	**Resultados**
De práctica: tiempo, duración y creación	17
El tiempo, definido	13
Propiedades de las relaciones	10
La Ética en lógica de enunciados	6
La Ética con una sola conectiva	7
Spinoza, Ética I	77
Spinoza: solo con lo que él cita	10
Spinoza: el núcleo mínimo	77
Spinoza: diagnóstico del «en cuanto»	6
Spinoza: premisas que sobran	7
Las cinco vías	12
Las antinomias de Kant: el comienzo y lo simple	15
Las antinomias de Kant: la libertad y el ser necesario	15
Del ser al cosmos: Parménides y sus herederos	12
Del ser al cosmos: el Demiurgo	14

Índice

De práctica: tiempo, duración y creación 3
El tiempo, definido 6
Propiedades de las relaciones 9
La Ética en lógica de enunciados 12
La Ética con una sola conectiva 14
Spinoza, Ética I 16
Spinoza: solo con lo que él cita 36
Spinoza: el núcleo mínimo 39
Spinoza: diagnóstico del «en cuanto» 56
Spinoza: premisas que sobran 57
Las cinco vías 59
Las antinomias de Kant: el comienzo y lo simple 63
Las antinomias de Kant: la libertad y el ser necesario 69
Del ser al cosmos: Parménides y sus herederos 74
Del ser al cosmos: el Demiurgo 77

De práctica: tiempo, duración y creación

La teoría de práctica de la mesa: empezar antes, durar más y crear, con sus premisas y las proposiciones que se siguen o no.

Fichero `Mundo_tiempo_creacion.thy` ▪ 17 resultados demostrados ▪ 170 líneas

```
theory Mundo_tiempo_creacion
  imports Main
begin

(* Mundo de práctica del Laboratorio (21-10-2026): cosas que se distinguen por la precedencia
   en el tiempo, la duración y si unas fueron creadas por otras. Mismos nombres que la mesa web. *)

typedecl cosa

consts
  Antes :: "cosa ⇒ cosa ⇒ bool"
  DuraMas :: "cosa ⇒ cosa ⇒ bool"
  Crea :: "cosa ⇒ cosa ⇒ bool"

definition T_irreflexiva :: bool where "T_irreflexiva ≡ ∀x. ¬ Antes x x"
definition T_transitiva :: bool where "T_transitiva ≡ ∀x y z. Antes x y ∧ Antes y z ⟶ Antes x z"
definition D_irreflexiva :: bool where "D_irreflexiva ≡ ∀x. ¬ DuraMas x x"
definition D_transitiva :: bool where "D_transitiva ≡ ∀x y z. DuraMas x y ∧ DuraMas y z ⟶ DuraMas x z"
definition C_creador_anterior :: bool where "C_creador_anterior ≡ ∀x y. Crea x y ⟶ Antes x y"
definition C_un_creador :: bool where "C_un_creador ≡ ∀x y z. Crea x z ∧ Crea y z ⟶ x = y"
definition C_creador_dura_mas :: bool where "C_creador_dura_mas ≡ ∀x y. Crea x y ⟶ DuraMas x y"
definition C_todo_creado :: bool where "C_todo_creado ≡ ∀x. ∃y. Crea y x"

definition increado :: "cosa ⇒ bool" where "increado x ≡ ¬ (∃y. Crea y x)"

(* Q1: Nada se crea a sí mismo. *)
theorem Q1:
  assumes "C_creador_anterior" "T_irreflexiva"
  shows "¬ Crea x x"
  using assms unfolding C_creador_anterior_def T_irreflexiva_def by blast

(* Q1 por otro camino, el de la duración (propuesto por el profesor, 21-10): si x se creara a sí mismo,
   duraría más que sí mismo. Usa la premisa discutible: si se quita, este camino se cae y queda el del
tiempo. *)
theorem Q1_por_duracion:
  assumes "C_creador_dura_mas" "D_irreflexiva"
  shows "¬ Crea x x"
  using assms unfolding C_creador_dura_mas_def D_irreflexiva_def by blast

lemma Q1_sin_irreflexiva: "C_creador_anterior ⟹ ¬ Crea x x"
  nitpick [expect = genuine, timeout = 120]
  oops

(* Q2: Si x crea a y, y no crea a x. *)
theorem Q2:
  assumes "C_creador_anterior" "T_irreflexiva" "T_transitiva"
  shows "Crea x y ⟶ ¬ Crea y x"
  using assms unfolding C_creador_anterior_def T_irreflexiva_def T_transitiva_def by blast

lemma Q2_sin_transitiva: "C_creador_anterior ⟹ T_irreflexiva ⟹ Crea x y ⟶ ¬ Crea y x"
  nitpick [expect = genuine, timeout = 120]
  oops

(* Q3: El creador del creador es anterior a lo creado. *)
theorem Q3:
  assumes "C_creador_anterior" "T_transitiva"
  shows "Crea x y ∧ Crea y z ⟶ Antes x z"
  using assms unfolding C_creador_anterior_def T_transitiva_def by blast

(* Q4: Lo creado no dura más que su creador. *)
theorem Q4:
```

```
  assumes "C_creador_dura_mas" "D_irreflexiva" "D_transitiva"
  shows "Crea x y ⟶ ¬ DuraMas y x"
  using assms unfolding C_creador_dura_mas_def D_irreflexiva_def D_transitiva_def by blast

lemma Q4_sin_dura_mas:
  "T_irreflexiva ⟹ T_transitiva ⟹ D_irreflexiva ⟹ D_transitiva ⟹ C_creador_anterior ⟹
C_un_creador ⟹
   Crea x y ⟶ ¬ DuraMas y x"
  nitpick [expect = genuine, timeout = 120]
  oops

(* Q8 (en la mesa, la 5): Lo creado puede sobrevivir a su creador. Es una POSIBILIDAD: hay un mundo
   que cumple todas las premisas salvo la discutible (y «todo ha sido creado») en el que algo creado
   dura más que quien lo creó: el autor que muere y el libro que queda. Mundo de dos cosas:
   True (el autor) crea y empieza antes que False (el libro), y False dura más que True. *)
lemma Q8_posible:
  fixes A D C :: "bool ⇒ bool ⇒ bool"
  defines "A ≡ λx y. x ∧ ¬ y" and "D ≡ λx y. ¬ x ∧ y" and "C ≡ λx y. x ∧ ¬ y"
  shows "(∀x. ¬ A x x) ∧ (∀x y z. A x y ∧ A y z ⟶ A x z)
         ∧ (∀x. ¬ D x x) ∧ (∀x y z. D x y ∧ D y z ⟶ D x z)
         ∧ (∀x y. C x y ⟶ A x y) ∧ (∀x y z. C x z ∧ C y z ⟶ x = y)
         ∧ (∃x y. C x y ∧ D y x)"
  unfolding A_def D_def C_def by auto

(* Y con la premisa discutible ya no es posible: es Q4. *)
corollary Q8_imposible_con_dura_mas:
  assumes "C_creador_dura_mas" "D_irreflexiva" "D_transitiva"
  shows "¬ (∃x y. Crea x y ∧ DuraMas y x)"
  using Q4 [OF assms] by blast

(* Q9 (en la mesa, la 9): Algo puede tener dos creadores. Otra POSIBILIDAD: sin «nada tiene dos
   creadores» hay un mundo, con todas las demás premisas compatibles (también la discutible), en el que
   dos cosas distintas crean a una tercera. Hacen falta tres cosas: 0 y 1 crean a 2. *)
lemma Q9_posible:
  fixes A D C :: "nat ⇒ nat ⇒ bool"
  defines "A ≡ λx y. x < y" and "C ≡ λx y. x < 2 ∧ y = 2" and "D ≡ λx y. x < 2 ∧ y = 2"
  shows "(∀x. ¬ A x x) ∧ (∀x y z. A x y ∧ A y z ⟶ A x z)
         ∧ (∀x. ¬ D x x) ∧ (∀x y z. D x y ∧ D y z ⟶ D x z)
         ∧ (∀x y. C x y ⟶ A x y) ∧ (∀x y. C x y ⟶ D x y)
         ∧ (∃x y z. x ≠ y ∧ C x z ∧ C y z)"
  unfolding A_def C_def D_def
  apply (intro conjI)
        apply auto[6]
  by (rule exI [of _ 0], rule exI [of _ 1], rule exI [of _ 2]) simp

(* Con «nada tiene dos creadores» no puede ocurrir: es su definición. *)
lemma Q9_imposible_con_un_creador:
  assumes "C_un_creador"
  shows "¬ (∃x y z. x ≠ y ∧ Crea x z ∧ Crea y z)"
  using assms unfolding C_un_creador_def by blast

(* Q5: Lo anterior dura más. No se sigue ni con todas las premisas (salvo la incompatible). *)
lemma Q5_no_se_sigue:
  "T_irreflexiva ⟹ T_transitiva ⟹ D_irreflexiva ⟹ D_transitiva ⟹ C_creador_anterior ⟹
C_un_creador ⟹
   C_creador_dura_mas ⟹ Antes x y ⟶ DuraMas x y"
  nitpick [expect = genuine, timeout = 120]
  oops

(* Q6: Hay algo increado. Vale en todo mundo FINITO... *)
theorem Q6_finito:
  assumes fin: "finite (UNIV :: cosa set)"
    and "C_creador_anterior" "T_irreflexiva" "T_transitiva"
  shows "∃x. increado x"
proof -
  let ?R = "{(y, x). Crea y x}"
  have ant: "⋀a b. (a, b) ∈ ?R⇧+ ⟹ Antes a b"
  proof -
```

```
    fix a b assume "(a, b) ∈ ?R⇧+"
    then show "Antes a b"
    proof (induction rule: trancl_induct)
      case (base y) then show ?case using assms(2) unfolding C_creador_anterior_def by blast
    next
      case (step y z) then show ?case using assms(2,4) unfolding C_creador_anterior_def T_transitiva_def
by blast
    qed
  qed
  have "acyclic ?R" unfolding acyclic_def using ant assms(3) unfolding T_irreflexiva_def by blast
  moreover have "finite ?R"
  proof -
    have "finite (UNIV :: (cosa × cosa) set)" using fin by (simp add: finite_Prod_UNIV)
    then show ?thesis by (rule finite_subset[rotated]) simp
  qed
  ultimately have "wf ?R" by (simp add: finite_acyclic_wf)
  then obtain x where "x ∈ (UNIV :: cosa set)" "∀y. (y, x) ∈ ?R ⟶ y ∉ UNIV"
    using wfE_min [of ?R "undefined" UNIV] by blast
  then show ?thesis unfolding increado_def by blast
qed

(* ... y por eso, en un mundo finito, «todo ha sido creado» es incompatible con las tres premisas. *)
corollary todo_creado_incompatible_finito:
  assumes "finite (UNIV :: cosa set)" "C_creador_anterior" "T_irreflexiva" "T_transitiva"
  shows "¬ C_todo_creado"
  using Q6_finito [OF assms] unfolding increado_def C_todo_creado_def by blast

(* ... pero no en uno infinito: los números naturales, donde n + 1 crea a n y lo mayor es anterior.
   Cumple las premisas de tiempo y creación, y todo ha sido creado: no hay nada increado. *)
lemma Q6_contramodelo_infinito:
  fixes A C :: "nat ⇒ nat ⇒ bool"
  defines "A ≡ λx y. y < x" and "C ≡ λx y. x = Suc y"
  shows "(∀x. ¬ A x x) ∧ (∀x y z. A x y ∧ A y z ⟶ A x z) ∧ (∀x y. C x y ⟶ A x y)
         ∧ (∀x y z. C x z ∧ C y z ⟶ x = y) ∧ (∀x. ∃y. C y x) ∧ ¬ (∃x. ¬ (∃y. C y x))"
  unfolding A_def C_def by auto

(* Q7: Hay un único increado. No se sigue: puede haber dos orígenes. *)
lemma Q7_no_se_sigue:
  "T_irreflexiva ⟹ T_transitiva ⟹ D_irreflexiva ⟹ D_transitiva ⟹ C_creador_anterior ⟹
C_un_creador ⟹
   C_creador_dura_mas ⟹ increado x ⟹ increado y ⟹ x = y"
  nitpick [expect = genuine, timeout = 120]
  oops

end
```

El tiempo, definido

Lo mismo, pero con instantes: empezar, acabar, durar y sobrevivir dejan de ser relaciones sueltas y se definen.

Fichero `Mundo_tiempo_definido.thy` · 13 resultados demostrados · 123 líneas

```
theory Mundo_tiempo_definido
  imports Main
begin

(* Tercer mundo del Laboratorio (21-10-2026, propuesta del profesor): «El tiempo, definido».
   El mundo de práctica dejaba Antes y DuraMas como relaciones sueltas. Aquí se dice qué es el tiempo:
   instantes ordenados (los números naturales: el tiempo tiene un principio) y, para cada cosa, el
instante
   en que empieza y el instante en que acaba. Antes, DuraMas, Sobrevive y ExisteEn pasan a ser
definiciones;
   Crea sigue siendo primitiva. Mismos nombres que la mesa web. *)

typedecl cosa

consts
  empieza :: "cosa ⇒ nat"
  acaba :: "cosa ⇒ nat"
  Crea :: "cosa ⇒ cosa ⇒ bool"

(* Lo que es el tiempo en este mundo: nada acaba antes de empezar. *)
definition Tiempo :: bool where "Tiempo ≡ ∀x. empieza x ≤ acaba x"

definition Antes :: "cosa ⇒ cosa ⇒ bool" where "Antes x y ≡ empieza x < empieza y"
definition Sobrevive :: "cosa ⇒ cosa ⇒ bool" where "Sobrevive x y ≡ acaba y < acaba x"
definition DuraMas :: "cosa ⇒ cosa ⇒ bool" where "DuraMas x y ≡ acaba y - empieza y < acaba x - empieza
x"
definition ExisteEn :: "cosa ⇒ nat ⇒ bool" where "ExisteEn x t ≡ empieza x ≤ t ∧ t ≤ acaba x"
definition increado :: "cosa ⇒ bool" where "increado x ≡ ¬ (∃y. Crea y x)"

(* Premisas de la creación: estas sí se pueden quitar. *)
definition K_existe :: bool where "K_existe ≡ ∀x y. Crea x y ⟶ ExisteEn x (empieza y)"
definition K_antes :: bool where "K_antes ≡ ∀x y. Crea x y ⟶ Antes x y"
definition K_no_despues :: bool where "K_no_despues ≡ ∀x y. Crea x y ⟶ empieza x ≤ empieza y"
definition K_un_creador :: bool where "K_un_creador ≡ ∀x y z. Crea x z ∧ Crea y z ⟶ x = y"
definition K_todo_creado :: bool where "K_todo_creado ≡ ∀x. ∃y. Crea y x"

(* R1-R3: lo que en el mundo de práctica eran premisas, aquí son teoremas, sin premisa ninguna. *)
theorem R1: "¬ Antes x x" unfolding Antes_def by simp
theorem R2: "Antes x y ∧ Antes y z ⟶ Antes x z" unfolding Antes_def by simp
theorem R3: "Antes x y ⟶ ¬ Antes y x" unfolding Antes_def by simp

(* R4: Nada se crea a sí mismo. *)
theorem R4:
  assumes "K_antes"
  shows "¬ Crea x x"
  using assms R1 unfolding K_antes_def by blast

lemma R4_sin_premisa: "¬ Crea x x"
  nitpick [expect = genuine, timeout = 120]
  oops

(* R5 y R6: posibilidades. Un autor (True) que empieza en 0 y acaba en 2 crea en el instante 1 un libro
   (False) que acaba en 3: el libro sobrevive al autor. Con el libro hasta 5, además dura más. *)
lemma R5_posible:
  fixes e a :: "bool ⇒ nat" and C :: "bool ⇒ bool ⇒ bool"
  defines "e ≡ λx. if x then 0 else 1" and "a ≡ λx. if x then 2 else 3" and "C ≡ λx y. x ∧ ¬ y"
  shows "(∀x. e x ≤ a x) ∧ (∀x y. C x y ⟶ e x ≤ e y ∧ e y ≤ a x) ∧ (∀x y. C x y ⟶ e x < e y)
         ∧ (∀x y z. C x z ∧ C y z ⟶ x = y) ∧ (∃x y. C x y ∧ a x < a y)"
  unfolding e_def a_def C_def by auto

lemma R6_posible:
```

```
  fixes e a :: "bool ⇒ nat" and C :: "bool ⇒ bool ⇒ bool"
  defines "e ≡ λx. if x then 0 else 1" and "a ≡ λx. if x then 2 else 5" and "C ≡ λx y. x ∧ ¬ y"
  shows "(∀x. e x ≤ a x) ∧ (∀x y. C x y ⟶ e x ≤ e y ∧ e y ≤ a x) ∧ (∀x y. C x y ⟶ e x < e y)
         ∧ (∀x y z. C x z ∧ C y z ⟶ x = y) ∧ (∃x y. C x y ∧ a x - e x < a y - e y)"
  unfolding e_def a_def C_def by auto

(* R7: Lo que sobrevive dura más. No se sigue: se puede empezar tarde y acabar un poco después. *)
lemma R7_no_se_sigue: "Tiempo ⟹ Sobrevive x y ⟶ DuraMas x y"
  nitpick [expect = genuine, timeout = 120]
  oops

(* R8: Quien empieza antes y acaba después dura más. Se sigue de lo que es el tiempo. *)
theorem R8:
  assumes "Tiempo"
  shows "Antes x y ∧ Sobrevive x y ⟶ DuraMas x y"
proof
  assume h: "Antes x y ∧ Sobrevive x y"
  have "empieza y ≤ acaba y" using assms unfolding Tiempo_def by blast
  then show "DuraMas x y" using h unfolding Antes_def Sobrevive_def DuraMas_def by linarith
qed

lemma R8_sin_tiempo: "Antes x y ∧ Sobrevive x y ⟶ DuraMas x y"
  nitpick [expect = genuine, timeout = 120]
  oops

(* R9: Hay algo increado. Como el tiempo tiene un principio (los naturales), lo que empieza primero no
puede
   tener creador si quien crea empezó antes. Vale con cualquier número de cosas, también infinito:
   es el contraste con el mundo de práctica, donde solo valía en los mundos finitos. *)
theorem R9:
  assumes "K_antes"
  shows "∃x. increado x"
proof -
  obtain x where min: "∀y. empieza x ≤ empieza y"
  proof -
    let ?n = "LEAST n. ∃z. empieza z = n"
    have "∃z. empieza z = ?n" by (rule LeastI_ex) blast
    then obtain z where z: "empieza z = ?n" by blast
    have "∀y. empieza z ≤ empieza y" using z by (metis (mono_tags) Least_le)
    then show ?thesis using that by blast
  qed
  have "¬ (∃y. Crea y x)" using assms min unfolding K_antes_def Antes_def by (meson leD)
  then show ?thesis unfolding increado_def by blast
qed

(* Y por eso «todo ha sido creado» es incompatible con «quien crea empezó antes», aquí sin necesidad de
que el mundo sea finito. *)
corollary todo_creado_incompatible:
  assumes "K_antes"
  shows "¬ K_todo_creado"
  using R9 [OF assms] unfolding increado_def K_todo_creado_def by blast

(* La 9 no obliga a aceptar algo increado: dice que no caben juntas tres cosas (todo tiene creador, quien
crea empezó
   antes y el tiempo tiene principio). Observación del profesor (21-10): quien quiera que todo tenga
creador puede renunciar
   a K_antes, y admitir que se crea A LA VEZ o que algo se crea a sí mismo (la causa sui de Spinoza, Ética
I, def. 1).
   R10: «Todo puede tener creador». Con K_no_despues (crear a la vez), K_existe y K_un_creador, una sola
cosa que se crea a sí misma. *)
lemma R10_posible:
  fixes e a :: "bool ⇒ nat" and C :: "bool ⇒ bool ⇒ bool"
  defines "e ≡ λx. 0" and "a ≡ λx. 1" and "C ≡ λx y. x = y"
  shows "(∀x. e x ≤ a x) ∧ (∀x y. C x y ⟶ e x ≤ e y ∧ e y ≤ a x) ∧ (∀x y. C x y ⟶ e x ≤ e y)
         ∧ (∀x y z. C x z ∧ C y z ⟶ x = y) ∧ (∀x. ∃y. C y x)"
  unfolding e_def a_def C_def by auto

(* Y con K_antes no es posible: es la 9 (todo_creado_incompatible). *)
```

```
end
```

Propiedades de las relaciones

Qué propiedad de una relación (reflexiva, simétrica, transitiva, sin regreso al infinito…) se sigue de cuáles, con prueba, y cuáles no, con contraejemplo.

Fichero `Propiedades_relaciones.thy` ▪ 10 resultados demostrados ▪ 188 líneas

```
theory Propiedades_relaciones
  imports Main
begin

(* Mapa de propiedades de las relaciones (22-10-2026), para la versión «por capas» de la mesa del
Laboratorio:
   qué propiedad de una relación se sigue de cuáles, con prueba, y cuáles no, con contraejemplo.
   Vale para cualquier relación R (Antes, Crea, Mueve, Causa, MasPerfecto…). Mismos nombres que la mesa
web.
   Pruebas escritas paso a paso, sin auto ni presburger: en Cinco_vias.thy una de esas búsquedas no
terminaba. *)

definition irreflexiva :: "('a ⇒ 'a ⇒ bool) ⇒ bool" where "irreflexiva R ≡ ∀x. ¬ R x x"
definition transitiva :: "('a ⇒ 'a ⇒ bool) ⇒ bool" where "transitiva R ≡ ∀x y z. R x y ∧ R y z ⟶ R x
z"
definition asimetrica :: "('a ⇒ 'a ⇒ bool) ⇒ bool" where "asimetrica R ≡ ∀x y. R x y ⟶ ¬ R y x"
(* Sin regreso al infinito: no hay cadena infinita x1 ← x2 ← x3 ← …; la relación está bien fundada. *)
definition sin_regreso :: "('a ⇒ 'a ⇒ bool) ⇒ bool" where "sin_regreso R ≡ wf {(y, x). R y x}"
definition total :: "('a ⇒ 'a ⇒ bool) ⇒ bool" where "total R ≡ ∀x y. x ≠ y ⟶ R x y ∨ R y x"

(* ---------- Lo que se sigue ---------- *)

(* M1. Irreflexiva y transitiva: asimétrica. *)
lemma M1_irreflexiva_transitiva_asimetrica:
  assumes "irreflexiva R" "transitiva R"
  shows "asimetrica R"
  unfolding asimetrica_def
proof (intro allI impI notI)
  fix x y assume "R x y" "R y x"
  then have "R x x" using assms(2) unfolding transitiva_def by blast
  then show False using assms(1) unfolding irreflexiva_def by simp
qed

(* M2. Asimétrica: irreflexiva. *)
lemma M2_asimetrica_irreflexiva:
  assumes "asimetrica R"
  shows "irreflexiva R"
  unfolding irreflexiva_def
proof (intro allI notI)
  fix x assume "R x x"
  then show False using assms unfolding asimetrica_def by blast
qed

(* M3. Sin regreso al infinito: asimétrica (y por M2, irreflexiva). *)
lemma M3_sin_regreso_asimetrica:
  assumes "sin_regreso R"
  shows "asimetrica R"
  unfolding asimetrica_def
proof (intro allI impI notI)
  fix x y assume xy: "R x y" and yx: "R y x"
  let ?r = "{(y, x). R y x}"
  have w: "wf ?r" using assms unfolding sin_regreso_def .
  have "(x, y) ∈ ?r⇧+" using xy by (intro r_into_trancl) simp
  moreover have "(y, x) ∈ ?r⇧+" using yx by (intro r_into_trancl) simp
  ultimately have "(x, x) ∈ ?r⇧+" by (rule trancl_trans)
  moreover have "(x, x) ∉ ?r⇧+" using wf_trancl [OF w] by (rule wf_not_refl)
  ultimately show False by contradiction
qed

(* M4. Sin regreso al infinito, si hay alguna flecha, hay un primero: algo que tiene flechas salientes y
ninguna entrante. *)
```

```
lemma M4_sin_regreso_hay_primero:
  assumes "sin_regreso R" "R a b"
  shows "∃x. ¬ (∃y. R y x) ∧ (∃z. R x z)"
proof -
  let ?r = "{(y, x). R y x}" and ?S = "{x. ∃z. R x z}"
  have "a ∈ ?S" using assms(2) by blast
  have "∃z∈?S. ∀y. (y, z) ∈ ?r ⟶ y ∉ ?S"
    using wf_eq_minimal[THEN iffD1, OF assms(1)[unfolded sin_regreso_def], rule_format, OF ‹a ∈ ?S›] .
  then obtain x where "x ∈ ?S" and min: "∀y. (y, x) ∈ ?r ⟶ y ∉ ?S" by blast
  have "¬ (∃y. R y x)"
  proof
    assume "∃y. R y x"
    then obtain y where "R y x" by blast
    then have "y ∈ ?S" by blast
    moreover have "(y, x) ∈ ?r" using ‹R y x› by simp
    ultimately show False using min by blast
  qed
  then show ?thesis using ‹x ∈ ?S› by blast
qed

(* M5. En un mundo finito, irreflexiva y transitiva: sin regreso al infinito. *)
lemma M5_finito_sin_regreso:
  assumes fin: "finite (UNIV :: 'a set)" and irr: "irreflexiva (R :: 'a ⇒ 'a ⇒ bool)" and tr: "transitiva
R"
  shows "sin_regreso R"
proof -
  let ?r = "{(y, x). R y x}"
  have cl: "(a, b) ∈ ?r⇧+ ⟹ R a b" for a b
  proof (induction rule: trancl_induct)
    case (base y) then show ?case by simp
  next
    case (step y z) then show ?case using tr unfolding transitiva_def by blast
  qed
  have "acyclic ?r" unfolding acyclic_def
  proof (intro allI notI)
    fix x assume "(x, x) ∈ ?r⇧+"
    then have "R x x" by (rule cl)
    then show False using irr unfolding irreflexiva_def by simp
  qed
  moreover have "finite ?r"
  proof -
    have "finite (UNIV :: ('a × 'a) set)" using fin by (simp add: finite_Prod_UNIV)
    then show ?thesis by (rule finite_subset[rotated]) simp
  qed
  ultimately show ?thesis unfolding sin_regreso_def by (simp add: finite_acyclic_wf)
qed

(* M6. En un mundo finito, un orden estricto y total tiene máximo: algo que supera a todo lo demás. *)
lemma M6_finito_total_maximo:
  assumes fin: "finite (UNIV :: 'a set)" and irr: "irreflexiva (R :: 'a ⇒ 'a ⇒ bool)" and tr: "transitiva
R" and tot: "total R"
  shows "∃x. ∀y. y ≠ x ⟶ R x y"
proof -
  let ?r = "{(y, x). R y x}"
  have "wf ?r" using M5_finito_sin_regreso[OF fin irr tr] unfolding sin_regreso_def .
  then have "∃z∈(UNIV :: 'a set). ∀y. (y, z) ∈ ?r ⟶ y ∉ UNIV"
    by (rule wf_eq_minimal[THEN iffD1, rule_format, OF _ UNIV_I])
  then obtain x where "∀y. (y, x) ∈ ?r ⟶ y ∉ (UNIV :: 'a set)" by blast
  then have nadie: "∀y. ¬ R y x" by simp
  show ?thesis
  proof (intro exI allI impI)
    fix y assume "y ≠ x"
    then have "R y x ∨ R x y" using tot unfolding total_def by blast
    then show "R x y" using nadie by blast
  qed
qed

(* ---------- Lo que no se sigue (contraejemplos) ---------- *)
```

```
(* N1. Asimétrica no implica transitiva: 0 → 1 → 2, pero no 0 → 2. *)
lemma N1_asimetrica_no_transitiva:
  fixes R :: "nat ⇒ nat ⇒ bool"
  defines "R ≡ λx y. y = Suc x"
  shows "asimetrica R ∧ ¬ transitiva R"
proof
  show "asimetrica R" unfolding asimetrica_def R_def by simp
  show "¬ transitiva R"
  proof
    assume "transitiva R"
    then have "R 0 1 ∧ R 1 2 ⟶ R 0 2" unfolding transitiva_def by blast
    then show False unfolding R_def by simp
  qed
qed

(* N2. Irreflexiva no implica asimétrica: «es distinto de». *)
lemma N2_irreflexiva_no_asimetrica:
  fixes R :: "bool ⇒ bool ⇒ bool"
  defines "R ≡ λx y. x ≠ y"
  shows "irreflexiva R ∧ ¬ asimetrica R"
proof
  show "irreflexiva R" unfolding irreflexiva_def R_def by simp
  show "¬ asimetrica R"
  proof
    assume "asimetrica R"
    then have "R True False ⟶ ¬ R False True" unfolding asimetrica_def by blast
    then show False unfolding R_def by simp
  qed
qed

(* N3. Transitiva no implica irreflexiva: la relación que une todo con todo. *)
lemma N3_transitiva_no_irreflexiva:
  fixes R :: "bool ⇒ bool ⇒ bool"
  defines "R ≡ λx y. True"
  shows "transitiva R ∧ ¬ irreflexiva R"
  unfolding transitiva_def irreflexiva_def R_def by simp

(* N4. Con infinitas cosas, irreflexiva y transitiva no basta para que no haya regreso al infinito:
   en los números, a cada uno lo precede (en la relación) el siguiente, sin fin. Compárese con M5. *)
lemma N4_infinito_con_regreso:
  fixes R :: "nat ⇒ nat ⇒ bool"
  defines "R ≡ λx y. y < x"
  shows "irreflexiva R ∧ transitiva R ∧ ¬ sin_regreso R"
proof (intro conjI)
  show "irreflexiva R" unfolding irreflexiva_def R_def by simp
  show "transitiva R" unfolding transitiva_def R_def
  proof (intro allI impI)
    fix x y z :: nat assume "y < x ∧ z < y"
    then show "z < x" by (elim conjE) (rule less_trans)
  qed
  show "¬ sin_regreso R"
  proof
    assume "sin_regreso R"
    then have w: "wf {(y, x). R y x}" unfolding sin_regreso_def .
    have h: "∀i. (Suc i, i) ∈ {(y, x). R y x}" unfolding R_def by simp
    have "∃f. ∀i. (f (Suc i), f i) ∈ {(y, x). R y x}"
    proof (rule exI [of _ "λn. n"])
      show "∀i. ((λn. n) (Suc i), (λn. n) i) ∈ {(y, x). R y x}" using h by simp
    qed
    then show False using w unfolding wf_iff_no_infinite_down_chain by blast
  qed
qed

end
```

La Ética en lógica de enunciados

El esqueleto de la Ética de Spinoza, solo con negación, disyunción y conjunción: el condicional y el bicondicional se reducen a ellas.

Fichero `Etica_proposicional.thy` ▪ 6 resultados demostrados ▪ 87 líneas

```
theory Etica_proposicional
imports Main
begin

text ‹La Ética entera, en argumentación proposicional — ahora solo con negación,
  disyunción y conjunción (¬, ∨, ∧). Se prescinde del condicional y del
  bicondicional mediante sus equivalentes:

    A ⟶ B  ≡  ¬ A ∨ B                (el condicional es una disyunción)
    A ⟷ B  ≡  (A ∧ B) ∨ (¬ A ∧ ¬ B)   («o ambos o ninguno»)

  Todo se demuestra por pura lógica proposicional (blast), sin ⟶ ni
  ⟷ ni cuantificadores.›

consts
  (* Parte I *)
  Eficaz :: bool        (* P36: todo es eficaz *)
  Contingencia :: bool  (* P29: hay algo contingente (se niega) *)
  (* Parte II *)
  Identidad :: bool     (* lectura de identidad (II.7) *)
  Paralelismo :: bool   (* el orden de las ideas = el orden de las cosas *)
  Adecuacion :: bool    (* II.43: la verdad es la adecuación *)
  (* Parte III *)
  Conato :: bool        (* III.6: todo persevera en su ser *)
  Actividad :: bool     (* III.1-3: obrar = causa adecuada *)
  Alegria :: bool       (* III.11 *)
  Amor :: bool          (* III.13 *)
  (* Parte IV *)
  Virtud :: bool        (* IV.8: virtud = potencia *)
  Bueno :: bool         (* def.1 *)
  Malo :: bool          (* def.2 *)
  (* Parte V *)
  Beatitud :: bool      (* V.42: beatitud = virtud *)

axiomatization where
  A_identidad:        "¬ Identidad ∨ Paralelismo" and
  A_no_contingencia:  "¬ Contingencia" and
  A_p36:              "Eficaz" and
  A_conato_es_potencia:   "(Conato ∧ Eficaz) ∨ (¬ Conato ∧ ¬ Eficaz)" and
  A_adecuacion_actividad: "(Actividad ∧ Adecuacion) ∨ (¬ Actividad ∧ ¬ Adecuacion)" and
  A_virtud_actividad:     "(Virtud ∧ Actividad) ∨ (¬ Virtud ∧ ¬ Actividad)" and
  A_virtud_es_conato: "¬ Virtud ∨ Conato" and
  A_beatitud_virtud:  "(Beatitud ∧ Virtud) ∨ (¬ Beatitud ∧ ¬ Virtud)" and
  A_amor_es_alegria:  "¬ Amor ∨ Alegria" and
  A_bueno_malo:       "¬ Bueno ∨ ¬ Malo"

text ‹III.6 — todo persevera. El «modus ponens» se vuelve silogismo disyuntivo:
  de «Eficaz» y de «Conato ∨ ¬ Eficaz» (la mitad del bicondicional) sale
  «Conato».›
theorem III6: "Conato"
  using A_p36 A_conato_es_potencia by blast

text ‹La cadena de la libertad (II ⟶ V) se comprime en un solo bicondicional:
  adecuación y beatitud «o ambas o ninguna».›
theorem libertad_es_beatitud:
  "(Adecuacion ∧ Beatitud) ∨ (¬ Adecuacion ∧ ¬ Beatitud)"
  using A_adecuacion_actividad A_virtud_actividad A_beatitud_virtud by blast

text ‹La beatitud es una forma del conato: «no-beatitud o conato» (pero no al
  revés: no todo lo que persevera es libre).›
theorem beatitud_implica_conato: "¬ Beatitud ∨ Conato"
```

```
  using A_beatitud_virtud A_virtud_es_conato by blast

text ‹El amor es una forma de la alegría.›
theorem amor_presupone_alegria: "¬ Amor ∨ Alegria"
  using A_amor_es_alegria by blast

text ‹El bien excluye el mal: no puede darse «bueno y malo» a la vez.›
theorem bueno_no_malo: "¬ (Bueno ∧ Malo)"
  using A_bueno_malo by blast

text ‹Las dos cadenas del libro, en un solo enunciado (solo ¬, ∨, ∧):›
theorem las_dos_cadenas:
  "((Conato ∧ Eficaz) ∨ (¬ Conato ∧ ¬ Eficaz)) ∧
   ((Adecuacion ∧ Beatitud) ∨ (¬ Adecuacion ∧ ¬ Beatitud)) ∧
   (¬ Beatitud ∨ Conato) ∧ (¬ Contingencia)"
proof -
  have c1: "(Conato ∧ Eficaz) ∨ (¬ Conato ∧ ¬ Eficaz)" using A_conato_es_potencia by blast
  have c2: "(Adecuacion ∧ Beatitud) ∨ (¬ Adecuacion ∧ ¬ Beatitud)"
    using A_adecuacion_actividad A_virtud_actividad A_beatitud_virtud by blast
  have c3: "¬ Beatitud ∨ Conato" using A_beatitud_virtud A_virtud_es_conato by blast
  have c4: "¬ Contingencia" using A_no_contingencia by blast
  from c1 c2 c3 c4 show ?thesis by blast
qed

end
```

La Ética con una sola conectiva

Todo, con una única conectiva, NAND (la barra de Sheffer): de cuatro conectivas a una sola.

Fichero `Etica_nand.thy` · 7 resultados demostrados · 69 líneas

```
theory Etica_nand
imports Main
begin

text ‹La única conectiva: NAND, la barra de Sheffer. Se escribe como función prefija
  ‹nand a b› (para evitar ambigüedades de precedencia; en la literatura es la barra
  $a ↑ b$). Todo lo demás se define a partir de ella. Es el mismo gesto que en la
  metafísica: de cuatro conectivas (y antes, de veinte relaciones) a UNA sola.›

definition nand :: "bool ⇒ bool ⇒ bool" where
  "nand a b ≡ ¬ (a ∧ b)"

text ‹Las tres conectivas básicas se reducen a NAND:›
lemma neg_nand:  "((¬ a) = nand a a)"  by (auto simp add: nand_def)
lemma conj_nand: "((a ∧ b) = nand (nand a b) (nand a b))"  by (auto simp add: nand_def)
lemma disj_nand: "((a ∨ b) = nand (nand a a) (nand b b))"  by (auto simp add: nand_def)

text ‹Y las otras dos, por composición:›
lemma imp_nand:  "((a ⟶ b) = nand a (nand b b))"  by (auto simp add: nand_def)
(* el bicondicional ya se había reducido a ¬/∨/∧, así que compone *)

text ‹Abreviaturas en NAND puro (no = negación, y = conjunción, ou = disyunción,
  en español):›
abbreviation no :: "bool ⇒ bool" where "no a ≡ nand a a"
abbreviation y  :: "bool ⇒ bool ⇒ bool" where "y a b ≡ nand (nand a b) (nand a b)"
abbreviation ou :: "bool ⇒ bool ⇒ bool" where "ou a b ≡ nand (nand a a) (nand b b)"

consts
  Adecuacion :: bool
  Actividad :: bool
  Virtud :: bool
  Beatitud :: bool
  Eficaz :: bool
  Conato :: bool

axiomatization where
  A_adecuacion_actividad: "(Actividad ∧ Adecuacion) ∨ (¬ Actividad ∧ ¬ Adecuacion)" and
  A_virtud_actividad:     "(Virtud ∧ Actividad) ∨ (¬ Virtud ∧ ¬ Actividad)" and
  A_beatitud_virtud:      "(Beatitud ∧ Virtud) ∨ (¬ Beatitud ∧ ¬ Virtud)" and
  A_conato_es_potencia:   "(Conato ∧ Eficaz) ∨ (¬ Conato ∧ ¬ Eficaz)"

text ‹La espina del libro —adecuación = actividad = virtud = beatitud— escrita
  ENTERAMENTE en NAND (solo ‹nand›, sin ¬ ni ∨ ni ∧), y demostrada.›
theorem libertad_es_beatitud_nand:
  "ou (y Adecuacion Beatitud) (y (no Adecuacion) (no Beatitud))"
  unfolding nand_def
  using A_adecuacion_actividad A_virtud_actividad A_beatitud_virtud
  by blast

text ‹Y la cadena de la potencia —conato = eficacia— también en NAND:›
theorem conato_es_potencia_nand:
  "ou (y Conato Eficaz) (y (no Conato) (no Eficaz))"
  using A_conato_es_potencia unfolding nand_def by blast

text ‹La fórmula gigante: el bicondicional «adecuación ⟷ beatitud» expandido
  íntegro a pura NAND (23 nand, sin ninguna otra conectiva), y demostrado igual al
  bicondicional.›
lemma gigante:
  "nand (nand (nand (nand Adecuacion Beatitud) (nand Adecuacion Beatitud))
              (nand (nand Adecuacion Beatitud) (nand Adecuacion Beatitud)))
        (nand (nand (nand (nand Adecuacion Adecuacion) (nand Beatitud Beatitud))
                    (nand (nand Adecuacion Adecuacion) (nand Beatitud Beatitud)))
```

```
              (nand (nand (nand Adecuacion Adecuacion) (nand Beatitud Beatitud))
                    (nand (nand Adecuacion Adecuacion) (nand Beatitud Beatitud))))
   = (Adecuacion ⟷ Beatitud)"
  by (auto simp add: nand_def)

end
```

Spinoza, Ética I

La reconstrucción de la Parte I de la Ética (definiciones, axiomas y proposiciones): cada axioma es una premisa con nombre, y cada premisa que Spinoza no enuncia pero necesita, otra (H_…).

Fichero `Spinoza_Etica_I.thy` ▪ 77 resultados demostrados ▪ 1299 líneas

```
(*  Spinoza, Ethica more geometrico demonstrata, Parte I (def. 1-8, ax. 1-7, prop. 1-19, 21-24, 26, 28,
29, 32 y 36)
    Reconstrucción en Isabelle/HOL.

    Idea: cada axioma de Spinoza es una constante booleana (A1, A2...) y cada
    premisa que Spinoza NO enuncia pero necesita es otra (H_...). Así cada
    teorema muestra exactamente de qué depende, y Nitpick (buscador de
    contramodelos) enseña qué pasa si se quita la premisa oculta.
*)

theory Spinoza_Etica_I
  imports Main
begin

section ‹Vocabulario primitivo›

typedecl cosa
typedecl atributo

consts
  En      :: "cosa ⇒ cosa ⇒ bool"      — ‹En x y: x es en y (inest)›
  SeConcibePor :: "cosa ⇒ cosa ⇒ bool"      — ‹SeConcibePor x y: x se concibe por y›
  Causa   :: "cosa ⇒ cosa ⇒ bool"      — ‹Causa x y: x es causa de y›
  AlgoEnComun   :: "cosa ⇒ cosa ⇒ bool"      — ‹AlgoEnComun x y: x e y tienen algo en común›
  TieneAtributo   :: "cosa ⇒ atributo ⇒ bool"  — ‹TieneAtributo s a: a es atributo de s›


section ‹Definiciones›

text ‹Def. 3: «Por sustancia entiendo aquello que es en sí y se concibe por sí, esto es,
  aquello cuyo concepto no necesita del concepto de otra cosa». La cláusula final
  (no se concibe por ninguna otra cosa) es la que hace casi todo el trabajo.
  Versión endurecida: tampoco es en ninguna otra cosa. Sin esta cláusula, Nitpick
  construía sustancias que, además de ser en sí, eran en otra.›

definition sustancia :: "cosa ⇒ bool" where
  "sustancia x ≡ En x x ∧ SeConcibePor x x ∧ (∀y. En x y ⟶ y = x)
                  ∧ (∀y. SeConcibePor x y ⟶ y = x)"

text ‹Def. 5: «Por modo entiendo las afecciones de la sustancia, o sea, aquello que es
  en otra cosa, por medio de la cual también se concibe».›

definition modo :: "cosa ⇒ bool" where
  "modo x ≡ ∃y. y ≠ x ∧ En x y ∧ SeConcibePor x y"

definition afeccion :: "cosa ⇒ cosa ⇒ bool" where
  "afeccion m s ≡ m ≠ s ∧ En m s ∧ SeConcibePor m s"

text ‹«Anterior por naturaleza»: y se concibe por x, pero x no por y.›

definition anterior_por_naturaleza :: "cosa ⇒ cosa ⇒ bool" where
  "anterior_por_naturaleza x y ≡ SeConcibePor y x ∧ ¬ SeConcibePor x y"


section ‹Axiomas de Spinoza›

definition A1 :: bool where   — ‹Todo lo que es, es en sí o en otra cosa.›
  "A1 ≡ ∀x. En x x ∨ (∃y. y ≠ x ∧ En x y)"

definition A2 :: bool where   — ‹Lo que no puede concebirse por otra cosa, debe concebirse por sí.›
  "A2 ≡ ∀x. ¬ (∃y. y ≠ x ∧ SeConcibePor x y) ⟶ SeConcibePor x x"
```

```
definition A4 :: bool where   — ‹El conocimiento del efecto depende del de la causa y lo implica.›
  "A4 ≡ ∀x y. Causa x y ⟶ SeConcibePor y x"

definition A5 :: bool where   — ‹Las cosas que no tienen nada en común no se entienden una por otra.›
  "A5 ≡ ∀x y. ¬ AlgoEnComun x y ⟶ ¬ SeConcibePor x y ∧ ¬ SeConcibePor y x"


section ‹Premisas que Spinoza no enuncia›

text ‹H_conversa_A5: la conversa del axioma 5. Si dos cosas tienen algo en común,
  una se concibe por la otra. Spinoza la usa en la prop. 2 sin decirlo.›

definition H_conversa_A5 :: bool where
  "H_conversa_A5 ≡ ∀x y. AlgoEnComun x y ⟶ SeConcibePor x y ∨ SeConcibePor y x"

text ‹H_quien_comparte_uno_comparte_todos: si dos sustancias comparten un atributo, comparten todos.
  Es lo que Spinoza da por concedido en la prop. 5 y lo que Leibniz le objetó:
  ¿por qué no dos sustancias con un atributo común y otro distinto?›

definition H_quien_comparte_uno_comparte_todos :: bool where
  "H_quien_comparte_uno_comparte_todos ≡ ∀s t a. TieneAtributo s a ∧ TieneAtributo t a ⟶ (∀b.
TieneAtributo s b ⟷ TieneAtributo t b)"

text ‹H_individuacion_por_atributos: lo que la prop. 4 más la prop. 1 le dan a Spinoza en la prop. 5.
  Dejadas aparte las afecciones, dos sustancias solo se distinguen por sus atributos.›

definition H_individuacion_por_atributos :: bool where
  "H_individuacion_por_atributos ≡ ∀s t. sustancia s ∧ sustancia t ∧ (∀b. TieneAtributo s b ⟷
TieneAtributo t b) ⟶ s = t"

text ‹H_ser_en_igual_concebirse: «ser en» y «concebirse por» son la misma relación.›

definition H_ser_en_igual_concebirse :: bool where
  "H_ser_en_igual_concebirse ≡ ∀x y. En x y ⟷ SeConcibePor x y"

text ‹Versión debilitada, solo para sustancias (se usa desde que la fuerte chocó con
  las partes y las causas de los modos; ver «Las partes de los modos»). Lo que es en
  otra cosa se concibe por ella, para todo; pero «concebirse por ⟹ ser en» solo se
  exige a lo que no es en nada más que en sí mismo. Un modo puede así concebirse por
  sus causas y sus partes sin ser en ellas.›

definition H_ser_en_implica_concebirse :: bool where
  "H_ser_en_implica_concebirse ≡ ∀x y. En x y ⟶ SeConcibePor x y"

definition H_solo_en_si_solo_por_si :: bool where
  "H_solo_en_si_solo_por_si ≡ ∀x. (∀y. En x y ⟶ y = x) ⟶ (∀y. SeConcibePor x y ⟶ En x y)"

lemma H_en_concibe_implica_debiles:
  "H_ser_en_igual_concebirse ⟹ H_ser_en_implica_concebirse ∧ H_solo_en_si_solo_por_si"
  unfolding H_ser_en_igual_concebirse_def H_ser_en_implica_concebirse_def H_solo_en_si_solo_por_si_def by
blast

definition H_atributo_compartido_es_comun :: bool where   — ‹Compartir un atributo es tener algo en
común.›
  "H_atributo_compartido_es_comun ≡ ∀s t a. TieneAtributo s a ∧ TieneAtributo t a ⟶ AlgoEnComun s t"


section ‹Definición negativa de sustancia›

text ‹Lectura alternativa de la def. 3, más cercana al «esto es» de Spinoza:
  sustancia es lo que no es en otra cosa ni se concibe por otra cosa.›

definition sustancia_negativa :: "cosa ⇒ bool" where
  "sustancia_negativa x ≡ ¬ (∃y. y ≠ x ∧ En x y) ∧ ¬ (∃y. y ≠ x ∧ SeConcibePor x y)"

text ‹Con los ax. 1 y 2, las dos definiciones son equivalentes: el ax. 1 da el «es en
  sí» a lo que no es en otro, y el ax. 2 da el «se concibe por sí» a lo que no se
```

```
  concibe por otro.›

theorem sustancia_equiv_neg:
  assumes "A1" "A2"
  shows "sustancia x ⟷ sustancia_negativa x"
  using assms unfolding A1_def A2_def sustancia_def sustancia_negativa_def by blast

text ‹Sin los axiomas, la negativa es más débil: valdría también para algo que no es
  en nada ni se concibe por nada.›

lemma sustancia_neg_sin_axiomas:
  "sustancia_negativa x ⟹ sustancia x"
  nitpick [show_consts]
  oops


section ‹Todo es sustancia o modo›

text ‹Spinoza lo da por sentado (prop. 4, dem.: «nada hay fuera del entendimiento
  excepto las sustancias y sus afecciones»). Con A1 y A2 solos no sale:›

lemma todo_sustancia_o_modo_sin_premisa:
  "A1 ⟹ A2 ⟹ sustancia x ∨ modo x"
  nitpick [show_consts]
  oops

theorem todo_sustancia_o_modo:
  assumes "A1" "H_ser_en_implica_concebirse" "H_solo_en_si_solo_por_si"
  shows "sustancia x ∨ modo x"
proof (cases "∃y. y ≠ x ∧ En x y")
  case True
  then obtain y where "y ≠ x" "En x y" by blast
  moreover have "SeConcibePor x y"
    using ‹En x y› ‹H_ser_en_implica_concebirse› unfolding H_ser_en_implica_concebirse_def by blast
  ultimately show ?thesis unfolding modo_def by blast
next
  case False
  then have solo: "∀y. En x y ⟶ y = x" by blast
  have "En x x" using ‹A1› False unfolding A1_def by blast
  moreover have "SeConcibePor x x"
    using ‹En x x› ‹H_ser_en_implica_concebirse› unfolding H_ser_en_implica_concebirse_def by blast
  moreover have "∀y. SeConcibePor x y ⟶ y = x"
    using solo ‹H_solo_en_si_solo_por_si› unfolding H_solo_en_si_solo_por_si_def by blast
  ultimately show ?thesis unfolding sustancia_def using solo by blast
qed


section ‹Proposiciones 1-6›

text ‹Prop. 1: La sustancia es anterior_por_naturaleza por naturaleza a sus afecciones.
  Sale solo de las definiciones.›

theorem P1:
  assumes "sustancia s" and "afeccion m s"
  shows "anterior_por_naturaleza s m"
  using assms unfolding sustancia_def afeccion_def anterior_por_naturaleza_def by blast

text ‹Prop. 2: Dos sustancias que tienen atributos distintos no tienen nada en común.
  Spinoza: «es evidente por la def. 3». Con los axiomas solos, no:›

lemma P2_sin_premisa:
  "A1 ⟹ A2 ⟹ A4 ⟹ A5 ⟹ sustancia s ⟹ sustancia t ⟹ s ≠ t ⟹ ¬ AlgoEnComun s t"
  nitpick [show_consts]
  oops

text ‹Con la premisa oculta sí. Y nótese que la hipótesis «atributos distintos»
  sobra: basta con que sean dos sustancias distintas.›
```

```
theorem P2:
  assumes "H_conversa_A5" "sustancia s" "sustancia t" "s ≠ t"
  shows "¬ AlgoEnComun s t"
  using assms unfolding H_conversa_A5_def sustancia_def by blast

text ‹Prop. 3: Si dos cosas no tienen nada en común, una no puede ser causa de la otra.
  Esta sí sale limpia de los axiomas 4 y 5, tal como dice Spinoza.›

theorem P3:
  assumes "A4" "A5" "¬ AlgoEnComun x y"
  shows "¬ Causa x y ∧ ¬ Causa y x"
  using assms unfolding A4_def A5_def by blast

text ‹Prop. 5: No puede haber dos o más sustancias con la misma naturaleza o atributo.
  Contramodelo de Leibniz: s con atributos {a, b} y t con {a, c}.›

lemma P5_sin_premisa:
  "A1 ⟹ A2 ⟹ A4 ⟹ A5 ⟹ H_individuacion_por_atributos ⟹ sustancia s ⟹ sustancia t ⟹
TieneAtributo s a ⟹ TieneAtributo t a ⟹ s = t"
  nitpick [show_consts]
  oops

theorem P5:
  assumes "H_individuacion_por_atributos" "H_quien_comparte_uno_comparte_todos" "sustancia s" "sustancia
t" "TieneAtributo s a" "TieneAtributo t a"
  shows "s = t"
  using assms unfolding H_individuacion_por_atributos_def H_quien_comparte_uno_comparte_todos_def by blast

text ‹Un camino que Spinoza no toma: con H_conversa_A5 la prop. 5 sale de la prop. 2,
  sin necesitar la premisa que objetó Leibniz.›

theorem P5_por_P2:
  assumes "H_conversa_A5" "H_atributo_compartido_es_comun" "sustancia s" "sustancia t" "TieneAtributo s a"
"TieneAtributo t a"
  shows "s = t"
  using assms P2 [of s t] unfolding H_atributo_compartido_es_comun_def by blast

text ‹Prop. 6: Una sustancia no puede ser producida por otra sustancia.
  Prueba de Spinoza: props. 5, 2 y 3.›

theorem P6:
  assumes "H_conversa_A5" "A4" "A5" "sustancia s" "sustancia t" "s ≠ t"
  shows "¬ Causa s t"
  using assms P2 [of s t] P3 [of s t] by blast

text ‹Corolario, prueba alternativa de Spinoza («se demuestra más fácilmente por el
  absurdo»): por el ax. 4 y la def. 3. No necesita ninguna premisa oculta.›

theorem P6_aliter:
  assumes "A4" "sustancia s" "sustancia t" "s ≠ t"
  shows "¬ Causa s t"
  using assms unfolding A4_def sustancia_def by blast

section ‹Proposiciones 7-11: vocabulario y definiciones nuevas›

consts
  EsenciaImplicaExistencia     :: "cosa ⇒ bool"            — ‹su esencia implica la existencia›
  ConcebibleComoNoExistente :: "cosa ⇒ bool"            — ‹puede concebirse como no existente›
  Existe                :: "cosa ⇒ bool"            — ‹existe fuera del concepto›
  Limita                :: "cosa ⇒ cosa ⇒ bool"     — ‹Limita y x: y limita a x›
  AtributoSeConcibePor            :: "atributo ⇒ atributo ⇒ bool"  — ‹el atributo a se concibe por b›
  MasRealidad           :: "cosa ⇒ cosa ⇒ bool"     — ‹x tiene más realidad o ser que y›
  Dios                  :: cosa
  Voluntad              :: cosa

text ‹Def. 1: «Por causa de sí entiendo aquello cuya esencia implica la existencia».›

definition D1 :: bool where
```

```
  "D1 ≡ ∀x. Causa x x ⟷ EsenciaImplicaExistencia x"

text ‹Def. 2: «Se dice finita en su género aquella cosa que puede ser limitada por otra
  de la misma naturaleza». «Misma naturaleza» se lee como compartir un atributo.›

definition finita :: "cosa ⇒ bool" where
  "finita x ≡ ∃y. y ≠ x ∧ Limita y x ∧ (∃a. TieneAtributo y a ∧ TieneAtributo x a)"

text ‹Def. 4: el atributo es lo que el entendimiento percibe de una sustancia como
  constitutivo de su esencia. Aquí: solo las sustancias tienen atributos.›

definition D4 :: bool where
  "D4 ≡ ∀x a. TieneAtributo x a ⟶ sustancia x"

text ‹Def. 6: Dios es un ente absolutamente infinito, una sustancia que consta de infinitos
  atributos. En un modelo finito, «infinitos» se lee como «todos».›

definition D6 :: bool where
  "D6 ≡ sustancia Dios ∧ (∀a. TieneAtributo Dios a)"

text ‹Ax. 7: «Si una cosa puede concebirse como no existente, su esencia no implica
  la existencia».›

definition A7 :: bool where
  "A7 ≡ ∀x. ConcebibleComoNoExistente x ⟶ ¬ EsenciaImplicaExistencia x"

text ‹H_todo_tiene_causa: todo tiene una causa, sea otra cosa o ella misma. Spinoza la usa en la
  prop. 7 sin enunciarla; la formula después, en la 2.ª prueba de la prop. 11.›

definition H_todo_tiene_causa :: bool where
  "H_todo_tiene_causa ≡ ∀x. ∃y. Causa y x"


section ‹Proposiciones 7-11›

text ‹Prop. 6, corolario: una sustancia no puede ser producida por nada distinto de
  ella (no solo por otra sustancia). Con la def. 3 endurecida basta el ax. 4.›

theorem P6_cor:
  assumes "A4" "sustancia s" "y ≠ s"
  shows "¬ Causa y s"
  using assms unfolding A4_def sustancia_def by blast

text ‹Prop. 7: A la naturaleza de la sustancia pertenece existir.
  Spinoza: no la produce otra cosa (cor. prop. 6), «luego será causa de sí». Ese «luego»
  supone que todo tiene causa. Sin H_todo_tiene_causa, una sustancia sin causa alguna:›

lemma P7_sin_premisa:
  "A4 ⟹ D1 ⟹ sustancia s ⟹ EsenciaImplicaExistencia s"
  nitpick [show_consts]
  oops

theorem P7:
  assumes "A4" "D1" "H_todo_tiene_causa" "sustancia s"
  shows "EsenciaImplicaExistencia s"
proof -
  obtain y where c: "Causa y s" using ‹H_todo_tiene_causa› unfolding H_todo_tiene_causa_def by blast
  then have "y = s" using P6_cor [of s y] ‹A4› ‹sustancia s› by blast
  then show ?thesis using c ‹D1› unfolding D1_def by blast
qed

text ‹Prop. 8: Toda sustancia es necesariamente infinita.
  Spinoza: si fuera finita, la limitaría otra de la misma naturaleza, que también
  existiría (prop. 7), y habría dos sustancias del mismo atributo, contra la prop. 5.
  Aquí la prop. 7 no hace falta: en esta lógica, cuantificar sobre «otra cosa» ya
  supone que existe. Lo que sí hace falta es todo lo que necesitaba la prop. 5.›

theorem P8:
```

```
  assumes "D4" "H_individuacion_por_atributos" "H_quien_comparte_uno_comparte_todos" "sustancia s"
  shows "¬ finita s"
proof
  assume "finita s"
  then obtain y a where "y ≠ s" "TieneAtributo y a" "TieneAtributo s a" unfolding finita_def by blast
  moreover have "sustancia y" using ‹D4› ‹TieneAtributo y a› unfolding D4_def by blast
  ultimately show False using P5 [of y s a] assms by blast
qed

text ‹Prop. 9: Cuanta más realidad o ser tiene una cosa, tantos más atributos le
  competen. Spinoza: «es evidente por la def. 4». Pero la def. 4 no habla de grados
  de realidad:›

lemma P9_sin_premisa:
  "D4 ⟹ MasRealidad x y ⟹ {a. TieneAtributo y a} ⊂ {a. TieneAtributo x a}"
  nitpick [show_consts]
  oops

text ‹Para que salga hay que identificar grado de realidad con cantidad de atributos,
  y entonces la prop. 9 es esa misma premisa.›

definition H_realidad_es_atributos :: bool where
  "H_realidad_es_atributos ≡ ∀x y. MasRealidad x y ⟶ {a. TieneAtributo y a} ⊂ {a. TieneAtributo x a}"

theorem P9:
  assumes "H_realidad_es_atributos" "MasRealidad x y"
  shows "{a. TieneAtributo y a} ⊂ {a. TieneAtributo x a}"
  using assms unfolding H_realidad_es_atributos_def by blast

text ‹Prop. 10: Cada atributo de una sustancia debe concebirse por sí.
  Spinoza: por la def. 4 y la def. 3. Hace falta un puente entre atributos y
  sustancias: si un atributo se concibe por otro, su sustancia se concibe por la
  sustancia del otro.›

definition H_puente_atributo_sustancia :: bool where
  "H_puente_atributo_sustancia ≡ ∀s t a b. TieneAtributo s a ∧ TieneAtributo t b ∧ AtributoSeConcibePor a
b ⟶ SeConcibePor s t"

text ‹Lo que sí sale: un atributo solo puede concebirse por atributos de su misma
  sustancia.›

theorem P10_debil:
  assumes "H_puente_atributo_sustancia" "D4" "TieneAtributo s a" "TieneAtributo t b" "AtributoSeConcibePor
a b"
  shows "s = t"
  using assms unfolding H_puente_atributo_sustancia_def D4_def sustancia_def by blast

text ‹Lo que no sale: que un atributo no se conciba por otro de la misma sustancia
  (el escolio: atributos «realmente distintos»). Dios con dos atributos, uno
  concebido por el otro:›

lemma P10_sin_premisa:
  "H_puente_atributo_sustancia ⟹ D4 ⟹ D6 ⟹ TieneAtributo s a ⟹ AtributoSeConcibePor a b ⟹ a = b"
  nitpick [show_consts]
  oops

text ‹Prop. 11: Dios, o sea, una sustancia que consta de infinitos atributos, existe
  necesariamente. 1.ª prueba: si se niega, concíbase a Dios como no existente; por el
  ax. 7 su esencia no implicaría la existencia, contra la prop. 7.›

theorem P11:
  assumes "A4" "D1" "H_todo_tiene_causa" "A7" "D6"
  shows "¬ ConcebibleComoNoExistente Dios"
proof
  assume "ConcebibleComoNoExistente Dios"
  then have "¬ EsenciaImplicaExistencia Dios" using ‹A7› unfolding A7_def by blast
  moreover have "EsenciaImplicaExistencia Dios"
    using P7 [of Dios] assms unfolding D6_def by blast
```

```
  ultimately show False by blast
qed

text ‹La prueba da que Dios no puede concebirse como no existente. Que exista fuera
  del concepto no se sigue de nada de lo anterior_por_naturaleza:›

lemma P11_existencia_real:
  "A4 ⟹ D1 ⟹ H_todo_tiene_causa ⟹ A7 ⟹ D6 ⟹ Existe Dios"
  nitpick [show_consts]
  oops

text ‹Además, en HOL la constante Dios denota algo del dominio desde que se declara:
  la existencia del sujeto ya está puesta en el lenguaje antes de la prueba. Es el
  viejo reproche al argumento ontológico. Spinoza cerraría el paso con el ax. 6 (la
  idea verdadera debe convenir con su objeto), y ese es el punto que discutió Kant.›

section ‹Proposiciones 12-15›

consts
  EsParteDe :: "cosa ⇒ cosa ⇒ bool"   — ‹EsParteDe p x: p es parte de x›

definition divisible :: "cosa ⇒ bool" where
  "divisible x ≡ ∃p. p ≠ x ∧ EsParteDe p x"

text ‹Spinoza, en el primer cuerno del dilema de la prop. 12: el todo «podría ser y
  concebirse sin sus partes, lo cual nadie dudará que es absurdo». No es una premisa
  oculta: la enuncia, pero como evidente.›

definition H_todo_por_sus_partes :: bool where
  "H_todo_por_sus_partes ≡ ∀p x. EsParteDe p x ⟶ SeConcibePor x p"

text ‹Prop. 12: No puede concebirse con verdad ningún atributo de la sustancia del que
  se siga que la sustancia puede dividirse. Spinoza la prueba con un dilema (las partes
  conservan o no la naturaleza de sustancia) que recorre las props. 2, 5, 6, 7, 8 y 10.
  Aquí basta la premisa anterior_por_naturaleza más la def. 3: lo que se concibe por sus partes no se
  concibe solo por sí.›

lemma P12_sin_premisa:
  "A1 ⟹ A2 ⟹ A4 ⟹ A5 ⟹ sustancia s ⟹ ¬ divisible s"
  nitpick [show_consts]
  oops

theorem P12:
  assumes "H_todo_por_sus_partes" "sustancia s"
  shows "¬ divisible s"
  using assms unfolding H_todo_por_sus_partes_def sustancia_def divisible_def by blast

text ‹Prop. 13: La sustancia absolutamente infinita es indivisible.›

theorem P13:
  assumes "H_todo_por_sus_partes" "D6"
  shows "¬ divisible Dios"
  using assms P12 [of Dios] unfolding D6_def by blast

text ‹Prop. 14: Fuera de Dios no puede darse ni concebirse ninguna sustancia.
  Spinoza: Dios tiene todos los atributos (def. 6); otra sustancia tendría que
  explicarse por alguno de ellos y habría dos sustancias del mismo atributo, contra la
  prop. 5. Ese «tendría que explicarse por alguno» supone que toda sustancia tiene al
  menos un atributo. Sin esa premisa, una sustancia sin atributos convive con Dios:›

definition H_sustancia_con_atributo :: bool where
  "H_sustancia_con_atributo ≡ ∀s. sustancia s ⟶ (∃a. TieneAtributo s a)"

lemma P14_sin_premisa:
  "D4 ⟹ D6 ⟹ H_individuacion_por_atributos ⟹ H_quien_comparte_uno_comparte_todos ⟹ sustancia s ⟹
s = Dios"
  nitpick [show_consts]
  oops
```

```
theorem P14:
  assumes "D6" "H_sustancia_con_atributo" "H_individuacion_por_atributos"
"H_quien_comparte_uno_comparte_todos" "sustancia s"
  shows "s = Dios"
proof -
  obtain a where "TieneAtributo s a" using ‹H_sustancia_con_atributo› ‹sustancia s› unfolding
H_sustancia_con_atributo_def by blast
  moreover have "sustancia Dios" "TieneAtributo Dios a" using ‹D6› unfolding D6_def by blast+
  ultimately show ?thesis using P5 [of s Dios a] assms by blast
qed

text ‹Corolario 1: Dios es único. No necesita H_sustancia_con_atributo: en HOL todo tipo tiene
  al menos un elemento, así que dos sustancias con todos los atributos comparten uno.
  (Ese «al menos un atributo» lo pone el lenguaje, no Spinoza.)›

theorem P14_cor1:
  assumes "D6" "H_individuacion_por_atributos" "H_quien_comparte_uno_comparte_todos" "sustancia x" "∀a.
TieneAtributo x a"
  shows "x = Dios"
proof -
  have "sustancia Dios" "TieneAtributo Dios undefined" using ‹D6› unfolding D6_def by blast+
  moreover have "TieneAtributo x undefined" using assms by blast
  ultimately show ?thesis using P5 [of x Dios undefined] assms by blast
qed

text ‹Prop. 15: Todo lo que es, es en Dios, y sin Dios nada puede ser ni concebirse.
  Spinoza: fuera de Dios no hay sustancias (prop. 14); los modos no pueden ser ni
  concebirse sin la sustancia (def. 5); y fuera de sustancias y modos no hay nada
  (ax. 1). Pero la def. 5 solo dice que el modo es «en otra cosa», que podría ser otro
  modo. Sin más, un modo en otro modo queda fuera de Dios:›

definition H_modo_en_sustancia :: bool where
  "H_modo_en_sustancia ≡ ∀x y. modo x ∧ y ≠ x ∧ En x y ⟶ sustancia y"

lemma P15_sin_premisa:
  "A1 ⟹ H_ser_en_implica_concebirse ⟹ H_solo_en_si_solo_por_si ⟹ D6 ⟹ H_sustancia_con_atributo ⟹
H_individuacion_por_atributos ⟹ H_quien_comparte_uno_comparte_todos ⟹ En x Dios"
  nitpick [show_consts]
  oops

theorem P15:
  assumes "A1" "H_ser_en_implica_concebirse" "H_solo_en_si_solo_por_si" "H_modo_en_sustancia" "D6"
"H_sustancia_con_atributo" "H_individuacion_por_atributos" "H_quien_comparte_uno_comparte_todos"
  shows "En x Dios ∧ SeConcibePor x Dios"
proof -
  have "sustancia x ∨ modo x" using todo_sustancia_o_modo ‹A1› ‹H_ser_en_implica_concebirse›
‹H_solo_en_si_solo_por_si› by blast
  then have "En x Dios"
  proof
    assume "sustancia x"
    then have "x = Dios" using P14 assms by blast
    then show "En x Dios" using ‹sustancia x› unfolding sustancia_def by blast
  next
    assume m: "modo x"
    then obtain y where "y ≠ x" "En x y" unfolding modo_def by blast
    then have "sustancia y" using m ‹H_modo_en_sustancia› unfolding H_modo_en_sustancia_def by blast
    then have "y = Dios" using P14 assms by blast
    then show "En x Dios" using ‹En x y› by blast
  qed
  then show ?thesis using ‹H_ser_en_implica_concebirse› unfolding H_ser_en_implica_concebirse_def by blast
qed


section ‹Las partes de los modos›

text ‹El primer modelo de compatibilidad con modos divisibles hacía de Dios una parte
  de un modo. Spinoza no lo admitiría: las partes de un modo son modos.›
```

```
definition H_partes_de_modos :: bool where
  "H_partes_de_modos ≡ ∀p x. modo x ∧ EsParteDe p x ∧ p ≠ x ⟶ modo p"

text ‹Ninguna cosa es a la vez sustancia y modo.›

lemma sustancia_no_modo:
  assumes "sustancia x"
  shows "¬ modo x"
  using assms unfolding sustancia_def modo_def by blast

text ‹Pero con esta premisa, junto a «ser en = concebirse por» y «un modo solo es en
  sustancias», ningún modo puede tener partes. Una parte p del modo m: m se concibe
  por p (el todo por sus partes), luego es en p, luego p es sustancia; y a la vez p es
  modo. Los cuerpos quedarían sin partes, contra lo que Spinoza dice de ellos en la
  Parte II.›

theorem modos_indivisibles:
  assumes "H_todo_por_sus_partes" "H_ser_en_igual_concebirse" "H_modo_en_sustancia" "H_partes_de_modos"
"modo m"
  shows "¬ divisible m"
proof
  assume "divisible m"
  then obtain p where "p ≠ m" "EsParteDe p m" unfolding divisible_def by blast
  then have "En m p"
    using ‹H_todo_por_sus_partes› ‹H_ser_en_igual_concebirse› unfolding H_todo_por_sus_partes_def
H_ser_en_igual_concebirse_def by blast
  then have "sustancia p"
    using ‹H_modo_en_sustancia› ‹modo m› ‹p ≠ m› unfolding H_modo_en_sustancia_def by blast
  moreover have "modo p"
    using ‹H_partes_de_modos› ‹modo m› ‹EsParteDe p m› ‹p ≠ m› unfolding H_partes_de_modos_def by blast
  ultimately show False using sustancia_no_modo by blast
qed

text ‹El mismo mecanismo choca con la causalidad entre modos: por el ax. 4 el efecto se
  concibe por su causa, luego es en ella, luego la causa es sustancia, luego es Dios.
  Ningún modo podría causar otro modo, contra la prop. 28 (cada cosa finita es causada
  por otra cosa finita).›

theorem modos_causados_solo_por_Dios:
  assumes "A4" "H_ser_en_igual_concebirse" "H_modo_en_sustancia" "D6" "H_sustancia_con_atributo"
"H_individuacion_por_atributos" "H_quien_comparte_uno_comparte_todos"
      "modo m" "Causa y m" "y ≠ m"
  shows "y = Dios"
proof -
  have "En m y"
    using ‹A4› ‹H_ser_en_igual_concebirse› ‹Causa y m› unfolding A4_def H_ser_en_igual_concebirse_def by
blast
  then have "sustancia y"
    using ‹H_modo_en_sustancia› ‹modo m› ‹y ≠ m› unfolding H_modo_en_sustancia_def by blast
  then show ?thesis using P14 assms by blast
qed

text ‹El culpable es H_ser_en_igual_concebirse: para las sustancias, «ser en» y «concebirse por»
  coinciden; para los modos, no. Un modo es en Dios, pero se concibe también por sus
  causas y por sus partes, que son modos.›

text ‹Por eso, desde aquí la teoría usa la versión debilitada (H_ser_en_implica_concebirse y
  H_solo_en_si_solo_por_si). Los dos teoremas anteriores siguen siendo válidos, pero solo
  para la premisa fuerte, que ya no se usa.›


section ‹«En sí o en otro», «por sí o por otro»: ¿excluyentes?›

text ‹El ax. 1 dice que todo es en sí o en otra cosa, pero no que no pueda ser las dos.
  En el modelo con modos divisibles, todo era «en sí» además de ser en Dios.
  Lectura excluyente:›
```

```
definition A1_excluyente :: bool where
  "A1_excluyente ≡ ∀x. (En x x ∧ ¬ (∃y. y ≠ x ∧ En x y)) ∨ (¬ En x x ∧ (∃y. y ≠ x ∧ En x y))"

lemma A1_excl_implica_A1: "A1_excluyente ⟹ A1"
  unfolding A1_excluyente_def A1_def by blast

text ‹Lo mismo con la concepción: lo que se concibe por otra cosa (def. 5) no se
  concibe por sí (def. 3). Spinoza contrapone «per se» y «per aliud» como si se
  excluyeran, pero no lo dice.›

definition H_por_si_excluye_por_otro :: bool where
  "H_por_si_excluye_por_otro ≡ ∀x. (∃y. y ≠ x ∧ SeConcibePor x y) ⟶ ¬ SeConcibePor x x"

text ‹Prop. 24: La esencia de las cosas producidas por Dios no implica la existencia.
  Spinoza: «es evidente por la def. 1», porque lo que la implica es causa de sí.
  Pero en el modelo anterior_por_naturaleza había un modo causa de sí. Sin premisas, sale de nuevo:›

lemma P24_sin_premisa:
  "A4 ⟹ D1 ⟹ modo m ⟹ ¬ EsenciaImplicaExistencia m"
  nitpick [show_consts]
  oops

text ‹La lectura excluyente del ax. 1 no basta: un modo que no es en sí, pero que se
  concibe a la vez por sí y por Dios, puede seguir siendo causa de sí.›

lemma P24_solo_A1_excl:
  "A1_excluyente ⟹ A4 ⟹ D1 ⟹ H_ser_en_implica_concebirse ⟹ modo m ⟹ ¬ EsenciaImplicaExistencia
m"
  nitpick [show_consts]
  oops

text ‹Lo que hace falta es la exclusividad de la concepción: si su esencia implicara
  la existencia, sería causa de sí (def. 1), luego se concebiría por sí (ax. 4); pero
  como modo se concibe por otra cosa.›

theorem P24:
  assumes "A4" "D1" "H_por_si_excluye_por_otro" "modo m"
  shows "¬ EsenciaImplicaExistencia m ∧ ¬ Causa m m"
proof -
  obtain y where "y ≠ m" "SeConcibePor m y" using ‹modo m› unfolding modo_def by blast
  then have no: "¬ SeConcibePor m m" using ‹H_por_si_excluye_por_otro› unfolding
H_por_si_excluye_por_otro_def by blast
  then have "¬ Causa m m" using ‹A4› unfolding A4_def by blast
  then show ?thesis using ‹D1› unfolding D1_def by blast
qed


section ‹Proposiciones 16-36: Dios como causa (primera tanda)›

text ‹Def. 7: «Se dice libre la cosa que existe por la sola necesidad de su naturaleza
  y es determinada a obrar por sí sola». Leemos «ser determinado por» como «ser causado
  por».›

definition libre :: "cosa ⇒ bool" where
  "libre x ≡ EsenciaImplicaExistencia x ∧ (∀y. Causa y x ⟶ y = x)"

text ‹Def. 8: «Por eternidad entiendo la existencia misma, en cuanto se concibe que se
  sigue necesariamente de la sola definición de la cosa eterna».›

definition eterno :: "cosa ⇒ bool" where
  "eterno x ≡ EsenciaImplicaExistencia x"

text ‹Un atributo es eterno si lo es la sustancia cuya esencia expresa.›

definition atributo_eterno :: "atributo ⇒ bool" where
  "atributo_eterno a ≡ ∀s. TieneAtributo s a ⟶ eterno s"

text ‹Causa inmanente: el efecto es en la causa. Causa transitiva: el efecto no es en ella.›
```

```
definition causa_inmanente :: "cosa ⇒ cosa ⇒ bool" where
  "causa_inmanente y x ≡ Causa y x ∧ En x y"

definition causa_transitiva :: "cosa ⇒ cosa ⇒ bool" where
  "causa_transitiva y x ≡ Causa y x ∧ ¬ En x y"

text ‹Prop. 33, esc. 1: una cosa se dice necesaria por su esencia o por su causa;
  contingente, si no lo es por ninguna de las dos.›

definition contingente :: "cosa ⇒ bool" where
  "contingente x ≡ ¬ EsenciaImplicaExistencia x ∧ ¬ (∃y. Causa y x)"

text ‹Las premisas de la prop. 15, agrupadas para no repetirlas.›

definition premisas_P15 :: bool where
  "premisas_P15 ≡ A1 ∧ H_ser_en_implica_concebirse ∧ H_solo_en_si_solo_por_si
     ∧ H_modo_en_sustancia ∧ D6 ∧ H_sustancia_con_atributo
     ∧ H_individuacion_por_atributos ∧ H_quien_comparte_uno_comparte_todos"

lemma P15_por_premisas: "premisas_P15 ⟹ En x Dios ∧ SeConcibePor x Dios"
  unfolding premisas_P15_def using P15 by blast

text ‹H_inherencia_es_causalidad: si una cosa es en otra, esa otra es su causa.
  Spinoza pasa de «todo es en Dios» (prop. 15) a «Dios es causa de todo» (prop. 16,
  cor. 1; prop. 18) sin enunciarla.›

definition H_inherencia_es_causalidad :: bool where
  "H_inherencia_es_causalidad ≡ ∀x y. En x y ∧ y ≠ x ⟶ Causa y x"

text ‹Prop. 16: De la necesidad de la naturaleza divina deben seguirse infinitas cosas
  de infinitos modos, esto es, todo lo que puede caer bajo un entendimiento infinito.
  Cor. 1: Dios es causa eficiente de todas las cosas.
  En este lenguaje solo hay cosas que existen, así que «todo lo que puede caer bajo un
  entendimiento infinito» se reduce a «todo lo que hay». Formalizamos el corolario 1.
  Sin H_inherencia_es_causalidad no sale: un modo en Dios causado solo por otro modo.›

lemma P16_cor1_sin_premisa:
  "premisas_P15 ⟹ A4 ⟹ D1 ⟹ H_todo_tiene_causa ⟹ Causa Dios x"
  nitpick [show_consts]
  oops

theorem P16_cor1:
  assumes "premisas_P15" "A4" "H_todo_tiene_causa" "H_inherencia_es_causalidad"
  shows "Causa Dios x"
proof (cases "x = Dios")
  case True
  have sD: "sustancia Dios" using ‹premisas_P15› unfolding premisas_P15_def D6_def by blast
  obtain y where c: "Causa y Dios" using ‹H_todo_tiene_causa› unfolding H_todo_tiene_causa_def by blast
  have "y = Dios" using P6_cor [of Dios y] ‹A4› sD c by blast
  then show ?thesis using c True by simp
next
  case False
  have "En x Dios" using P15_por_premisas ‹premisas_P15› by blast
  then show ?thesis using False ‹H_inherencia_es_causalidad›
    unfolding H_inherencia_es_causalidad_def by blast
qed

text ‹Prop. 16, cor. 3, y prop. 17: Dios es causa absolutamente primera y obra por las
  solas leyes de su naturaleza, sin ser coaccionado por nadie. Es el corolario de la
  prop. 6 aplicado a Dios.›

theorem P17:
  assumes "A4" "D6" "Causa y Dios"
  shows "y = Dios"
  using assms P6_cor [of Dios y] unfolding D6_def by blast

text ‹Prop. 17, cor. 2: Solo Dios es causa libre.›
```

```
theorem P17_cor2:
  assumes "premisas_P15" "A4" "D1" "H_todo_tiene_causa" "H_por_si_excluye_por_otro"
  shows "libre x ⟷ x = Dios"
proof
  assume "libre x"
  then have e: "EsenciaImplicaExistencia x" unfolding libre_def by blast
  have "sustancia x ∨ modo x"
    using todo_sustancia_o_modo ‹premisas_P15› unfolding premisas_P15_def by blast
  moreover have "¬ modo x" using P24 [of x] ‹A4› ‹D1› ‹H_por_si_excluye_por_otro› e by blast
  ultimately have "sustancia x" by blast
  then show "x = Dios" using P14 ‹premisas_P15› unfolding premisas_P15_def by blast
next
  assume x: "x = Dios"
  have sD: "sustancia Dios" using ‹premisas_P15› unfolding premisas_P15_def D6_def by blast
  have "EsenciaImplicaExistencia Dios" using P7 [of Dios] ‹A4› ‹D1› ‹H_todo_tiene_causa› sD by blast
  moreover have "∀y. Causa y Dios ⟶ y = Dios" using P6_cor ‹A4› sD by blast
  ultimately show "libre x" using x unfolding libre_def by blast
qed

text ‹Prop. 18: Dios es causa inmanente, y no transitiva, de todas las cosas.›

theorem P18:
  assumes "premisas_P15" "A4" "H_todo_tiene_causa" "H_inherencia_es_causalidad"
  shows "causa_inmanente Dios x ∧ ¬ causa_transitiva Dios x"
  using P16_cor1 [OF assms] P15_por_premisas [OF ‹premisas_P15›]
  unfolding causa_inmanente_def causa_transitiva_def by blast

text ‹Prop. 19: Dios, o sea, todos los atributos de Dios, son eternos.›

theorem P19:
  assumes "A4" "D1" "D4" "D6" "H_todo_tiene_causa"
  shows "eterno Dios ∧ (∀a. atributo_eterno a)"
proof -
  have "∀s. sustancia s ⟶ EsenciaImplicaExistencia s" using P7 assms by blast
  then show ?thesis using ‹D4› ‹D6› unfolding eterno_def atributo_eterno_def D4_def D6_def by blast
qed

text ‹Prop. 26: Una cosa determinada a obrar lo ha sido necesariamente por Dios; y la
  que no ha sido determinada por Dios no puede determinarse a sí misma.
  Con el cor. 1 de la prop. 16 es inmediata.›

theorem P26:
  assumes "premisas_P15" "A4" "H_todo_tiene_causa" "H_inherencia_es_causalidad"
  shows "((∃y. Causa y x) ⟶ Causa Dios x) ∧ (¬ Causa Dios x ⟶ ¬ Causa x x)"
  using P16_cor1 [OF assms] by blast

text ‹Prop. 29: En la naturaleza no hay nada contingente. Con la definición del
  esc. 1 de la prop. 33, es exactamente H_todo_tiene_causa: no hace falta nada más.›

theorem P29:
  assumes "H_todo_tiene_causa"
  shows "¬ contingente x"
  using assms unfolding H_todo_tiene_causa_def contingente_def by blast

text ‹Prop. 36: Nada existe de cuya naturaleza no se siga algún efecto.
  Spinoza se apoya en el cor. de la prop. 25 y en la prop. 34 (la potencia de Dios es
  su esencia), que no hemos formalizado. Con todo lo anterior no sale: un modo sin
  efecto ninguno es compatible con todas las premisas.›

lemma P36_sin_premisa:
  "premisas_P15 ⟹ A4 ⟹ D1 ⟹ H_todo_tiene_causa ⟹ H_inherencia_es_causalidad
   ⟹ H_por_si_excluye_por_otro ⟹ ∃y. Causa x y"
  nitpick [show_consts]
  oops


text ‹Prop. 32: La voluntad no puede llamarse causa libre, sino solo necesaria.
```

```
  Spinoza cita la def. 7 y las props. 23, 28 y 29. La prueba la encontró Sledgehammer
  en el piloto (Spinoza_Etica_I_piloto.thy) y no usa lo citado: sale de la prop. 24.
  Lo libre tiene una esencia que implica la existencia, y la de un modo no la implica.›

theorem P32:
  assumes "A4" "D1" "H_por_si_excluye_por_otro" "modo Voluntad"
  shows "¬ libre Voluntad"
  using assms P24 libre_def by blast


section ‹Modos infinitos y doble eternidad (props. 21-23 y 28)›

consts
  ModoDeAtributo          :: "cosa ⇒ atributo ⇒ bool"  — ‹m es un modo del atributo a›
  SigueDeAtributoAbsoluto :: "cosa ⇒ atributo ⇒ bool"  — ‹x se sigue de la naturaleza absoluta de a›
  SigueDeModificacion     :: "cosa ⇒ cosa ⇒ bool"      — ‹x se sigue de un atributo modificado por y›

text ‹Def. 2 para modos: es finito el modo al que limita otro modo del mismo atributo.›

definition finita_modo :: "cosa ⇒ bool" where
  "finita_modo x ≡ ∃y. y ≠ x ∧ Limita y x ∧ (∃a. ModoDeAtributo y a ∧ ModoDeAtributo x a)"

text ‹Premisas que dan sentido al vocabulario nuevo.
  H_modo_de_atributo_es_modo: lo que es modo de un atributo es un modo.
  H_todo_modo_de_algun_atributo: y todo modo lo es de algún atributo (cf. II, prop. 6).
  H_limitar_es_determinar: lo que limita a una cosa la determina, es decir, es causa
    suya (def. 2 leída con la def. 7: lo limitado está «coaccionado» por otro).
  H_lo_que_limita_es_finito: lo que limita a otra cosa es a su vez finito (como el
    cuerpo mayor del ejemplo de la def. 2, que otro mayor limita).
  H_absoluto_solo_de_Dios: lo que se sigue de la naturaleza absoluta de un atributo es
    modo de ese atributo y no tiene más causa que Dios (se sigue sin intermediarios).
  H_mediato_de_Dios_y_su_modificacion: lo que se sigue de un atributo modificado por y
    tiene por causa a y, que es un modo, y no tiene más causas que Dios e y. (Sin «que es
    un modo», Nitpick hacía seguirse un modo de un atributo «modificado por Dios».)
  H_dos_maneras_de_seguirse: todo modo se sigue de Dios de una de las dos maneras.
  H_de_lo_finito_lo_finito: lo que se sigue de una modificación finita es finito.›

definition H_modo_de_atributo_es_modo :: bool where
  "H_modo_de_atributo_es_modo ≡ ∀y a. ModoDeAtributo y a ⟶ modo y"

definition H_todo_modo_de_algun_atributo :: bool where
  "H_todo_modo_de_algun_atributo ≡ ∀m. modo m ⟶ (∃a. ModoDeAtributo m a)"

definition H_limitar_es_determinar :: bool where
  "H_limitar_es_determinar ≡ ∀x y. Limita y x ⟶ Causa y x"

definition H_lo_que_limita_es_finito :: bool where
  "H_lo_que_limita_es_finito ≡ ∀x y. Limita y x ∧ y ≠ x ⟶ finita_modo y"

definition H_absoluto_solo_de_Dios :: bool where
  "H_absoluto_solo_de_Dios ≡ ∀x a. SigueDeAtributoAbsoluto x a
     ⟶ ModoDeAtributo x a ∧ (∀z. Causa z x ⟶ z = Dios)"

definition H_mediato_de_Dios_y_su_modificacion :: bool where
  "H_mediato_de_Dios_y_su_modificacion ≡ ∀x y. SigueDeModificacion x y
     ⟶ modo y ∧ Causa y x ∧ (∀z. Causa z x ⟶ z = Dios ∨ z = y)"

definition H_dos_maneras_de_seguirse :: bool where
  "H_dos_maneras_de_seguirse ≡ ∀m. modo m
     ⟶ (∃a. SigueDeAtributoAbsoluto m a) ∨ (∃y. SigueDeModificacion m y)"

definition H_de_lo_finito_lo_finito :: bool where
  "H_de_lo_finito_lo_finito ≡ ∀m y. SigueDeModificacion m y ∧ finita_modo y ⟶ finita_modo m"

text ‹Doble eternidad. La def. 8, como la leímos, es la eternidad por la esencia
  (eterno: la esencia implica la existencia). Spinoza dice además que los modos
  infinitos son eternos «por el atributo» (prop. 21): porque se siguen, inmediata o
  mediatamente, de un atributo eterno.›
```

```
definition eterno_por_atributo :: "cosa ⇒ bool" where
  "eterno_por_atributo x ≡ (∃a. SigueDeAtributoAbsoluto x a)
     ∨ (∃y a. SigueDeModificacion x y ∧ SigueDeAtributoAbsoluto y a)"

lemma ningun_modo_eterno_por_esencia:
  assumes "A4" "D1" "H_por_si_excluye_por_otro" "modo x"
  shows "¬ eterno x"
  using P24 [of x] assms unfolding eterno_def by blast

text ‹Prop. 21: Todo lo que se sigue de la naturaleza absoluta de un atributo de Dios
  ha debido existir siempre y ser infinito, o sea, es eterno e infinito por ese
  atributo. Sin H_limitar_es_determinar no sale:›

lemma P21_sin_limitar:
  "D6 ⟹ H_modo_de_atributo_es_modo ⟹ H_absoluto_solo_de_Dios
   ⟹ SigueDeAtributoAbsoluto x a ⟹ ¬ finita_modo x"
  nitpick [show_consts]
  oops

theorem P21:
  assumes "D6" "H_modo_de_atributo_es_modo" "H_limitar_es_determinar"
      "H_absoluto_solo_de_Dios" "SigueDeAtributoAbsoluto x a"
  shows "¬ finita_modo x"
proof
  assume "finita_modo x"
  then obtain y b where y: "y ≠ x" "Limita y x" "ModoDeAtributo y b" unfolding finita_modo_def by blast
  then have "Causa y x" using ‹H_limitar_es_determinar› unfolding H_limitar_es_determinar_def by blast
  then have "y = Dios" using ‹H_absoluto_solo_de_Dios› ‹SigueDeAtributoAbsoluto x a›
    unfolding H_absoluto_solo_de_Dios_def by blast
  moreover have "modo y" using ‹H_modo_de_atributo_es_modo› y unfolding H_modo_de_atributo_es_modo_def by
blast
  moreover have "sustancia Dios" using ‹D6› unfolding D6_def by blast
  ultimately show False using sustancia_no_modo by blast
qed

text ‹Prop. 21 entera, con la doble eternidad: lo que se sigue de la naturaleza
  absoluta de un atributo es infinito, eterno por el atributo y no eterno por su esencia.›

theorem P21_doble_eternidad:
  assumes "D6" "A4" "D1" "H_por_si_excluye_por_otro" "H_modo_de_atributo_es_modo"
      "H_limitar_es_determinar" "H_absoluto_solo_de_Dios" "SigueDeAtributoAbsoluto x a"
  shows "¬ finita_modo x ∧ eterno_por_atributo x ∧ ¬ eterno x"
proof -
  have "¬ finita_modo x" using P21 [of x a] assms by blast
  moreover have "modo x" using ‹H_absoluto_solo_de_Dios› ‹H_modo_de_atributo_es_modo›
‹SigueDeAtributoAbsoluto x a›
    unfolding H_absoluto_solo_de_Dios_def H_modo_de_atributo_es_modo_def by blast
  ultimately show ?thesis using ningun_modo_eterno_por_esencia [of x] assms
    unfolding eterno_por_atributo_def by blast
qed

text ‹Prop. 22: Lo que se sigue de un atributo en cuanto modificado por una modificación
  que existe necesariamente y es infinita, también es infinito.›

theorem P22:
  assumes "D6" "H_modo_de_atributo_es_modo" "H_limitar_es_determinar" "H_lo_que_limita_es_finito"
      "H_mediato_de_Dios_y_su_modificacion" "SigueDeModificacion x y" "¬ finita_modo y"
  shows "¬ finita_modo x"
proof
  assume "finita_modo x"
  then obtain w b where w: "w ≠ x" "Limita w x" "ModoDeAtributo w b" unfolding finita_modo_def by blast
  then have "Causa w x" using ‹H_limitar_es_determinar› unfolding H_limitar_es_determinar_def by blast
  then have "w = Dios ∨ w = y" using ‹H_mediato_de_Dios_y_su_modificacion› ‹SigueDeModificacion x y›
    unfolding H_mediato_de_Dios_y_su_modificacion_def by blast
  moreover have "w ≠ Dios"
  proof
    assume "w = Dios"
```

```
    moreover have "modo w" using ‹H_modo_de_atributo_es_modo› w unfolding H_modo_de_atributo_es_modo_def
by blast
    moreover have "sustancia Dios" using ‹D6› unfolding D6_def by blast
    ultimately show False using sustancia_no_modo by blast
  qed
  moreover have "finita_modo w" using ‹H_lo_que_limita_es_finito› w unfolding
H_lo_que_limita_es_finito_def by blast
  ultimately show False using ‹¬ finita_modo y› by blast
qed

text ‹Prop. 23: Todo modo que existe necesariamente y es infinito ha debido seguirse
  de la naturaleza absoluta de un atributo, o de un atributo modificado por una
  modificación infinita. Sin H_de_lo_finito_lo_finito no sale:›

lemma P23_sin_finito:
  "H_dos_maneras_de_seguirse ⟹ modo m ⟹ ¬ finita_modo m
   ⟹ (∃a. SigueDeAtributoAbsoluto m a) ∨ (∃y. SigueDeModificacion m y ∧ ¬ finita_modo y)"
  nitpick [show_consts]
  oops

theorem P23:
  assumes "H_dos_maneras_de_seguirse" "H_de_lo_finito_lo_finito" "modo m" "¬ finita_modo m"
  shows "(∃a. SigueDeAtributoAbsoluto m a) ∨ (∃y. SigueDeModificacion m y ∧ ¬ finita_modo y)"
  using assms unfolding H_dos_maneras_de_seguirse_def H_de_lo_finito_lo_finito_def by blast

text ‹Prop. 28: Toda cosa singular, o sea, toda cosa finita, no puede existir ni ser
  determinada a obrar si no es determinada por otra causa, que también es finita.
  No necesita H_de_lo_finito_lo_finito: basta con las props. 21, 22 y 24.›

theorem P28:
  assumes "D6" "A4" "D1" "H_por_si_excluye_por_otro" "H_modo_de_atributo_es_modo"
      "H_limitar_es_determinar" "H_lo_que_limita_es_finito" "H_absoluto_solo_de_Dios"
      "H_mediato_de_Dios_y_su_modificacion" "H_dos_maneras_de_seguirse" "modo m" "finita_modo m"
  shows "∃y. y ≠ m ∧ modo y ∧ finita_modo y ∧ Causa y m"
proof -
  have "¬ (∃a. SigueDeAtributoAbsoluto m a)" using P21 assms by blast
  then obtain y where s: "SigueDeModificacion m y"
    using ‹H_dos_maneras_de_seguirse› ‹modo m› unfolding H_dos_maneras_de_seguirse_def by blast
  have fy: "finita_modo y" using P22 [of m y] assms s by blast
  have c: "Causa y m" using ‹H_mediato_de_Dios_y_su_modificacion› s
    unfolding H_mediato_de_Dios_y_su_modificacion_def by blast
  have "y ≠ m" using P24 [of m] ‹A4› ‹D1› ‹H_por_si_excluye_por_otro› ‹modo m› c by blast
  moreover have "modo y" using fy ‹H_modo_de_atributo_es_modo›
    unfolding finita_modo_def H_modo_de_atributo_es_modo_def by blast
  ultimately show ?thesis using fy c by blast
qed


section ‹Mente y cuerpo (II, props. 6 y 7; III, prop. 2)›

consts
  Pensamiento :: atributo
  Extension   :: atributo
  IdeaDe      :: "cosa ⇒ cosa ⇒ bool"   — ‹IdeaDe i x: i es la idea de x›

text ‹Premisas.
  H_atributos_distintos: pensamiento y extensión son atributos distintos.
  H_modo_se_concibe_por_su_atributo: un modo de un atributo solo se concibe, fuera de
    Dios, por cosas de ese mismo atributo. Es la versión para los modos de la prop. 10
    fuerte, y la razón que da Spinoza en la demostración de II, 6: los modos de cada
    atributo implican el concepto de su atributo y no el de otro.
  H_un_atributo_por_modo: cada modo lo es de un solo atributo. Choca con la lectura
    literal del escolio de II, 7 («un modo de la extensión y la idea de ese modo son una
    y la misma cosa»): aquí el cuerpo y su idea son dos cosas del dominio.
  H_paralelismo: si una cosa causa otra, la idea de la primera causa la idea de la
    segunda (II, 7: el orden y la conexión de las ideas es el de las cosas).
  H_ideas_del_pensamiento: las ideas son modos del pensamiento.›
```

```
definition H_atributos_distintos :: bool where
  "H_atributos_distintos ≡ Pensamiento ≠ Extension"

definition H_modo_se_concibe_por_su_atributo :: bool where
  "H_modo_se_concibe_por_su_atributo ≡ ∀x y b. ModoDeAtributo y b ∧ SeConcibePor y x ∧ x ≠ Dios
     ⟶ ModoDeAtributo x b"

definition H_un_atributo_por_modo :: bool where
  "H_un_atributo_por_modo ≡ ∀x a b. ModoDeAtributo x a ∧ ModoDeAtributo x b ⟶ a = b"

definition H_paralelismo :: bool where
  "H_paralelismo ≡ ∀x y i j. Causa x y ∧ IdeaDe i x ∧ IdeaDe j y ⟶ Causa i j"

definition H_ideas_del_pensamiento :: bool where
  "H_ideas_del_pensamiento ≡ ∀i x. IdeaDe i x ⟶ ModoDeAtributo i Pensamiento"

text ‹II, prop. 6: Los modos de cualquier atributo tienen por causa a Dios solo en
  cuanto se le considera bajo el atributo del que son modos, y no bajo otro.
  Formalizado: lo que causa un modo de un atributo, si no es Dios, es del mismo
  atributo. Con el axioma 4 solo, no sale:›

lemma II6_sin_premisa:
  "A4 ⟹ ModoDeAtributo y b ⟹ Causa x y ⟹ x ≠ Dios ⟹ ModoDeAtributo x b"
  nitpick [show_consts]
  oops

theorem II6:
  assumes "A4" "H_modo_se_concibe_por_su_atributo" "ModoDeAtributo y b" "Causa x y" "x ≠ Dios"
  shows "ModoDeAtributo x b"
  using assms unfolding A4_def H_modo_se_concibe_por_su_atributo_def by blast

text ‹III, prop. 2: Ni el cuerpo puede determinar a la mente a pensar, ni la mente al
  cuerpo al movimiento o al reposo. Formalizado: ningún modo de un atributo causa un
  modo de otro atributo, ni al revés.
  Sin H_modo_se_concibe_por_su_atributo, el mundo cartesiano:›

lemma III2_sin_concepto:
  "A4 ⟹ D6 ⟹ H_modo_de_atributo_es_modo ⟹ H_un_atributo_por_modo
   ⟹ ModoDeAtributo x a ⟹ ModoDeAtributo y b ⟹ a ≠ b ⟹ ¬ Causa x y"
  nitpick [show_consts]
  oops

text ‹Sin H_un_atributo_por_modo: una cosa que es a la vez modo de los dos atributos.›

lemma III2_sin_un_atributo:
  "A4 ⟹ D6 ⟹ H_modo_de_atributo_es_modo ⟹ H_modo_se_concibe_por_su_atributo
   ⟹ ModoDeAtributo x a ⟹ ModoDeAtributo y b ⟹ a ≠ b ⟹ ¬ Causa x y"
  nitpick [show_consts]
  oops

theorem III2:
  assumes "A4" "D6" "H_modo_de_atributo_es_modo" "H_modo_se_concibe_por_su_atributo"
      "H_un_atributo_por_modo" "ModoDeAtributo x a" "ModoDeAtributo y b" "a ≠ b"
  shows "¬ Causa x y ∧ ¬ Causa y x"
proof -
  have sD: "sustancia Dios" using ‹D6› unfolding D6_def by blast
  have "modo x" "modo y" using ‹H_modo_de_atributo_es_modo› ‹ModoDeAtributo x a› ‹ModoDeAtributo y b›
    unfolding H_modo_de_atributo_es_modo_def by blast+
  then have dx: "x ≠ Dios" and dy: "y ≠ Dios" using sD sustancia_no_modo by blast+
  have "¬ Causa x y"
  proof
    assume "Causa x y"
    then have "ModoDeAtributo x b"
      using II6 [of y b x] ‹A4› ‹H_modo_se_concibe_por_su_atributo› ‹ModoDeAtributo y b› dx by blast
    then show False using ‹H_un_atributo_por_modo› ‹ModoDeAtributo x a› ‹a ≠ b›
      unfolding H_un_atributo_por_modo_def by blast
  qed
  moreover have "¬ Causa y x"
```

```
  proof
    assume "Causa y x"
    then have "ModoDeAtributo y a"
      using II6 [of x a y] ‹A4› ‹H_modo_se_concibe_por_su_atributo› ‹ModoDeAtributo x a› dy by blast
    then show False using ‹H_un_atributo_por_modo› ‹ModoDeAtributo y b› ‹a ≠ b›
      unfolding H_un_atributo_por_modo_def by blast
  qed
  ultimately show ?thesis by blast
qed

text ‹II, prop. 7: «Es evidente por el axioma 4 de la Parte I: la idea de cada cosa
  causada depende del conocimiento de la causa de la que es efecto». Para que salga del
  axioma 4 hace falta un puente: que una cosa se conciba por otra significa que la idea
  de la segunda causa la idea de la primera. Sin él, no sale:›

definition H_concebir_es_causar_ideas :: bool where
  "H_concebir_es_causar_ideas ≡ ∀x y i j. SeConcibePor y x ∧ IdeaDe i x ∧ IdeaDe j y ⟶ Causa i j"

lemma II7_sin_puente:
  "A4 ⟹ H_paralelismo"
  nitpick [show_consts]
  oops

theorem II7:
  assumes "A4" "H_concebir_es_causar_ideas"
  shows "H_paralelismo"
  using assms unfolding A4_def H_concebir_es_causar_ideas_def H_paralelismo_def by blast


text ‹Correspondencia sin interacción (II, 7 con III, 2): si un cuerpo causa otro, la
  idea del primero causa la idea del segundo; y ni el cuerpo causa la idea ni la idea
  el cuerpo.›

theorem correspondencia_sin_interaccion:
  assumes "A4" "D6" "H_atributos_distintos" "H_modo_de_atributo_es_modo"
      "H_modo_se_concibe_por_su_atributo" "H_un_atributo_por_modo" "H_paralelismo"
      "H_ideas_del_pensamiento" "ModoDeAtributo c1 Extension" "ModoDeAtributo c2 Extension"
      "Causa c1 c2" "IdeaDe i1 c1" "IdeaDe i2 c2"
  shows "Causa i1 i2 ∧ ¬ Causa c1 i2 ∧ ¬ Causa i1 c2"
proof -
  have "Causa i1 i2" using ‹H_paralelismo› ‹Causa c1 c2› ‹IdeaDe i1 c1› ‹IdeaDe i2 c2›
    unfolding H_paralelismo_def by blast
  moreover have p1: "ModoDeAtributo i1 Pensamiento" and p2: "ModoDeAtributo i2 Pensamiento"
    using ‹H_ideas_del_pensamiento› ‹IdeaDe i1 c1› ‹IdeaDe i2 c2› unfolding H_ideas_del_pensamiento_def by
blast+
  moreover have d1: "Extension ≠ Pensamiento" and d2: "Pensamiento ≠ Extension"
    using ‹H_atributos_distintos› unfolding H_atributos_distintos_def by auto
  moreover have "¬ Causa c1 i2"
    using III2 [of c1 Extension i2 Pensamiento] assms p2 d1 by blast
  moreover have "¬ Causa i1 c2"
    using III2 [of i1 Pensamiento c2 Extension] assms p1 d2 by blast
  ultimately show ?thesis by blast
qed


section ‹El «en cuanto» (quatenus): cuerpo e idea, una misma cosa›

text ‹Spinoza dice que un modo de la extensión y su idea son «una y la misma cosa,
  expresada de dos maneras» (II, 7, esc.), y aun así que el cuerpo no causa la mente
  (III, 2). Lo hace compatible con el «en cuanto»: una cosa causa algo en cuanto se la
  considera bajo un atributo. Relativizamos la causalidad y la concepción a los
  atributos, y dejamos que una cosa sea modo de varios (sin H_un_atributo_por_modo).›

consts
  CausaBajo     :: "atributo ⇒ atributo ⇒ cosa ⇒ cosa ⇒ bool"
    — ‹CausaBajo a b x y: x, en cuanto se le considera bajo a, causa y en cuanto bajo b›
  SeConcibeBajo :: "atributo ⇒ atributo ⇒ cosa ⇒ cosa ⇒ bool"
    — ‹SeConcibeBajo b a y x: y, en cuanto bajo b, se concibe por x en cuanto bajo a›
```

```
text ‹Premisas, versiones «en cuanto» de las anteriores.
  A4_quatenus: el efecto, en cuanto b, se concibe por la causa en cuanto a (ax. 4).
  H_atributo_por_si_quatenus: lo que se concibe bajo un atributo solo se concibe por
    algo bajo ese mismo atributo (prop. 10 fuerte, «sin el otro»).
  H_en_cuanto_requiere_atributo: para causar o ser causado en cuanto a, hay que tener a
    (Dios, como causa, los tiene todos).
  H_mismo_orden: si x causa y bajo un atributo, lo causa bajo cualquier otro que ambos
    tengan (II, 7, esc.: «un mismo orden, una misma conexión de causas»).
  H_causa_relativa_es_causa: causar en cuanto es causar.›

definition A4_quatenus :: bool where
  "A4_quatenus ≡ ∀a b x y. CausaBajo a b x y ⟶ SeConcibeBajo b a y x"

definition H_atributo_por_si_quatenus :: bool where
  "H_atributo_por_si_quatenus ≡ ∀a b x y. SeConcibeBajo b a y x ⟶ a = b"

definition H_en_cuanto_requiere_atributo :: bool where
  "H_en_cuanto_requiere_atributo ≡ ∀a b x y. CausaBajo a b x y
     ⟶ (x = Dios ∨ ModoDeAtributo x a) ∧ ModoDeAtributo y b"

definition H_mismo_orden :: bool where
  "H_mismo_orden ≡ ∀a b x y. CausaBajo a a x y ∧ ModoDeAtributo x b ∧ ModoDeAtributo y b
     ⟶ CausaBajo b b x y"

definition H_causa_relativa_es_causa :: bool where
  "H_causa_relativa_es_causa ≡ ∀a b x y. CausaBajo a b x y ⟶ Causa x y"

text ‹III, 2 con el «en cuanto»: nada, en cuanto se le considera bajo un atributo,
  causa algo en cuanto se le considera bajo otro. Sin la prop. 10 fuerte relativizada,
  vuelve el mundo cartesiano:›

lemma III2_quatenus_sin_10:
  "A4_quatenus ⟹ H_atributos_distintos ⟹ ¬ CausaBajo Extension Pensamiento x y"
  nitpick [show_consts]
  oops

theorem III2_quatenus:
  assumes "A4_quatenus" "H_atributo_por_si_quatenus" "CausaBajo a b x y"
  shows "a = b"
  using assms unfolding A4_quatenus_def H_atributo_por_si_quatenus_def by blast

corollary III2_quatenus_mente_cuerpo:
  assumes "A4_quatenus" "H_atributo_por_si_quatenus" "H_atributos_distintos"
  shows "¬ CausaBajo Extension Pensamiento x y ∧ ¬ CausaBajo Pensamiento Extension x y"
  using assms III2_quatenus unfolding H_atributos_distintos_def by metis

text ‹Correspondencia con identidad: c1 y c2 son cada uno cuerpo y mente a la vez. Si
  c1 causa c2 en cuanto cuerpos, también lo causa en cuanto mentes; y nunca lo causa de
  un atributo al otro.›

theorem correspondencia_quatenus:
  assumes "A4_quatenus" "H_atributo_por_si_quatenus" "H_atributos_distintos" "H_mismo_orden"
      "ModoDeAtributo c1 Extension" "ModoDeAtributo c1 Pensamiento"
      "ModoDeAtributo c2 Extension" "ModoDeAtributo c2 Pensamiento"
      "CausaBajo Extension Extension c1 c2"
  shows "CausaBajo Pensamiento Pensamiento c1 c2
         ∧ ¬ CausaBajo Extension Pensamiento c1 c2 ∧ ¬ CausaBajo Pensamiento Extension c1 c2"
proof -
  have "CausaBajo Pensamiento Pensamiento c1 c2"
    using ‹H_mismo_orden› assms(6) assms(8) assms(9) unfolding H_mismo_orden_def by blast
  then show ?thesis using III2_quatenus_mente_cuerpo [of c1 c2] assms(1-3) by blast
qed

text ‹Y sin el «en cuanto»: con la identidad, en el sentido llano de «causa» hay una
  cosa de la extensión que causa una cosa del pensamiento. Por eso la III, 2 sin
  relativizar necesitaba H_un_atributo_por_modo.›
```

```
theorem sin_quatenus_un_cuerpo_causa_una_mente:
  assumes "H_causa_relativa_es_causa" "ModoDeAtributo c1 Extension" "ModoDeAtributo c2 Pensamiento"
      "CausaBajo Extension Extension c1 c2"
  shows "∃x y. ModoDeAtributo x Extension ∧ ModoDeAtributo y Pensamiento ∧ Causa x y"
  using assms unfolding H_causa_relativa_es_causa_def by blast


section ‹¿Son compatibles todas las premisas?›

text ‹Si el conjunto fuera contradictorio, de él se seguiría cualquier cosa y todas
  las pruebas anteriores serían trivialmente válidas. Nitpick busca un mundo en que
  se cumplan todas a la vez, las de las props. 1-15 y 24.›

lemma consistencia:
  "A1_excluyente ∧ H_por_si_excluye_por_otro ∧ A2 ∧ A4 ∧ A5 ∧ A7 ∧ D1 ∧ D4 ∧ D6 ∧ H_conversa_A5 ∧
H_quien_comparte_uno_comparte_todos ∧ H_individuacion_por_atributos ∧ H_ser_en_implica_concebirse ∧
H_solo_en_si_solo_por_si
   ∧ H_atributo_compartido_es_comun ∧ H_todo_tiene_causa ∧ H_realidad_es_atributos ∧
H_puente_atributo_sustancia ∧ H_todo_por_sus_partes ∧ H_sustancia_con_atributo
   ∧ H_modo_en_sustancia ∧ H_partes_de_modos ∧ H_inherencia_es_causalidad"
  nitpick [satisfy, show_consts]
  oops

text ‹Y lo mismo, exigiendo además que algún modo sea divisible (los cuerpos tienen
  partes, aunque la sustancia no) y que algún modo cause a otro (prop. 28). Con la
  premisa fuerte no había modelo; con la debilitada debe haberlo. Desde la prop. 24,
  el ax. 1 va en su lectura excluyente y se añade H_por_si_excluye_por_otro.›

lemma consistencia_con_modos_divisibles_y_causas:
  "A1_excluyente ∧ H_por_si_excluye_por_otro ∧ A2 ∧ A4 ∧ A5 ∧ A7 ∧ D1 ∧ D4 ∧ D6 ∧ H_conversa_A5 ∧
H_quien_comparte_uno_comparte_todos ∧ H_individuacion_por_atributos ∧ H_ser_en_implica_concebirse ∧
H_solo_en_si_solo_por_si
   ∧ H_atributo_compartido_es_comun ∧ H_todo_tiene_causa ∧ H_realidad_es_atributos ∧
H_puente_atributo_sustancia ∧ H_todo_por_sus_partes ∧ H_sustancia_con_atributo
   ∧ H_modo_en_sustancia ∧ H_partes_de_modos ∧ H_inherencia_es_causalidad ∧ (∃m. modo m ∧ divisible m)
   ∧ (∃m n. modo m ∧ modo n ∧ m ≠ n ∧ Causa n m)"
  nitpick [satisfy, show_consts, timeout = 300]
  oops

text ‹Y, además, con la prop. 36: todo produce algún efecto. En un modelo finito esto
  obliga a que las cadenas de causas entre modos se cierren en círculo.›

lemma consistencia_con_P36:
  "A1_excluyente ∧ H_por_si_excluye_por_otro ∧ A2 ∧ A4 ∧ A5 ∧ A7 ∧ D1 ∧ D4 ∧ D6
   ∧ H_conversa_A5 ∧ H_quien_comparte_uno_comparte_todos ∧ H_individuacion_por_atributos
   ∧ H_ser_en_implica_concebirse ∧ H_solo_en_si_solo_por_si ∧ H_atributo_compartido_es_comun
   ∧ H_todo_tiene_causa ∧ H_realidad_es_atributos ∧ H_puente_atributo_sustancia
   ∧ H_todo_por_sus_partes ∧ H_sustancia_con_atributo ∧ H_modo_en_sustancia
   ∧ H_partes_de_modos ∧ H_inherencia_es_causalidad
   ∧ (∃m n. modo m ∧ modo n ∧ m ≠ n) ∧ (∀x. ∃y. Causa x y)"
  nitpick [satisfy, show_consts, timeout = 300]
  oops

text ‹Con los modos infinitos: todas las premisas anteriores, las de esta sección, un
  modo infinito inmediato, uno mediato y un modo finito.›

lemma consistencia_modos_infinitos:
  "A1_excluyente ∧ H_por_si_excluye_por_otro ∧ A2 ∧ A4 ∧ A5 ∧ A7 ∧ D1 ∧ D4 ∧ D6
   ∧ H_conversa_A5 ∧ H_quien_comparte_uno_comparte_todos ∧ H_individuacion_por_atributos
   ∧ H_ser_en_implica_concebirse ∧ H_solo_en_si_solo_por_si ∧ H_atributo_compartido_es_comun
   ∧ H_todo_tiene_causa ∧ H_realidad_es_atributos ∧ H_puente_atributo_sustancia
   ∧ H_todo_por_sus_partes ∧ H_sustancia_con_atributo ∧ H_modo_en_sustancia
   ∧ H_partes_de_modos ∧ H_inherencia_es_causalidad
   ∧ H_modo_de_atributo_es_modo ∧ H_todo_modo_de_algun_atributo ∧ H_limitar_es_determinar ∧
H_lo_que_limita_es_finito
   ∧ H_absoluto_solo_de_Dios ∧ H_mediato_de_Dios_y_su_modificacion
   ∧ H_dos_maneras_de_seguirse ∧ H_de_lo_finito_lo_finito
   ∧ (∃i a. SigueDeAtributoAbsoluto i a)
```

```
   ∧ (∃j i. SigueDeModificacion j i ∧ ¬ finita_modo i)
   ∧ (∃f. modo f ∧ finita_modo f)"
  nitpick [satisfy, show_consts, timeout = 600]
  oops

text ‹Con mente y cuerpo: todas las premisas, dos cuerpos que se causan y sus ideas.›

lemma consistencia_mente_cuerpo:
  "A1_excluyente ∧ H_por_si_excluye_por_otro ∧ A2 ∧ A4 ∧ A5 ∧ A7 ∧ D1 ∧ D4 ∧ D6
   ∧ H_conversa_A5 ∧ H_quien_comparte_uno_comparte_todos ∧ H_individuacion_por_atributos
   ∧ H_ser_en_implica_concebirse ∧ H_solo_en_si_solo_por_si ∧ H_atributo_compartido_es_comun
   ∧ H_todo_tiene_causa ∧ H_realidad_es_atributos ∧ H_puente_atributo_sustancia
   ∧ H_todo_por_sus_partes ∧ H_sustancia_con_atributo ∧ H_modo_en_sustancia
   ∧ H_partes_de_modos ∧ H_inherencia_es_causalidad
   ∧ H_modo_de_atributo_es_modo ∧ H_todo_modo_de_algun_atributo ∧ H_limitar_es_determinar
   ∧ H_lo_que_limita_es_finito ∧ H_absoluto_solo_de_Dios ∧ H_mediato_de_Dios_y_su_modificacion
   ∧ H_dos_maneras_de_seguirse ∧ H_de_lo_finito_lo_finito
   ∧ H_atributos_distintos ∧ H_modo_se_concibe_por_su_atributo ∧ H_un_atributo_por_modo
   ∧ H_paralelismo ∧ H_ideas_del_pensamiento
   ∧ (∃c1 c2 i1 i2. ModoDeAtributo c1 Extension ∧ ModoDeAtributo c2 Extension ∧ c1 ≠ c2
        ∧ Causa c1 c2 ∧ IdeaDe i1 c1 ∧ IdeaDe i2 c2)"
  nitpick [satisfy, show_consts, timeout = 600]
  oops

text ‹Con el «en cuanto» y la identidad: todas las premisas salvo H_un_atributo_por_modo,
  y dos cosas distintas que son cada una cuerpo y mente, una causa de la otra.›

lemma consistencia_quatenus:
  "A1_excluyente ∧ H_por_si_excluye_por_otro ∧ A2 ∧ A4 ∧ A5 ∧ A7 ∧ D1 ∧ D4 ∧ D6
   ∧ H_conversa_A5 ∧ H_quien_comparte_uno_comparte_todos ∧ H_individuacion_por_atributos
   ∧ H_ser_en_implica_concebirse ∧ H_solo_en_si_solo_por_si ∧ H_atributo_compartido_es_comun
   ∧ H_todo_tiene_causa ∧ H_realidad_es_atributos ∧ H_puente_atributo_sustancia
   ∧ H_todo_por_sus_partes ∧ H_sustancia_con_atributo ∧ H_modo_en_sustancia
   ∧ H_partes_de_modos ∧ H_inherencia_es_causalidad
   ∧ H_modo_de_atributo_es_modo ∧ H_todo_modo_de_algun_atributo ∧ H_limitar_es_determinar
   ∧ H_lo_que_limita_es_finito ∧ H_absoluto_solo_de_Dios ∧ H_mediato_de_Dios_y_su_modificacion
   ∧ H_dos_maneras_de_seguirse ∧ H_de_lo_finito_lo_finito
   ∧ H_atributos_distintos ∧ H_modo_se_concibe_por_su_atributo
   ∧ H_paralelismo ∧ H_ideas_del_pensamiento ∧ H_concebir_es_causar_ideas
   ∧ A4_quatenus ∧ H_atributo_por_si_quatenus ∧ H_en_cuanto_requiere_atributo
   ∧ H_mismo_orden ∧ H_causa_relativa_es_causa
   ∧ (∃c1 c2. c1 ≠ c2 ∧ ModoDeAtributo c1 Extension ∧ ModoDeAtributo c1 Pensamiento
        ∧ ModoDeAtributo c2 Extension ∧ ModoDeAtributo c2 Pensamiento
        ∧ CausaBajo Extension Extension c1 c2 ∧ IdeaDe c1 c1 ∧ IdeaDe c2 c2)"
  nitpick [satisfy, show_consts, timeout = 600]
  oops

end
```

Spinoza: solo con lo que él cita

Para cada proposición, solo las premisas que Spinoza cita en su demostración, según el grafo «Mapping Spinoza's Ethics» (Boston College).

Fichero `Spinoza_Etica_I_piloto.thy` ▪ 10 resultados demostrados ▪ 104 líneas

```
(* Piloto generado por generar_piloto_desde_grafo.py: no editar a mano.
   Para cada proposición, las hipótesis son lo que Spinoza cita según el grafo de
   Mapping Spinoza's Ethics (Boston College). Nitpick busca contramodelos y
   Sledgehammer busca pruebas con esos mismos hechos. *)

theory Spinoza_Etica_I_piloto
  imports Spinoza_Etica_I
begin

section ‹Proposiciones con solo lo que Spinoza cita›

text ‹Prop. 21: Lo que se sigue de la naturaleza absoluta de un atributo de Dios es infinito.
  Spinoza cita: 1D2, 1P11, 1P20c2.
  Sin formalizar (no entran): 1P20c2.›

lemma P21_con_citas:
  "A4 ⟹ D1 ⟹ H_todo_tiene_causa ⟹ A7 ⟹ D6 ⟹ SigueDeAtributoAbsoluto x a ⟹ ¬ finita_modo x"
  nitpick [timeout = 60]
  sledgehammer [timeout = 30] (finita_def finita_modo_def P11)
  oops

text ‹Prop. 22: Lo que se sigue de un atributo en cuanto modificado por una modificación infinita es
también infinito.
  Spinoza cita: 1D2, 1P11, 1P20c2.
  Sin formalizar (no entran): 1P20c2.›

lemma P22_con_citas:
  "A4 ⟹ D1 ⟹ H_todo_tiene_causa ⟹ A7 ⟹ D6 ⟹ SigueDeModificacion x y ⟹ ¬ finita_modo y ⟹ ¬
finita_modo x"
  nitpick [timeout = 60]
  sledgehammer [timeout = 30] (finita_def finita_modo_def P11)
  oops

text ‹Prop. 23: Todo modo infinito se sigue de la naturaleza absoluta de un atributo o de un atributo
modificado por una modificación infinita.
  Spinoza cita: 1D5, 1D6, 1D8, 1P15, 1P19, 1P21, 1P22.›

lemma P23_con_citas:
  "D6 ⟹ premisas_P15 ⟹ A4 ⟹ D1 ⟹ D4 ⟹ H_todo_tiene_causa ⟹ H_modo_de_atributo_es_modo ⟹
H_limitar_es_determinar ⟹ H_absoluto_solo_de_Dios ⟹ H_lo_que_limita_es_finito ⟹
H_mediato_de_Dios_y_su_modificacion ⟹ modo m ⟹ ¬ finita_modo m ⟹ (∃a. SigueDeAtributoAbsoluto m a) ∨
(∃y. SigueDeModificacion m y ∧ ¬ finita_modo y)"
  nitpick [timeout = 60]
  sledgehammer [timeout = 30] (modo_def D6_def eterno_def P15_por_premisas P19 P21 P22)
  oops

text ‹Prop. 28: Toda cosa finita es determinada a existir y obrar por otra causa finita.
  Spinoza cita: 1D3, 1D5, 1A1, 1P21, 1P22, 1P24c, 1P25c, 1P26, 1P15.
  Sin formalizar (no entran): 1P24c, 1P25c.›

lemma P28_con_citas:
  "A1 ⟹ D6 ⟹ H_modo_de_atributo_es_modo ⟹ H_limitar_es_determinar ⟹ H_absoluto_solo_de_Dios ⟹
H_lo_que_limita_es_finito ⟹ H_mediato_de_Dios_y_su_modificacion ⟹ premisas_P15 ⟹ A4 ⟹
H_todo_tiene_causa ⟹ H_inherencia_es_causalidad ⟹ modo m ⟹ finita_modo m ⟹ ∃y. y ≠ m ∧ modo y ∧
finita_modo y ∧ Causa y m"
  nitpick [timeout = 60]
  sledgehammer [timeout = 30] (sustancia_def modo_def A1_def P21 P22 P26 P15_por_premisas)
  oops

text ‹Prop. 32: La voluntad no puede llamarse causa libre, sino solo necesaria.
  Spinoza cita: 1D7, 1P23, 1P28, 1P29.›
```

```
lemma P32_con_citas:
  "H_dos_maneras_de_seguirse ⟹ H_de_lo_finito_lo_finito ⟹ D6 ⟹ A4 ⟹ D1 ⟹
H_por_si_excluye_por_otro ⟹ H_modo_de_atributo_es_modo ⟹ H_limitar_es_determinar ⟹
H_lo_que_limita_es_finito ⟹ H_absoluto_solo_de_Dios ⟹ H_mediato_de_Dios_y_su_modificacion ⟹
H_todo_tiene_causa ⟹ modo Voluntad ⟹ ¬ libre Voluntad"
  nitpick [timeout = 60]
  sledgehammer [timeout = 30] (libre_def P23 P28 P29)
  oops

section ‹Segunda pasada: con todo lo que ya tenemos›

lemma P21_con_todo:
  "A1_excluyente ⟹ A2 ⟹ A4 ⟹ A5 ⟹ A7 ⟹ D1 ⟹ D4 ⟹ D6 ⟹ H_conversa_A5 ⟹
H_quien_comparte_uno_comparte_todos ⟹ H_individuacion_por_atributos ⟹ H_ser_en_implica_concebirse ⟹
H_solo_en_si_solo_por_si ⟹ H_atributo_compartido_es_comun ⟹ H_todo_tiene_causa ⟹
H_realidad_es_atributos ⟹ H_puente_atributo_sustancia ⟹ H_todo_por_sus_partes ⟹
H_sustancia_con_atributo ⟹ H_modo_en_sustancia ⟹ H_partes_de_modos ⟹ H_por_si_excluye_por_otro ⟹
H_inherencia_es_causalidad ⟹ H_modo_de_atributo_es_modo ⟹ H_limitar_es_determinar ⟹
H_lo_que_limita_es_finito ⟹ H_absoluto_solo_de_Dios ⟹ H_mediato_de_Dios_y_su_modificacion ⟹
H_dos_maneras_de_seguirse ⟹ H_de_lo_finito_lo_finito ⟹ SigueDeAtributoAbsoluto x a ⟹ ¬ finita_modo
x"
  nitpick [timeout = 60]
  sledgehammer [timeout = 30] (A1_excl_implica_A1 premisas_P15_def P15_por_premisas P16_cor1 P17 P17_cor2
P24 P14 sustancia_no_modo todo_sustancia_o_modo libre_def finita_modo_def modo_def sustancia_def D6_def
H_modo_de_atributo_es_modo_def H_limitar_es_determinar_def H_lo_que_limita_es_finito_def
H_absoluto_solo_de_Dios_def H_mediato_de_Dios_y_su_modificacion_def H_dos_maneras_de_seguirse_def
H_de_lo_finito_lo_finito_def)
  oops

lemma P22_con_todo:
  "A1_excluyente ⟹ A2 ⟹ A4 ⟹ A5 ⟹ A7 ⟹ D1 ⟹ D4 ⟹ D6 ⟹ H_conversa_A5 ⟹
H_quien_comparte_uno_comparte_todos ⟹ H_individuacion_por_atributos ⟹ H_ser_en_implica_concebirse ⟹
H_solo_en_si_solo_por_si ⟹ H_atributo_compartido_es_comun ⟹ H_todo_tiene_causa ⟹
H_realidad_es_atributos ⟹ H_puente_atributo_sustancia ⟹ H_todo_por_sus_partes ⟹
H_sustancia_con_atributo ⟹ H_modo_en_sustancia ⟹ H_partes_de_modos ⟹ H_por_si_excluye_por_otro ⟹
H_inherencia_es_causalidad ⟹ H_modo_de_atributo_es_modo ⟹ H_limitar_es_determinar ⟹
H_lo_que_limita_es_finito ⟹ H_absoluto_solo_de_Dios ⟹ H_mediato_de_Dios_y_su_modificacion ⟹
H_dos_maneras_de_seguirse ⟹ H_de_lo_finito_lo_finito ⟹ SigueDeModificacion x y ⟹ ¬ finita_modo y ⟹
¬ finita_modo x"
  nitpick [timeout = 60]
  sledgehammer [timeout = 30] (A1_excl_implica_A1 premisas_P15_def P15_por_premisas P16_cor1 P17 P17_cor2
P24 P14 sustancia_no_modo todo_sustancia_o_modo libre_def finita_modo_def modo_def sustancia_def D6_def
H_modo_de_atributo_es_modo_def H_limitar_es_determinar_def H_lo_que_limita_es_finito_def
H_absoluto_solo_de_Dios_def H_mediato_de_Dios_y_su_modificacion_def H_dos_maneras_de_seguirse_def
H_de_lo_finito_lo_finito_def)
  oops

lemma P23_con_todo:
  "A1_excluyente ⟹ A2 ⟹ A4 ⟹ A5 ⟹ A7 ⟹ D1 ⟹ D4 ⟹ D6 ⟹ H_conversa_A5 ⟹
H_quien_comparte_uno_comparte_todos ⟹ H_individuacion_por_atributos ⟹ H_ser_en_implica_concebirse ⟹
H_solo_en_si_solo_por_si ⟹ H_atributo_compartido_es_comun ⟹ H_todo_tiene_causa ⟹
H_realidad_es_atributos ⟹ H_puente_atributo_sustancia ⟹ H_todo_por_sus_partes ⟹
H_sustancia_con_atributo ⟹ H_modo_en_sustancia ⟹ H_partes_de_modos ⟹ H_por_si_excluye_por_otro ⟹
H_inherencia_es_causalidad ⟹ H_modo_de_atributo_es_modo ⟹ H_limitar_es_determinar ⟹
H_lo_que_limita_es_finito ⟹ H_absoluto_solo_de_Dios ⟹ H_mediato_de_Dios_y_su_modificacion ⟹
H_dos_maneras_de_seguirse ⟹ H_de_lo_finito_lo_finito ⟹ modo m ⟹ ¬ finita_modo m ⟹ (∃a.
SigueDeAtributoAbsoluto m a) ∨ (∃y. SigueDeModificacion m y ∧ ¬ finita_modo y)"
  nitpick [timeout = 60]
  sledgehammer [timeout = 30] (A1_excl_implica_A1 premisas_P15_def P15_por_premisas P16_cor1 P17 P17_cor2
P24 P14 sustancia_no_modo todo_sustancia_o_modo libre_def finita_modo_def modo_def sustancia_def D6_def
H_modo_de_atributo_es_modo_def H_limitar_es_determinar_def H_lo_que_limita_es_finito_def
H_absoluto_solo_de_Dios_def H_mediato_de_Dios_y_su_modificacion_def H_dos_maneras_de_seguirse_def
H_de_lo_finito_lo_finito_def)
  oops

lemma P28_con_todo:
  "A1_excluyente ⟹ A2 ⟹ A4 ⟹ A5 ⟹ A7 ⟹ D1 ⟹ D4 ⟹ D6 ⟹ H_conversa_A5 ⟹
```

```
H_quien_comparte_uno_comparte_todos ⟹ H_individuacion_por_atributos ⟹ H_ser_en_implica_concebirse ⟹
H_solo_en_si_solo_por_si ⟹ H_atributo_compartido_es_comun ⟹ H_todo_tiene_causa ⟹
H_realidad_es_atributos ⟹ H_puente_atributo_sustancia ⟹ H_todo_por_sus_partes ⟹
H_sustancia_con_atributo ⟹ H_modo_en_sustancia ⟹ H_partes_de_modos ⟹ H_por_si_excluye_por_otro ⟹
H_inherencia_es_causalidad ⟹ H_modo_de_atributo_es_modo ⟹ H_limitar_es_determinar ⟹
H_lo_que_limita_es_finito ⟹ H_absoluto_solo_de_Dios ⟹ H_mediato_de_Dios_y_su_modificacion ⟹
H_dos_maneras_de_seguirse ⟹ H_de_lo_finito_lo_finito ⟹ modo m ⟹ finita_modo m ⟹ ∃y. y ≠ m ∧ modo y
∧ finita_modo y ∧ Causa y m"
  nitpick [timeout = 60]
  sledgehammer [timeout = 30] (A1_excl_implica_A1 premisas_P15_def P15_por_premisas P16_cor1 P17 P17_cor2
P24 P14 sustancia_no_modo todo_sustancia_o_modo libre_def finita_modo_def modo_def sustancia_def D6_def
H_modo_de_atributo_es_modo_def H_limitar_es_determinar_def H_lo_que_limita_es_finito_def
H_absoluto_solo_de_Dios_def H_mediato_de_Dios_y_su_modificacion_def H_dos_maneras_de_seguirse_def
H_de_lo_finito_lo_finito_def)
  oops

lemma P32_con_todo:
  "A1_excluyente ⟹ A2 ⟹ A4 ⟹ A5 ⟹ A7 ⟹ D1 ⟹ D4 ⟹ D6 ⟹ H_conversa_A5 ⟹
H_quien_comparte_uno_comparte_todos ⟹ H_individuacion_por_atributos ⟹ H_ser_en_implica_concebirse ⟹
H_solo_en_si_solo_por_si ⟹ H_atributo_compartido_es_comun ⟹ H_todo_tiene_causa ⟹
H_realidad_es_atributos ⟹ H_puente_atributo_sustancia ⟹ H_todo_por_sus_partes ⟹
H_sustancia_con_atributo ⟹ H_modo_en_sustancia ⟹ H_partes_de_modos ⟹ H_por_si_excluye_por_otro ⟹
H_inherencia_es_causalidad ⟹ H_modo_de_atributo_es_modo ⟹ H_limitar_es_determinar ⟹
H_lo_que_limita_es_finito ⟹ H_absoluto_solo_de_Dios ⟹ H_mediato_de_Dios_y_su_modificacion ⟹
H_dos_maneras_de_seguirse ⟹ H_de_lo_finito_lo_finito ⟹ modo Voluntad ⟹ ¬ libre Voluntad"
  nitpick [timeout = 60]
  sledgehammer [timeout = 30] (A1_excl_implica_A1 premisas_P15_def P15_por_premisas P16_cor1 P17 P17_cor2
P24 P14 sustancia_no_modo todo_sustancia_o_modo libre_def finita_modo_def modo_def sustancia_def D6_def
H_modo_de_atributo_es_modo_def H_limitar_es_determinar_def H_lo_que_limita_es_finito_def
H_absoluto_solo_de_Dios_def H_mediato_de_Dios_y_su_modificacion_def H_dos_maneras_de_seguirse_def
H_de_lo_finito_lo_finito_def)
  oops

section ‹No formalizables con este vocabulario›

text ‹Prop. 20: identidad de esencia y existencia: pide tratarlas como objetos.
  Prop. 25: Dios causa de la esencia, no solo de la existencia: pide la esencia como objeto.
  Prop. 27: una cosa determinada por Dios no puede hacerse indeterminada: pide tiempo o modalidad (y el
ax. 3).
  Prop. 30: el entendimiento y lo que comprende: pide formalizar el entendimiento.
  Prop. 31: el entendimiento pertenece a la naturaleza naturada: pide naturante/naturada y el
entendimiento.
  Prop. 33: las cosas no pudieron producirse de otro modo: lógica modal.
  Prop. 34: la potencia de Dios es su esencia: pide potencia y esencia como objetos.
  Prop. 35: lo que está en la potestad de Dios existe necesariamente: lógica modal.›

end
```

Spinoza: el núcleo mínimo

Una versión mínima con la lectura de identidad, en el orden de exposición de Spinoza (definiciones 1, 3 y 5, axioma 1 y proposición 15).

Fichero `Spinoza_identidad_minimal.thy` ▪ 77 resultados demostrados ▪ 1111 líneas

```
(* Versión NUEVA: núcleo mínimo con la lectura de IDENTIDAD,
   y ahora siguiendo el ORDEN de exposición de Spinoza (def. 1, 3, 5 → ax. 1 → P15).

   Relación con la Parte I de Spinoza:
   - def. 1 (causa sui) y def. 3 (in se)  →  se unifican en "depende x x"
   - def. 5 (modo = in alio)              →  modo
   - ax. 1 (todo es en sí o en otro)      →  A1_exhaustividad
   - P14 (no hay sustancia fuera de Dios) →  A_unicidad (ASUMIDO, no derivado)
   - P15 (todo es en Dios)                →  ahora DEMOSTRABLE
   Las P2-P6, P7/P11, P13 quedan absorbidas por A_unicidad y A_Dios.
   Las P3-P4, P8-P10, P12 piden vocabulario (común, partes, infinito) que aquí se omite. *)

theory Spinoza_identidad_minimal
  imports Main
begin

section ‹Vocabulario: un solo primitivo›

typedecl cosa
typedecl atributo

consts
  depende  :: "cosa ⇒ cosa ⇒ bool"      — ‹fundamentación: ser-en = concebirse-por = ser-causado-por›
  expresa  :: "cosa ⇒ atributo ⇒ bool"  — ‹many-to-many: NO individualiza›
  Extension   :: atributo
  Pensamiento :: atributo
  Dios        :: cosa

section ‹Definiciones (orden de Spinoza: def. 1, 3, 5)›

text ‹Def. 1 (causa sui) = lo que es en sí. En este vocabulario, «ser en sí» y «ser
  causa de sí» son literalmente lo mismo: depender de sí mismo.›
definition causa_sui :: "cosa ⇒ bool" where
  "causa_sui x ≡ depende x x"

text ‹Def. 5 (modo) = lo que es en otra cosa, y no en sí. La cláusula «y no en sí»
  es la exclusividad de Spinoza entre «in se» y «in alio»: hace que un modo no pueda ser
  causa de sí, y por tanto no sea eterno (véase P24).›
definition modo :: "cosa ⇒ bool" where
  "modo x ≡ ∃y. y ≠ x ∧ depende x y ∧ ¬ depende x x"

text ‹Def. 3 (sustancia) = lo que es en sí y no en otra cosa.›
definition sustancia :: "cosa ⇒ bool" where
  "sustancia x ≡ depende x x ∧ (∀y. depende x y ⟶ y = x)"

text ‹La causa se define a partir de la fundamentación (absorbe el ax. 4 de Spinoza).›
definition causa :: "cosa ⇒ cosa ⇒ bool" where
  "causa x y ≡ depende y x"

section ‹Axiomas›

text ‹Ax. 1 de Spinoza: todo es en sí (sustancia) o en otro (modo).›
definition A1_exhaustividad :: bool where
  "A1_exhaustividad ≡ ∀x. sustancia x ∨ modo x"

text ‹Def. 6 (Dios): la sustancia que expresa todos los atributos.›
definition A_Dios :: bool where
  "A_Dios ≡ sustancia Dios ∧ (∀a. expresa Dios a)"

text ‹P14 (no hay sustancia fuera de Dios): aquí ASUMIDO, no derivado.›
definition A_unicidad :: bool where
```

```
  "A_unicidad ≡ ∀s. sustancia s ⟶ s = Dios"

text ‹Todo modo se funda en Dios (la mitad «modos» de P15).›
definition A_fundamento :: bool where
  "A_fundamento ≡ ∀m. modo m ⟶ depende m Dios"

text ‹Extensión y Pensamiento son atributos distintos (pero de la única sustancia).›
definition A_atributos_distintos :: bool where
  "A_atributos_distintos ≡ Extension ≠ Pensamiento"

text ‹La lectura de identidad: todo modo expresa AMBOS atributos (un humano = un modo).›
definition A_identidad :: bool where
  "A_identidad ≡ ∀m. modo m ⟶ expresa m Extension ∧ expresa m Pensamiento"

section ‹Proposiciones en el orden de Spinoza›

text ‹P15: lo que es, es en Dios, y nada puede ser ni concebirse sin Dios.
  En este vocabulario: toda cosa depende de Dios (también Dios, que es causa sui).›
theorem P15:
  assumes "A1_exhaustividad" "A_unicidad" "A_fundamento" "A_Dios"
  shows "∀x. depende x Dios"
proof
  fix x
  have ex: "sustancia x ∨ modo x" using assms(1) unfolding A1_exhaustividad_def by blast
  show "depende x Dios"
  proof (cases "sustancia x")
    case True
    then have "x = Dios" using assms(2) unfolding A_unicidad_def by blast
    moreover have "depende Dios Dios" using assms(4) unfolding A_Dios_def sustancia_def by blast
    ultimately show ?thesis by simp
  next
    case False
    then have "modo x" using ex by blast
    then show ?thesis using assms(3) unfolding A_fundamento_def by blast
  qed
qed

section ‹Simetría de los atributos: idea y objeto, mente y cuerpo›

text ‹«La mente es la idea del cuerpo» y «el cuerpo es el objeto de la mente» son la
  MISMA relación invertida: un único modo expresado bajo dos atributos, sin que ninguno
  tenga prioridad sobre el otro. En esta lectura no hay una relación asimétrica «IdeaDe»
  que vaya de la idea al objeto: la correspondencia ES la identidad. Formalmente, para
  todo modo, ser-extenso y ser-pensante son equivalentes: no cabe uno sin el otro.›

theorem simetria_atributos:
  assumes "A_identidad" "modo m"
  shows "expresa m Extension ⟷ expresa m Pensamiento"
  using assms unfolding A_identidad_def by blast

text ‹Consecuencia: no puede existir un modo que sea cuerpo pero no mente...›
lemma no_asimetria_cuerpo_sin_mente:
  "A_identidad ∧ (∃m. modo m ∧ expresa m Extension ∧ ¬ expresa m Pensamiento)"
  nitpick [satisfy, timeout = 120]
  oops

text ‹... ni un modo que sea mente pero no cuerpo. La simetría queda garantizada.›
lemma no_asimetria_mente_sin_cuerpo:
  "A_identidad ∧ (∃m. modo m ∧ expresa m Pensamiento ∧ ¬ expresa m Extension)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹La gradación del ser y del concebir›

consts
  MasRealidad   :: "cosa ⇒ cosa ⇒ bool"  — ‹x tiene más potencia de ser que y›
  MasConcebible :: "cosa ⇒ cosa ⇒ bool"  — ‹x es más inteligible que y›
  Concebible    :: "cosa ⇒ bool"          — ‹x es concebible›
```

```
  Inconsistente :: "cosa ⇒ bool"          — ‹x es racionalmente inconsistente›

text ‹El principio (P9 + paralelismo, en grados): el orden del ser y el orden del
  concebir son el mismo. A más potencia de ser, más potencia de concepto.›
definition A_gradacion :: bool where
  "A_gradacion ≡ ∀x y. MasRealidad x y ⟷ MasConcebible x y"

text ‹El corte lógico: lo racionalmente inconsistente no tiene ser, luego no es
  concebible (su concebibilidad es cero, no mínima).›
definition A_cero :: bool where
  "A_cero ≡ ∀x. Inconsistente x ⟶ ¬ Concebible x"

text ‹Lo concebible es lo que está bajo el Pensamiento: la concebibilidad ES el
  aspecto mental de la cosa.›
definition A_concebible_es_pensamiento :: bool where
  "A_concebible_es_pensamiento ≡ ∀x. Concebible x ⟷ expresa x Pensamiento"

text ‹Dios es lo más real (ens realissimum).›
definition A_Dios_realissimo :: bool where
  "A_Dios_realissimo ≡ ∀m. modo m ⟶ MasRealidad Dios m"

text ‹Consecuencia: ningún modo es inconsistente (todo modo es real y, por tanto,
  concebible). Lo inconsistente solo habita fuera del ser.›
theorem modo_no_inconsistente:
  assumes "A_cero" "A_concebible_es_pensamiento" "A_identidad" "modo m"
  shows "¬ Inconsistente m"
  using assms unfolding A_cero_def A_concebible_es_pensamiento_def A_identidad_def by blast

text ‹La identidad con atributos distintos: un modo expresa DOS atributos genuinamente
  distintos (no uno solo). Es la versión positiva de lo que la lectura antigua niega.›
theorem modo_expresa_atributos_distintos:
  assumes "A_identidad" "A_atributos_distintos" "modo m"
  shows "expresa m Extension ∧ expresa m Pensamiento ∧ Extension ≠ Pensamiento"
  using assms unfolding A_identidad_def A_atributos_distintos_def by blast

text ‹De la gradación: lo más real es lo más concebible. Dios, que es lo más real,
  es lo más concebible (la cúspide del orden del ser y del concebir).›
theorem Dios_lo_mas_concebible:
  assumes "A_gradacion" "A_Dios_realissimo"
  shows "∀m. modo m ⟶ MasConcebible Dios m"
  using assms unfolding A_gradacion_def A_Dios_realissimo_def by blast

text ‹Todo el sistema junto con la gradación: ¿sigue siendo consistente?›
lemma consistencia_gradacion:
  "A1_exhaustividad ∧ A_Dios ∧ A_unicidad ∧ A_fundamento ∧ A_atributos_distintos ∧ A_identidad
   ∧ A_gradacion ∧ A_cero ∧ A_concebible_es_pensamiento ∧ A_Dios_realissimo
   ∧ (∃m1 m2. modo m1 ∧ modo m2 ∧ m1 ≠ m2)"
  nitpick [satisfy, timeout = 120]
  oops

text ‹El cero es un corte real: nada puede ser a la vez inconsistente y concebible.›
lemma cero_imposible:
  "A_cero ∧ (∃x. Inconsistente x ∧ Concebible x)"
  nitpick [satisfy, timeout = 120]
  oops

text ‹La jerarquía: puede haber dos modos, uno con más realidad y más concebibilidad
  que el otro (la piedra y el humano). Ambos están «bajo el Pensamiento» (A_identidad),
  pero en grados distintos: la piedra no «piensa» igual que el humano.›
lemma jerarquia_modos:
  "A_gradacion ∧ (∃m1 m2. modo m1 ∧ modo m2 ∧ m1 ≠ m2
     ∧ MasRealidad m2 m1 ∧ MasConcebible m2 m1)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹P16-P18: Dios como causa›

text ‹P16 cor. 1: Dios es causa eficiente de todas las cosas. En este vocabulario es
```

```
  inmediato de P15: «ser en Dios» ES «ser causado por Dios» (definición de causa). La
  premisa oculta H_inherencia_es_causalidad que Spinoza necesitaba aquí es gratis.›
theorem P16_cor1:
  assumes "A1_exhaustividad" "A_unicidad" "A_fundamento" "A_Dios"
  shows "∀x. causa Dios x"
  using P15[OF assms] unfolding causa_def by blast

text ‹P17: Dios obra por las solas leyes de su naturaleza, sin ser coaccionado por
  nadie. Formalmente: nada distinto de Dios causa a Dios.›
theorem P17:
  assumes "A_Dios"
  shows "∀y. causa y Dios ⟶ y = Dios"
  using assms unfolding A_Dios_def sustancia_def causa_def by blast

text ‹P18: Dios es causa inmanente, y no transitiva, de todas las cosas. La immanencia
  es automática: como causa = fundamentación, el efecto siempre «es en» su causa.›
definition causa_inmanente :: "cosa ⇒ cosa ⇒ bool" where
  "causa_inmanente y x ≡ causa y x ∧ depende x y"

definition causa_transitiva :: "cosa ⇒ cosa ⇒ bool" where
  "causa_transitiva y x ≡ causa y x ∧ ¬ depende x y"

theorem P18:
  assumes "A1_exhaustividad" "A_unicidad" "A_fundamento" "A_Dios"
  shows "∀x. causa_inmanente Dios x ∧ ¬ causa_transitiva Dios x"
  using P15[OF assms] unfolding causa_def causa_inmanente_def causa_transitiva_def by blast

section ‹P19 y P24: la eternidad›

text ‹Def. 8 (eternidad): la existencia misma, en cuanto se concibe que se sigue
  necesariamente de la sola definición de la cosa eterna. En este vocabulario, «eterno»
  es «causa sui»: existir por su propia naturaleza, depender de sí mismo. Las defs. 1
  (causa sui), 3 (in se) y 8 (eterno) se unifican en una sola: $depende x x$.›
definition eterno :: "cosa ⇒ bool" where
  "eterno x ≡ depende x x"

text ‹Un atributo es eterno si lo expresa una sustancia (que es, por definición, eterna).›
definition atributo_eterno :: "atributo ⇒ bool" where
  "atributo_eterno a ≡ ∃s. sustancia s ∧ expresa s a"

text ‹P19: Dios, o sea, todos los atributos de Dios, son eternos.›
theorem P19:
  assumes "A_Dios"
  shows "eterno Dios ∧ (∀a. atributo_eterno a)"
proof -
  have e: "eterno Dios" using assms unfolding A_Dios_def eterno_def sustancia_def by blast
  moreover have "∀a. atributo_eterno a"
    using assms unfolding A_Dios_def atributo_eterno_def by blast
  ultimately show ?thesis by blast
qed

text ‹P24: la esencia de las cosas producidas por Dios no implica la existencia.
  O sea: ningún modo es eterno; su ser depende de otro, no de sí mismo.›
theorem P24:
  assumes "modo m"
  shows "¬ eterno m"
  using assms unfolding modo_def eterno_def by blast

section ‹P21-P23: la doble eternidad y los modos infinitos›

consts
  SigueDeAtributoAbsoluto :: "cosa ⇒ atributo ⇒ bool"  — ‹x se sigue de la naturaleza absoluta de a›
  SigueDeModificacion     :: "cosa ⇒ cosa ⇒ bool"      — ‹x se sigue del atributo modificado por y›

text ‹La segunda eternidad: eterno POR EL ATRIBUTO (se sigue, inmediata o mediatamente,
  de un atributo eterno). Es distinta de la eternidad por la esencia (= causa sui), que
  solo Dios tiene. Es la «doble eternidad» de Spinoza.›
definition eterno_por_atributo :: "cosa ⇒ bool" where
```

```
  "eterno_por_atributo x ≡ (∃a. SigueDeAtributoAbsoluto x a)
     ∨ (∃y a. SigueDeModificacion x y ∧ SigueDeAtributoAbsoluto y a)"

text ‹Lo que se sigue de la naturaleza absoluta de un atributo es un modo cuya única
  causa es Dios (se sigue sin intermediarios).›
definition H_absoluto :: bool where
  "H_absoluto ≡ ∀x a. SigueDeAtributoAbsoluto x a ⟶ modo x ∧ (∀z. causa z x ⟶ z = Dios)"

text ‹Todo modo se sigue de Dios de una de las dos maneras: absoluta o mediata.›
definition H_dos_maneras :: bool where
  "H_dos_maneras ≡ ∀m. modo m ⟶ (∃a. SigueDeAtributoAbsoluto m a) ∨ (∃y. SigueDeModificacion m y)"

text ‹P21: lo que se sigue de la naturaleza absoluta de un atributo es eterno POR EL
  ATRIBUTO, pero NO por su esencia. Es la doble eternidad: los modos infinitos son
  eternos sin ser causa de sí.›
theorem P21:
  assumes "H_absoluto" "SigueDeAtributoAbsoluto x a"
  shows "eterno_por_atributo x ∧ ¬ eterno x"
proof
  show "eterno_por_atributo x" using assms(2) unfolding eterno_por_atributo_def by blast
  show "¬ eterno x"
  proof (rule P24)
    show "modo x" using assms unfolding H_absoluto_def by blast
  qed
qed

text ‹¿Es consistente que exista un modo que se sigue de la naturaleza absoluta
  (un modo infinito inmediato)?›
lemma consistencia_modo_infinito:
  "H_absoluto ∧ H_dos_maneras ∧ (∃x a. SigueDeAtributoAbsoluto x a)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹P22-P23: el modo infinito mediato y lo finito›

consts
  Limita :: "cosa ⇒ cosa ⇒ bool"  — ‹Limita y x: y limita a x›

text ‹Def. 2 (finito): es finito lo que puede ser limitado por otro de la misma
  naturaleza. Aquí «infinito» es, simplemente, no-finito.›
definition finita_modo :: "cosa ⇒ bool" where
  "finita_modo x ≡ ∃y. y ≠ x ∧ Limita y x"

text ‹Limitar es determinar (causar): lo que limita a una cosa la causa.›
definition H_limitar_es_determinar :: bool where
  "H_limitar_es_determinar ≡ ∀x y. Limita y x ⟶ causa y x"

text ‹Lo que limita a otra cosa es un modo finito.›
definition H_lo_que_limita_es_modo_finito :: bool where
  "H_lo_que_limita_es_modo_finito ≡ ∀x y. Limita y x ∧ y ≠ x ⟶ modo y ∧ finita_modo y"

text ‹Lo que se sigue de un atributo modificado por y tiene a y por causa (y solo a
  Dios o a y).›
definition H_mediato :: bool where
  "H_mediato ≡ ∀x y. SigueDeModificacion x y ⟶ modo y ∧ causa y x ∧ (∀z. causa z x ⟶ z = Dios ∨ z =
y)"

text ‹De lo finito se sigue lo finito.›
definition H_de_lo_finito_lo_finito :: bool where
  "H_de_lo_finito_lo_finito ≡ ∀m y. SigueDeModificacion m y ∧ finita_modo y ⟶ finita_modo m"

lemma modo_no_sustancia:
  assumes "modo x"
  shows "¬ sustancia x"
  using assms unfolding modo_def sustancia_def by blast

text ‹P22: lo que se sigue de un atributo modificado por una modificación infinita es
  también infinito (el modo infinito MEDIATO).›
```

```
theorem P22:
  assumes "H_limitar_es_determinar" "H_lo_que_limita_es_modo_finito" "H_mediato" "A_Dios"
          "SigueDeModificacion x y" "¬ finita_modo y"
  shows "¬ finita_modo x"
proof
  assume fx: "finita_modo x"
  then obtain w where "w ≠ x" and "Limita w x" unfolding finita_modo_def by blast
  have cwx: "causa w x" using ‹Limita w x› assms(1) unfolding H_limitar_es_determinar_def by blast
  have mw: "modo w" and fw: "finita_modo w"
    using ‹w ≠ x› ‹Limita w x› assms(2) unfolding H_lo_que_limita_es_modo_finito_def by blast+
  have wy: "w = Dios ∨ w = y" using cwx assms(3) assms(5) unfolding H_mediato_def by blast
  have "w ≠ Dios"
  proof
    assume "w = Dios"
    then have "sustancia w" using assms(4) unfolding A_Dios_def by blast
    then show False using mw modo_no_sustancia [of w] by blast
  qed
  then have "w = y" using wy by blast
  then have "finita_modo y" using fw by simp
  then show False using assms(6) by blast
qed

text ‹P23: todo modo infinito se sigue de la naturaleza absoluta de un atributo, o de un
  atributo modificado por una modificación infinita.›
theorem P23:
  assumes "H_dos_maneras" "H_de_lo_finito_lo_finito" "modo m" "¬ finita_modo m"
  shows "(∃a. SigueDeAtributoAbsoluto m a) ∨ (∃y. SigueDeModificacion m y ∧ ¬ finita_modo y)"
proof (cases "∃a. SigueDeAtributoAbsoluto m a")
  case True
  then show ?thesis by blast
next
  case False
  then have "∃y. SigueDeModificacion m y"
    using assms(1) assms(3) unfolding H_dos_maneras_def by blast
  then obtain y where sy: "SigueDeModificacion m y" by blast
  have "¬ finita_modo y"
  proof
    assume "finita_modo y"
    then have "finita_modo m" using sy assms(2) unfolding H_de_lo_finito_lo_finito_def by blast
    then show False using assms(4) by blast
  qed
  then show ?thesis using sy by blast
qed

text ‹¿Es consistente un modo infinito mediato (que se sigue de una modificación infinita)?›
lemma consistencia_modo_infinito_mediato:
  "H_mediato ∧ H_de_lo_finito_lo_finito ∧ H_dos_maneras ∧ H_limitar_es_determinar ∧
H_lo_que_limita_es_modo_finito
   ∧ (∃x y a. SigueDeModificacion x y ∧ SigueDeAtributoAbsoluto y a ∧ ¬ finita_modo y)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹P28: la cadena de causas finitas›

text ‹P28: toda cosa finita es determinada a existir y obrar por otra cosa finita, y
  esta a su vez por otra, y así al infinito. Aquí, la pieza central: todo modo finito
  tiene por causa a otro modo finito. Es la cadena HORIZONTAL de causas, que se apoya
  verticalmente en Dios (P15) pero no arranca de ningún modo finito primero.›
theorem P28:
  assumes "H_limitar_es_determinar" "H_lo_que_limita_es_modo_finito" "finita_modo m"
  shows "∃y. y ≠ m ∧ modo y ∧ finita_modo y ∧ causa y m"
proof -
  obtain y where "y ≠ m" and "Limita y m" using assms(3) unfolding finita_modo_def by blast
  moreover have "causa y m" using ‹Limita y m› assms(1) unfolding H_limitar_es_determinar_def by blast
  moreover have "modo y ∧ finita_modo y"
    using ‹y ≠ m› ‹Limita y m› assms(2) unfolding H_lo_que_limita_es_modo_finito_def by blast
  ultimately show ?thesis by blast
qed
```

```
text ‹¿Es consistente que haya un modo finito (limitado por otro)?›
lemma consistencia_modo_finito:
  "H_limitar_es_determinar ∧ H_lo_que_limita_es_modo_finito ∧ (∃m. finita_modo m)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹P29, P32, P33: lo modal (necesidad y libertad)›

consts
  ConcebibleComoNoExistente :: "cosa ⇒ bool"  — ‹x puede concebirse como no existente›
  Voluntad :: cosa

text ‹Lo contingente es lo que puede concebirse como no existente: ni necesario ni
  imposible, «podría ser o no ser».›
definition Contingente :: "cosa ⇒ bool" where
  "Contingente x ≡ ConcebibleComoNoExistente x"

text ‹A3 de Spinoza: de una causa determinada se sigue necesariamente el efecto; lo
  causado no puede concebirse como no existente.›
definition A3 :: bool where
  "A3 ≡ ∀x y. causa y x ⟶ ¬ ConcebibleComoNoExistente x"

text ‹Def. 7 (libre): lo que se determina a obrar por sí solo. Como «eterno» y «causa
  sui», en este vocabulario «libre» es «depender de sí mismo».›
definition Libre :: "cosa ⇒ bool" where
  "Libre x ≡ eterno x"

text ‹P29: nada es contingente; todo es determinado por la necesidad de la naturaleza
  divina. Aquí: todo tiene causa (P16 cor. 1), y lo causado es necesario (A3).›
theorem P29:
  assumes "A3" "A1_exhaustividad" "A_unicidad" "A_fundamento" "A_Dios"
  shows "∀x. ¬ Contingente x"
proof
  fix x
  have "causa Dios x" using P16_cor1[OF assms(2) assms(3) assms(4) assms(5)] by blast
  then have "¬ ConcebibleComoNoExistente x" using assms(1) unfolding A3_def by blast
  then show "¬ Contingente x" unfolding Contingente_def by blast
qed

text ‹P32: la voluntad no puede llamarse causa libre, sino solo necesaria. La voluntad
  es un modo; los modos no son causa de sí (P24); luego la voluntad no es libre.›
theorem P32:
  assumes "modo Voluntad"
  shows "¬ Libre Voluntad"
  using assms P24 unfolding Libre_def by blast

consts
  persona :: cosa
  roca :: cosa

text ‹Ser causa y ser libre son independientes: la persona causa la roca sin ser libre
  de haberla lanzado. Devolverá MODELO: causar no exige ser causa de sí. En la roca la
  persona es causa verdadera (P28 horizontal) sin ser causa libre (P32).›
lemma causa_sin_libertad:
  "causa persona roca ∧ modo persona ∧ ¬ Libre persona"
  nitpick [satisfy, timeout = 120]
  oops

text ‹P33: las cosas no pudieron producirse de otro modo. En este vocabulario, la
  necesidad del orden es la misma que la de la existencia (P29): nada es contingente.›
theorem P33:
  assumes "A3" "A1_exhaustividad" "A_unicidad" "A_fundamento" "A_Dios"
  shows "∀x. ¬ Contingente x"
  using P29[OF assms] by blast

text ‹¿Es consistente un mundo con voluntad y sin contingencia?›
lemma consistencia_modal:
```

```
  "A3 ∧ A1_exhaustividad ∧ A_Dios ∧ A_unicidad ∧ A_fundamento ∧ A_identidad
   ∧ modo Voluntad ∧ (∀x. ¬ Contingente x)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹P34, P35, P36: la potencia de Dios es su esencia›

text ‹La potencia es la capacidad de producir; Dios la tiene sobre todas las cosas.›
definition Potencia :: "cosa ⇒ bool" where
  "Potencia x ≡ ∀y. causa x y"

text ‹La eficacia es producir algún efecto; la tienen todas las cosas (P36).›
definition Eficaz :: "cosa ⇒ bool" where
  "Eficaz x ≡ ∃y. causa x y"

text ‹P34: la potencia de Dios es su misma esencia. Como «esencia» (ser en sí) y
  «potencia» (producir todo) son las dos caras de la relación depende, P34 sale gratis:
  la esencia de Dios produce necesariamente todas las cosas (P16 cor. 1).›
theorem P34:
  assumes "A1_exhaustividad" "A_unicidad" "A_fundamento" "A_Dios"
  shows "Potencia Dios"
  using P16_cor1[OF assms] unfolding Potencia_def by blast

text ‹P35: lo que concebimos en la potencia de Dios, necesariamente es. Es A3 (de una
  causa determinada se sigue necesariamente el efecto) aplicado a Dios.›
theorem P35:
  assumes "A3"
  shows "∀y. causa Dios y ⟶ ¬ ConcebibleComoNoExistente y"
  using assms unfolding A3_def by blast

text ‹Todo expresa algo (un atributo): Dios por ser sustancia, los modos por la identidad.›
lemma todo_expresa:
  assumes "A1_exhaustividad" "A_unicidad" "A_identidad" "A_Dios"
  shows "∀x. ∃a. expresa x a"
proof
  fix x
  have "sustancia x ∨ modo x" using assms(1) unfolding A1_exhaustividad_def by blast
  then show "∃a. expresa x a"
  proof
    assume "sustancia x"
    then have "x = Dios" using assms(2) unfolding A_unicidad_def by blast
    moreover have "∃a. expresa Dios a" using assms(4) unfolding A_Dios_def by blast
    ultimately show "∃a. expresa x a" by blast
  next
    assume "modo x"
    then have "expresa x Extension ∧ expresa x Pensamiento" using assms(3) unfolding A_identidad_def by
blast
    then show "∃a. expresa x a" by blast
  qed
qed

text ‹Expresar la naturaleza de Dios es producir efectos.›
definition H_expresar_es_producir :: bool where
  "H_expresar_es_producir ≡ ∀x. (∃a. expresa x a) ⟶ Eficaz x"

text ‹P36: nada existe de cuya naturaleza no se siga algún efecto. Todo es eficaz.›
theorem P36:
  assumes "A1_exhaustividad" "A_unicidad" "A_identidad" "A_Dios" "H_expresar_es_producir"
  shows "∀x. Eficaz x"
  using todo_expresa[OF assms(1) assms(2) assms(3) assms(4)] assms(5)
  unfolding H_expresar_es_producir_def by blast

text ‹¿Es consistente que haya eficacia (algo produce efectos)?›
lemma consistencia_potencia:
  "H_expresar_es_producir ∧ A1_exhaustividad ∧ A_Dios ∧ A_unicidad ∧ A_identidad ∧ A_fundamento
   ∧ (∃x. Eficaz x)"
  nitpick [satisfy, timeout = 120]
  oops
```

```
section ‹¿Se sostiene? Nitpick busca un modelo›

text ‹Todo el sistema junto, con al menos dos modos distintos.›
lemma consistencia:
  "A1_exhaustividad ∧ A_Dios ∧ A_unicidad ∧ A_fundamento ∧ A_atributos_distintos ∧ A_identidad
   ∧ (∃m. modo m) ∧ (∃m1 m2. modo m1 ∧ modo m2 ∧ m1 ≠ m2)"
  nitpick [satisfy, timeout = 120]
  oops

text ‹La identidad en concreto: existe un modo que es a la vez cuerpo y mente.›
lemma identidad_instanciada:
  "A1_exhaustividad ∧ A_Dios ∧ A_fundamento ∧ A_atributos_distintos ∧ A_identidad
   ∧ (∃m. modo m ∧ expresa m Extension ∧ expresa m Pensamiento)"
  nitpick [satisfy, timeout = 120]
  oops

text ‹Contraste: si volvemos a «un modo = un atributo», la identidad se vuelve imposible.›
definition un_atributo_por_modo :: bool where
  "un_atributo_por_modo ≡ ∀x a b. expresa x a ∧ expresa x b ⟶ a = b"

lemma contraste_lectura_antigua:
  "A_atributos_distintos ∧ un_atributo_por_modo
   ∧ (∃m. modo m ∧ expresa m Extension ∧ expresa m Pensamiento)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹Parte II: la naturaleza de la mente — identidad y paralelismo›

consts Idea :: "cosa ⇒ cosa ⇒ bool"  — ‹Idea x y: x es la idea (mente) de y (cuerpo)›

text ‹Cuerpo y mente NO son dos cosas: son las dos caras de un mismo modo. Un cuerpo es
  un modo bajo la Extensión; una mente, ese mismo modo bajo el Pensamiento.›
definition Cuerpo :: "cosa ⇒ bool" where
  "Cuerpo m ≡ expresa m Extension"

definition Mente :: "cosa ⇒ bool" where
  "Mente m ≡ expresa m Pensamiento"

text ‹II.7 (escolio) — la identidad: la idea de una cosa ES la cosa misma. No hay una
  «idea» separada que dependa asimétricamente del cuerpo: idea y cuerpo son un único modo.
  Por eso no reintroducimos el IdeaDe asimétrico que rompía la lectura.›
definition A_idea :: bool where
  "A_idea ≡ ∀x y. Idea x y ⟶ x = y ∧ expresa x Pensamiento ∧ expresa y Extension"

text ‹II.7 — el paralelismo es gratis: hay UNA sola relación depende (no una para
  cuerpos y otra para ideas), así que «el orden y la conexión de las ideas» y «el orden y
  la conexión de las cosas» son el mismo por construcción. No hace falta axioma.›

text ‹II.13 — la mente es la idea del cuerpo. En la lectura de identidad: cada modo es
  la idea de sí mismo; su cara mental es la idea de su cara extensa.›
theorem II13:
  assumes "A_idea" "Idea m m" "modo m"
  shows "Mente m ∧ Cuerpo m"
  using assms unfolding A_idea_def Mente_def Cuerpo_def by blast

text ‹II.13 (corolario) — el hombre consta de mente y cuerpo. Es A_identidad, ahora con
  nombres: el modo que es un cuerpo ES una mente (y al revés).›
theorem II13_cor:
  assumes "A_identidad" "modo m"
  shows "Mente m ∧ Cuerpo m"
  using assms unfolding A_identidad_def Mente_def Cuerpo_def by blast

text ‹La simetría mente/cuerpo: no hay mente sin cuerpo ni cuerpo sin mente. En la
  lectura de identidad no hay asimetría entre los atributos.›
theorem simetria_mente_cuerpo:
  assumes "A_identidad" "modo m"
  shows "Mente m ⟷ Cuerpo m"
```

```
  using assms unfolding A_identidad_def Mente_def Cuerpo_def by blast

text ‹¿Es consistente un modo que es a la vez mente (idea) y cuerpo, sin asimetría?›
lemma consistencia_mente_cuerpo:
  "A_identidad ∧ A_idea ∧ (∃m. modo m ∧ Idea m m ∧ Mente m ∧ Cuerpo m)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹Parte II (cont.): ideas adecuadas e inadecuadas — verdad y error›

text ‹II.def.4 — idea adecuada: aquella de la que la mente es causa adecuada (completa).
  En este vocabulario: la idea depende SOLO de la mente. La inadecuada tiene causa parcial
  (depende de algo externo).›
definition Adecuada :: "cosa ⇒ cosa ⇒ bool" where
  "Adecuada m x ≡ ∀z. depende x z ⟶ z = m"

definition Inadecuada :: "cosa ⇒ cosa ⇒ bool" where
  "Inadecuada m x ≡ ∃z. z ≠ m ∧ depende x z"

text ‹II.43 — verdad = adecuación: una idea es verdadera si y solo si es adecuada. La
  verdad se auto-certifica (quien la tiene no puede dudar).›
definition Verdadera :: "cosa ⇒ cosa ⇒ bool" where
  "Verdadera m x ≡ Adecuada m x"

text ‹II.41 — la falsedad es privación: solo las ideas inadecuadas son falsas.›
definition Falsa :: "cosa ⇒ cosa ⇒ bool" where
  "Falsa m x ≡ Inadecuada m x"

text ‹II.40 — los tres géneros de conocimiento: imaginación (inadecuada), razón e
  intuición (adecuadas).›
consts
  Imaginacion :: "cosa ⇒ bool"
  Razon :: "cosa ⇒ bool"
  Intuicion :: "cosa ⇒ bool"

definition A_generos :: bool where
  "A_generos ≡ (∀m x. Imaginacion x ⟶ Inadecuada m x)
             ∧ (∀m x. Razon x ∨ Intuicion x ⟶ Adecuada m x)"

text ‹Consecuencia (II.41): la imaginación es la única fuente de falsedad; la razón y la
  intuición no engañan.›
theorem imaginacion_fuente_de_falsedad:
  assumes "A_generos"
  shows "(∀m x. Imaginacion x ⟶ Falsa m x) ∧ (∀m x. Razon x ∨ Intuicion x ⟶ Verdadera m x)"
  using assms unfolding A_generos_def Falsa_def Verdadera_def by blast

section ‹Parte III: los afectos — conato y alegría/tristeza/deseo›

text ‹III.6 — el conato: cada cosa se esfuerza por perseverar en su ser. En este
  vocabulario, el conato ES la potencia (la eficacia de P36): perseverar en el ser es
  producir efectos, ser causa. III.7 — ese esfuerzo es la esencia actual de la cosa.›
consts
  Conato :: "cosa ⇒ bool"
  Alegria :: "cosa ⇒ bool"
  Tristeza :: "cosa ⇒ bool"
  Deseo :: "cosa ⇒ bool"

definition A_conato_es_potencia :: bool where
  "A_conato_es_potencia ≡ ∀x. Conato x ⟷ Eficaz x"

text ‹III.6 — todo se esfuerza por perseverar en su ser. Se DERIVA de P36 (todo es
  eficaz) unido a la identidad conato = potencia. La Parte III se apoya en la Parte I.›
theorem III6:
  assumes "A_conato_es_potencia" "A1_exhaustividad" "A_unicidad" "A_identidad" "A_Dios"
"H_expresar_es_producir"
  shows "∀x. Conato x"
  using P36[OF assms(2) assms(3) assms(4) assms(5) assms(6)] assms(1)
  unfolding A_conato_es_potencia_def Eficaz_def by blast
```

```
text ‹III.9 (esc.) — el deseo es el conato con conciencia de sí: quien desea, persevera.›
definition A_deseo_es_conato :: bool where
  "A_deseo_es_conato ≡ ∀x. Deseo x ⟶ Conato x"

text ‹III.11 — los tres afectos primitivos: alegría (paso a mayor potencia de obrar),
  tristeza (paso a menor), deseo (el conato mismo, consciente). Todos los demás afectos
  nacen de estos tres.›

text ‹Consecuencia (III.11 + III.9): quien desea, persevera (el deseo presupone el conato).›
theorem deseo_presupone_conato:
  assumes "A_deseo_es_conato"
  shows "∀x. Deseo x ⟶ Conato x"
  using assms unfolding A_deseo_es_conato_def by blast

text ‹¿Es consistente un ser con conato y afectos?›
lemma consistencia_afectos:
  "A_conato_es_potencia ∧ A_deseo_es_conato ∧ (∃x. Conato x ∧ Deseo x ∧ Alegria x)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹Parte III (cont.): amor, odio, y obrar/padecer›

text ‹III.13 — el amor es alegría acompañada de la idea de una causa exterior; el odio es
  tristeza con idea de causa exterior.›
consts
  Amor :: "cosa ⇒ cosa ⇒ bool"
  Odio :: "cosa ⇒ cosa ⇒ bool"

definition A_amor :: bool where
  "A_amor ≡ ∀x y. Amor x y ⟷ (Alegria x ∧ causa y x ∧ y ≠ x)"

definition A_odio :: bool where
  "A_odio ≡ ∀x y. Odio x y ⟷ (Tristeza x ∧ causa y x ∧ y ≠ x)"

text ‹Consecuencia: el amor presupone alegría; el odio, tristeza.›
theorem amor_presupone_alegria:
  assumes "A_amor" "A_odio"
  shows "(∀x y. Amor x y ⟶ Alegria x) ∧ (∀x y. Odio x y ⟶ Tristeza x)"
  using assms unfolding A_amor_def A_odio_def by blast

text ‹III.1-3 — obrar y padecer: el alma obra en cuanto es causa adecuada (ideas
  adecuadas) y padece en cuanto es causa parcial (ideas inadecuadas). Es el puente de la
  Parte II a la III y de aquí a la libertad de IV-V.›
consts
  Activo :: "cosa ⇒ bool"
  Pasivo :: "cosa ⇒ bool"

definition A_activo_pasivo :: bool where
  "A_activo_pasivo ≡ (∀m. Activo m ⟷ (∃x. Adecuada m x)) ∧ (∀m. Pasivo m ⟷ (∃x. Inadecuada m x))"

text ‹Consecuencia: obrar es ser causa adecuada (tener ideas adecuadas).›
theorem actividad_es_adecuacion:
  assumes "A_activo_pasivo"
  shows "∀m. Activo m ⟷ (∃x. Adecuada m x)"
  using assms unfolding A_activo_pasivo_def by blast

text ‹Una idea no puede ser a la vez adecuada e inadecuada (para la misma mente): la
  causa completa excluye la parcial.›
theorem adecuada_no_inadecuada:
  assumes "Adecuada m x"
  shows "¬ Inadecuada m x"
  using assms unfolding Adecuada_def Inadecuada_def by blast

text ‹Los afectos son modos del conato: amar, odiar, alegrarse y entristecerse son formas
  del esfuerzo por perseverar (III.9 esc.).›
definition A_afectos_de_conato :: bool where
  "A_afectos_de_conato ≡ ∀x. Alegria x ∨ Tristeza x ∨ Deseo x ⟶ Conato x"
```

```
text ‹Consecuencia: quien siente un afecto, persevera (todo afecto es una modificación del
  conato).›
theorem afecto_presupone_conato:
  assumes "A_afectos_de_conato"
  shows "∀x. Alegria x ∨ Tristeza x ∨ Deseo x ⟶ Conato x"
  using assms unfolding A_afectos_de_conato_def by blast

text ‹¿Es consistente un ser activo y pasivo a la vez, con amor y odio?›
lemma consistencia_afectos_completa:
  "A_amor ∧ A_odio ∧ A_activo_pasivo ∧ A_afectos_de_conato
   ∧ (∃x y. Amor x y ∧ Alegria x ∧ Conato x)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹Parte IV: la servidumbre humana — bien, mal y virtud›

text ‹IV.def.1-2 — bueno es lo que aumenta nuestra potencia de obrar (causa alegría);
  malo, lo que la disminuye (causa tristeza). El bien y el mal son relativos al conato.›
consts
  Bueno :: "cosa ⇒ cosa ⇒ bool"
  Malo :: "cosa ⇒ cosa ⇒ bool"

definition A_bueno :: bool where
  "A_bueno ≡ ∀m x. Bueno m x ⟷ (Alegria m ∧ causa x m)"

definition A_malo :: bool where
  "A_malo ≡ ∀m x. Malo m x ⟷ (Tristeza m ∧ causa x m)"

text ‹La alegría y la tristeza son opuestas (aumento vs disminución de la potencia).›
definition A_alegria_tristeza :: bool where
  "A_alegria_tristeza ≡ ∀x. Alegria x ⟷ ¬ Tristeza x"

text ‹Consecuencia: nada es a la vez bueno y malo (para el mismo ser).›
theorem bueno_no_malo:
  assumes "A_bueno" "A_malo" "A_alegria_tristeza"
  shows "∀m x. Bueno m x ⟶ ¬ Malo m x"
  using assms unfolding A_bueno_def A_malo_def A_alegria_tristeza_def by blast

text ‹IV.def.8 — la virtud es obrar según la propia naturaleza: es la potencia misma (la
  actividad, la causa adecuada).›
consts Virtud :: "cosa ⇒ bool"

definition A_virtud :: bool where
  "A_virtud ≡ ∀m. Virtud m ⟷ Activo m"

text ‹IV.24 — obrar por virtud es conservar el propio ser por la guía de la razón: la
  virtud es el conato obrando.›
definition A_virtud_es_conato :: bool where
  "A_virtud_es_conato ≡ ∀m. Virtud m ⟶ Conato m"

text ‹Consecuencia: quien obra por virtud persevera en su ser.›
theorem virtud_presupone_conato:
  assumes "A_virtud_es_conato"
  shows "∀m. Virtud m ⟶ Conato m"
  using assms unfolding A_virtud_es_conato_def by blast

text ‹¿Es consistente un ser virtuoso, con bien y mal?›
lemma consistencia_virtud:
  "A_bueno ∧ A_malo ∧ A_alegria_tristeza ∧ A_virtud ∧ A_virtud_es_conato
   ∧ (∃m. Virtud m ∧ Conato m)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹Parte V: la potencia del entendimiento — libertad y beatitud›

text ‹V.23 — el alma es eterna en cuanto concibe bajo la especie de la eternidad (por
  ideas adecuadas).›
```

```
consts ConcebidoSubEternidad :: "cosa ⇒ bool"

definition A_eternidad_del_alma :: bool where
  "A_eternidad_del_alma ≡ ∀m x. ConcebidoSubEternidad x ⟶ Adecuada m x"

text ‹Consecuencia: concebir bajo la eternidad es tener ideas adecuadas.›
theorem eternidad_es_adecuacion:
  assumes "A_eternidad_del_alma"
  shows "∀m x. ConcebidoSubEternidad x ⟶ Adecuada m x"
  using assms unfolding A_eternidad_del_alma_def by blast

text ‹V.32-33 — el amor intelectual de Dios nace del tercer género de conocimiento (la
  intuición) y es eterno.›
consts AmorIntelectual :: "cosa ⇒ cosa ⇒ bool"

definition A_amor_intelectual :: bool where
  "A_amor_intelectual ≡ ∀m x. AmorIntelectual m x ⟷ (Intuicion x ∧ Adecuada m x)"

text ‹Consecuencia: el amor intelectual de Dios es un conocimiento adecuado.›
theorem amor_intelectual_es_adecuado:
  assumes "A_amor_intelectual"
  shows "∀m x. AmorIntelectual m x ⟶ Adecuada m x"
  using assms unfolding A_amor_intelectual_def by blast

text ‹V.42 — la beatitud no es el premio de la virtud, sino la virtud misma.›
consts Beatitud :: "cosa ⇒ bool"

definition A_beatitud :: bool where
  "A_beatitud ≡ ∀m. Beatitud m ⟷ Virtud m"

text ‹La cadena final: adecuación (Parte II) = actividad (III) = virtud (IV) = beatitud
  (V). La libertad no es libre albedrío: es obrar por ideas adecuadas, y eso ES la
  beatitud. Cierra el libro entero.›
theorem libertad_es_beatitud:
  assumes "A_virtud" "A_beatitud"
  shows "∀m. Beatitud m ⟷ Activo m"
  using assms unfolding A_virtud_def A_beatitud_def by blast

text ‹¿Es consistente un alma eterna, con amor intelectual y beatitud?›
lemma consistencia_beatitud:
  "A_eternidad_del_alma ∧ A_amor_intelectual ∧ A_beatitud ∧ (∃m. Beatitud m ∧ Virtud m)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹Parte II (cont.): imaginación, nociones comunes y eternidad del entendimiento›

text ‹II.16 — la idea de una afección del cuerpo implica la naturaleza de un cuerpo
  exterior; por eso es inadecuada: su causa es parcial (externa).›
definition A_externo_es_inadecuado :: bool where
  "A_externo_es_inadecuado ≡ ∀m x. (∃z. z ≠ m ∧ causa z x) ⟶ Inadecuada m x"

theorem lo_externo_inadecuado:
  assumes "A_externo_es_inadecuado"
  shows "∀m x. (∃z. z ≠ m ∧ causa z x) ⟶ Inadecuada m x"
  using assms unfolding A_externo_es_inadecuado_def by blast

text ‹II.24 — el conocimiento de las partes es inadecuado: solo el todo se conoce
  adecuadamente.›
consts ParteDe :: "cosa ⇒ cosa ⇒ bool"

definition A_partes_inadecuadas :: bool where
  "A_partes_inadecuadas ≡ ∀m x y. ParteDe x y ∧ finita_modo x ⟶ Inadecuada m x"

theorem parte_es_inadecuada:
  assumes "A_partes_inadecuadas"
  shows "∀m x y. ParteDe x y ∧ finita_modo x ⟶ Inadecuada m x"
  using assms unfolding A_partes_inadecuadas_def by blast
```

```
text ‹II.38 — lo común a todas las cosas se concibe adecuadamente (es la razón).›
consts NocionComun :: "cosa ⇒ bool"

definition A_nociones_comunes :: bool where
  "A_nociones_comunes ≡ ∀m x. NocionComun x ⟶ Adecuada m x"

theorem nocion_comun_es_adecuada:
  assumes "A_nociones_comunes"
  shows "∀m x. NocionComun x ⟶ Adecuada m x"
  using assms unfolding A_nociones_comunes_def by blast

text ‹II.36 — las ideas inadecuadas (la imaginación) se siguen con la misma necesidad
  que todo lo demás: nada es contingente. Se deriva de P29.›
theorem imaginacion_necesaria:
  assumes "A3" "A1_exhaustividad" "A_unicidad" "A_fundamento" "A_Dios"
  shows "∀x. Imaginacion x ⟶ ¬ Contingente x"
  using P29[OF assms(1) assms(2) assms(3) assms(4) assms(5)] by blast

section ‹Parte III (cont.): conato, contrarios y afectos›

text ‹III.4 — ninguna cosa puede ser destruida sino por una causa exterior: nada se
  esfuerza hacia su propia destrucción.›
consts Autodestruccion :: "cosa ⇒ bool"

definition A_no_autodestruccion :: bool where
  "A_no_autodestruccion ≡ ∀x. ¬ Autodestruccion x"

theorem nada_se_autodestruye:
  assumes "A_no_autodestruccion"
  shows "∀x. ¬ Autodestruccion x"
  using assms unfolding A_no_autodestruccion_def by blast

text ‹III.5 — dos cosas son contrarias en cuanto una puede destruir a la otra.›
consts Destruye :: "cosa ⇒ cosa ⇒ bool"

definition Contrario :: "cosa ⇒ cosa ⇒ bool" where
  "Contrario x y ≡ Destruye x y ∨ Destruye y x"

text ‹III.6-8 — el conato no envuelve tiempo finito: perseverar es indefinido.›
consts Indefinido :: "cosa ⇒ bool"

definition A_conato_indefinido :: bool where
  "A_conato_indefinido ≡ ∀x. Conato x ⟶ Indefinido x"

theorem conato_indefinido:
  assumes "A_conato_indefinido" "A_conato_es_potencia" "A1_exhaustividad" "A_unicidad" "A_identidad"
"A_Dios" "H_expresar_es_producir"
  shows "∀x. Indefinido x"
  using III6[OF assms(2) assms(3) assms(4) assms(5) assms(6) assms(7)] assms(1)
  unfolding A_conato_indefinido_def by blast

text ‹III.9 — el deseo es el apetito con conciencia de sí; el apetito es el conato.›
consts Apetito :: "cosa ⇒ bool"

definition A_deseo_apetito :: bool where
  "A_deseo_apetito ≡ ∀x. Deseo x ⟷ Apetito x"

theorem deseo_es_apetito:
  assumes "A_deseo_apetito"
  shows "∀x. Deseo x ⟷ Apetito x"
  using assms unfolding A_deseo_apetito_def by blast

section ‹Parte IV (cont.): la razón, el bien supremo y el hombre libre›

text ‹IV.18 — el dictamen de la razón es conservar el propio ser: la razón no pide nada
  contrario a la naturaleza, pide perseverar.›
consts DictadoDeRazon :: "cosa ⇒ bool"
```

```
definition A_razon_conserva :: bool where
  "A_razon_conserva ≡ ∀m. DictadoDeRazon m ⟶ Conato m"

theorem la_razon_pide_conservarse:
  assumes "A_razon_conserva"
  shows "∀m. DictadoDeRazon m ⟶ Conato m"
  using assms unfolding A_razon_conserva_def by blast

text ‹IV.28 — el bien supremo del alma es el conocimiento de Dios; y ese conocimiento
  es la virtud misma.›
consts Conoce :: "cosa ⇒ cosa ⇒ bool"

definition A_sumo_bien :: bool where
  "A_sumo_bien ≡ ∀m. Conoce m Dios ⟷ Virtud m"

theorem conocer_a_dios_es_virtud:
  assumes "A_sumo_bien"
  shows "∀m. Conoce m Dios ⟷ Virtud m"
  using assms unfolding A_sumo_bien_def by blast

text ‹IV.35 — los hombres concuerdan en naturaleza en cuanto viven según la razón.›
consts Concuerda :: "cosa ⇒ cosa ⇒ bool"

definition A_concordancia :: bool where
  "A_concordancia ≡ ∀m n. Virtud m ∧ Virtud n ⟶ Concuerda m n"

theorem los_virtuosos_concuerdan:
  assumes "A_concordancia"
  shows "∀m n. Virtud m ∧ Virtud n ⟶ Concuerda m n"
  using assms unfolding A_concordancia_def by blast

text ‹IV.67 — el hombre libre piensa en todo menos en la muerte: su sabiduría es una
  meditación sobre la vida, no sobre la muerte.›
consts
  HombreLibre :: "cosa ⇒ bool"
  PiensaEn :: "cosa ⇒ cosa ⇒ bool"
  Muerte :: "cosa"

definition A_hombre_libre :: bool where
  "A_hombre_libre ≡ ∀m. HombreLibre m ⟶ ¬ PiensaEn m Muerte"

theorem el_hombre_libre_no_piensa_en_la_muerte:
  assumes "A_hombre_libre"
  shows "∀m. HombreLibre m ⟶ ¬ PiensaEn m Muerte"
  using assms unfolding A_hombre_libre_def by blast

text ‹El hombre libre es el virtuoso — y por la cadena, el beato: la libertad, la virtud
  y la beatitud son la misma cosa.›
definition A_hombre_libre_es_virtuoso :: bool where
  "A_hombre_libre_es_virtuoso ≡ ∀m. HombreLibre m ⟷ Virtud m"

theorem hombre_libre_es_beato:
  assumes "A_hombre_libre_es_virtuoso" "A_beatitud"
  shows "∀m. HombreLibre m ⟷ Beatitud m"
  using assms unfolding A_hombre_libre_es_virtuoso_def A_beatitud_def by blast

section ‹Parte V (cont.): entendimiento, amor intelectual y eternidad›

text ‹V.24 — cuanto más entendemos las cosas singulares, más entendemos a Dios.›
consts EntiendeParticulares :: "cosa ⇒ bool"

definition A_entender_es_conocer_a_dios :: bool where
  "A_entender_es_conocer_a_dios ≡ ∀m. EntiendeParticulares m ⟶ Conoce m Dios"

theorem entender_particulares_es_conocer_a_dios:
  assumes "A_entender_es_conocer_a_dios"
  shows "∀m. EntiendeParticulares m ⟶ Conoce m Dios"
  using assms unfolding A_entender_es_conocer_a_dios_def by blast
```

```
text ‹V.25 — el tercer género de conocimiento (la intuición) es el supremo: es un
  conocimiento adecuado. Se sigue de A_generos.›
theorem intuicion_es_adecuada:
  assumes "A_generos"
  shows "∀m x. Intuicion x ⟶ Adecuada m x"
  using assms unfolding A_generos_def by blast

text ‹V.36 — el amor intelectual del alma hacia Dios es parte del amor infinito con que
  Dios se ama a sí mismo.›
definition A_amor_de_dios :: bool where
  "A_amor_de_dios ≡ ∀m. AmorIntelectual m Dios ⟶ Amor Dios Dios"

theorem amor_intelectual_es_amor_de_dios:
  assumes "A_amor_de_dios"
  shows "∀m. AmorIntelectual m Dios ⟶ Amor Dios Dios"
  using assms unfolding A_amor_de_dios_def by blast

text ‹V.40 — cuanto más perfecta es una cosa, más obra (más activa es).›
consts MasPerfecto :: "cosa ⇒ cosa ⇒ bool"

definition A_perfeccion_activa :: bool where
  "A_perfeccion_activa ≡ ∀x y. MasPerfecto x y ⟶ Activo x"

theorem mas_perfecto_es_mas_activo:
  assumes "A_perfeccion_activa"
  shows "∀x y. MasPerfecto x y ⟶ Activo x"
  using assms unfolding A_perfeccion_activa_def by blast

text ‹¿Es consistente el libro ampliado (Partes II-V)?›
lemma consistencia_libro_ampliado:
  "A_externo_es_inadecuado ∧ A_nociones_comunes ∧ A_no_autodestruccion
   ∧ A_conato_indefinido ∧ A_razon_conserva ∧ A_sumo_bien ∧ A_hombre_libre
   ∧ A_amor_de_dios ∧ A_perfeccion_activa
   ∧ (∃m. HombreLibre m ∧ Beatitud m ∧ Conoce m Dios)"
  nitpick [satisfy, timeout = 120]
  oops

section ‹Parte III (cont.): la taxonomía de los afectos›

text ‹III.18-19 — la esperanza es alegría inconstante (de cosa dudosa); el miedo,
  tristeza inconstante.›
consts
  Esperanza :: "cosa ⇒ bool"
  Miedo :: "cosa ⇒ bool"
  Incierto :: "cosa ⇒ bool"

definition A_esperanza :: bool where
  "A_esperanza ≡ ∀x. Esperanza x ⟷ (Alegria x ∧ Incierto x)"

definition A_miedo :: bool where
  "A_miedo ≡ ∀x. Miedo x ⟷ (Tristeza x ∧ Incierto x)"

text ‹III.22-23 — la compasión es tristeza por el mal ajeno; la envidia, tristeza por el
  bien ajeno (un odio disfrazado).›
consts
  Compasion :: "cosa ⇒ bool"
  Envidia :: "cosa ⇒ bool"
  BienAjeno :: "cosa ⇒ bool"
  MalAjeno :: "cosa ⇒ bool"

definition A_compasion :: bool where
  "A_compasion ≡ ∀x. Compasion x ⟷ (Tristeza x ∧ MalAjeno x)"

definition A_envidia :: bool where
  "A_envidia ≡ ∀x. Envidia x ⟷ (Tristeza x ∧ BienAjeno x)"

text ‹III.11 (esc.) — la taxonomía entera: todo afecto (amor, odio, esperanza, miedo,
```

```
  compasión, envidia) se reduce a uno de los tres primitivos: alegría, tristeza o deseo.›
theorem taxonomia_de_los_afectos:
  assumes "A_esperanza" "A_miedo" "A_compasion" "A_envidia" "A_amor" "A_odio"
  shows "(∀x. Esperanza x ∨ Miedo x ∨ Compasion x ∨ Envidia x ⟶ Alegria x ∨ Tristeza x)
         ∧ (∀x y. Amor x y ∨ Odio x y ⟶ Alegria x ∨ Tristeza x)"
  using assms unfolding A_esperanza_def A_miedo_def A_compasion_def A_envidia_def A_amor_def A_odio_def
  by blast

text ‹III.58-59 — no hay tristeza activa: la tristeza nace de la pasividad (ideas
  inadecuadas); los afectos que nacen de la razón son solo alegría y deseo.›
definition A_sin_tristeza_activa :: bool where
  "A_sin_tristeza_activa ≡ ∀x. Tristeza x ⟶ Pasivo x"

theorem tristeza_es_pasiva:
  assumes "A_sin_tristeza_activa"
  shows "∀x. Tristeza x ⟶ Pasivo x"
  using assms unfolding A_sin_tristeza_activa_def by blast

definition A_activo_es_alegria_o_deseo :: bool where
  "A_activo_es_alegria_o_deseo ≡ ∀x. Activo x ⟶ Alegria x ∨ Deseo x"

theorem lo_activo_es_alegria_o_deseo:
  assumes "A_activo_es_alegria_o_deseo"
  shows "∀x. Activo x ⟶ Alegria x ∨ Deseo x"
  using assms unfolding A_activo_es_alegria_o_deseo_def by blast

text ‹¿Es consistente la taxonomía completa de los afectos?›
lemma consistencia_taxonomia:
  "A_esperanza ∧ A_miedo ∧ A_compasion ∧ A_envidia ∧ A_sin_tristeza_activa
   ∧ (∃x. Esperanza x ∧ Alegria x) ∧ (∃x. Miedo x ∧ Tristeza x)"
  nitpick [satisfy, timeout = 120]
  oops

end
```

Spinoza: diagnóstico del «en cuanto»

Qué premisa impide que un cuerpo y su idea sean lo mismo visto bajo dos atributos.

Fichero `Spinoza_diagnostico_quatenus.thy` · 6 resultados demostrados · 50 líneas

```
theory Spinoza_diagnostico_quatenus
  imports Spinoza_Etica_I
begin

(* Diagnóstico: ¿qué premisa impide el mundo con identidad de cuerpo e idea? *)

abbreviation identidad :: bool where
  "identidad ≡ ∃c1 c2. c1 ≠ c2 ∧ ModoDeAtributo c1 Extension ∧ ModoDeAtributo c1 Pensamiento
        ∧ ModoDeAtributo c2 Extension ∧ ModoDeAtributo c2 Pensamiento
        ∧ CausaBajo Extension Extension c1 c2 ∧ IdeaDe c1 c1 ∧ IdeaDe c2 c2"

abbreviation base_I :: bool where
  "base_I ≡ A1_excluyente ∧ H_por_si_excluye_por_otro ∧ A2 ∧ A4 ∧ A5 ∧ A7 ∧ D1 ∧ D4 ∧ D6
   ∧ H_conversa_A5 ∧ H_quien_comparte_uno_comparte_todos ∧ H_individuacion_por_atributos
   ∧ H_ser_en_implica_concebirse ∧ H_solo_en_si_solo_por_si ∧ H_atributo_compartido_es_comun
   ∧ H_todo_tiene_causa ∧ H_sustancia_con_atributo ∧ H_modo_en_sustancia ∧ H_inherencia_es_causalidad"

abbreviation modos_inf :: bool where
  "modos_inf ≡ H_modo_de_atributo_es_modo ∧ H_todo_modo_de_algun_atributo ∧ H_limitar_es_determinar
   ∧ H_lo_que_limita_es_finito ∧ H_absoluto_solo_de_Dios ∧ H_mediato_de_Dios_y_su_modificacion
   ∧ H_dos_maneras_de_seguirse ∧ H_de_lo_finito_lo_finito"

abbreviation mente_cuerpo :: bool where
  "mente_cuerpo ≡ H_atributos_distintos ∧ H_modo_se_concibe_por_su_atributo
   ∧ H_paralelismo ∧ H_ideas_del_pensamiento ∧ H_concebir_es_causar_ideas"

abbreviation quatenus :: bool where
  "quatenus ≡ A4_quatenus ∧ H_atributo_por_si_quatenus ∧ H_en_cuanto_requiere_atributo
   ∧ H_mismo_orden ∧ H_causa_relativa_es_causa"

lemma solo_quatenus: "quatenus ∧ H_atributos_distintos ∧ identidad"
  nitpick [satisfy, timeout = 120] oops

lemma quatenus_mente_cuerpo: "quatenus ∧ mente_cuerpo ∧ identidad"
  nitpick [satisfy, timeout = 120] oops

lemma quatenus_base: "quatenus ∧ H_atributos_distintos ∧ base_I ∧ identidad"
  nitpick [satisfy, timeout = 120] oops

lemma quatenus_modos: "quatenus ∧ H_atributos_distintos ∧ modos_inf ∧ identidad"
  nitpick [satisfy, timeout = 120] oops

lemma quatenus_base_modos: "quatenus ∧ H_atributos_distintos ∧ base_I ∧ modos_inf ∧ identidad"
  nitpick [satisfy, timeout = 120] oops

lemma quatenus_base_mente: "quatenus ∧ base_I ∧ mente_cuerpo ∧ identidad"
  nitpick [satisfy, timeout = 120] oops

end
```

Spinoza: premisas que sobran

Se comprueba si sobran de verdad las premisas que la mesa no echaba en falta: sobran todas, cada una por separado.

Fichero `Spinoza_redundancia.thy` ▪ 7 resultados demostrados ▪ 65 líneas

```
theory Spinoza_redundancia
  imports Spinoza_Etica_I
begin

(* Revisión de premisas (21-10-2026). El buscador de contramodelos del Laboratorio no encuentra
   contramodelo (hasta 4 cosas y 2 atributos) al quitar estas premisas de las pruebas de Isabelle.
   Aquí se comprueba si de verdad sobran (Sledgehammer) o si hay un contramodelo mayor (Nitpick).
   Resultado: sobran todas, cada una por separado (no a la vez). Las premisas reducidas tienen modelos
   (buscador del Laboratorio), así que las pruebas no salen de una contradicción. P18 y P26 salen de
   P16 cor. 1 con las mismas premisas. *)

abbreviation P15_sin_A1 :: bool where
  "P15_sin_A1 ≡ H_ser_en_implica_concebirse ∧ H_solo_en_si_solo_por_si
     ∧ H_modo_en_sustancia ∧ D6 ∧ H_sustancia_con_atributo
     ∧ H_individuacion_por_atributos ∧ H_quien_comparte_uno_comparte_todos"

abbreviation P15_sin_concebirse :: bool where
  "P15_sin_concebirse ≡ A1 ∧ H_solo_en_si_solo_por_si
     ∧ H_modo_en_sustancia ∧ D6 ∧ H_sustancia_con_atributo
     ∧ H_individuacion_por_atributos ∧ H_quien_comparte_uno_comparte_todos"

abbreviation P15_sin_solo_en_si :: bool where
  "P15_sin_solo_en_si ≡ A1 ∧ H_ser_en_implica_concebirse
     ∧ H_modo_en_sustancia ∧ D6 ∧ H_sustancia_con_atributo
     ∧ H_individuacion_por_atributos ∧ H_quien_comparte_uno_comparte_todos"

abbreviation P15_sin_modo_en_sustancia :: bool where
  "P15_sin_modo_en_sustancia ≡ A1 ∧ H_ser_en_implica_concebirse ∧ H_solo_en_si_solo_por_si
     ∧ D6 ∧ H_sustancia_con_atributo
     ∧ H_individuacion_por_atributos ∧ H_quien_comparte_uno_comparte_todos"

lemma R_P16_cor1_sin_A1:
  "P15_sin_A1 ⟹ A4 ⟹ H_todo_tiene_causa ⟹ H_inherencia_es_causalidad ⟹ Causa Dios x"
  by (metis H_todo_tiene_causa_def premisas_P15_def modos_causados_solo_por_Dios P18 A1_def modo_def
sustancia_def H_solo_en_si_solo_por_si_def causa_inmanente_def A4_def)

lemma R_P16_cor1_sin_concebirse:
  "P15_sin_concebirse ⟹ A4 ⟹ H_todo_tiene_causa ⟹ H_inherencia_es_causalidad ⟹ Causa Dios x"
  by (metis H_todo_tiene_causa_def P14 P15 P18 P11 A1_excl_implica_A1 modo_def sustancia_def
P15_por_premisas A1_excluyente_def sustancia_no_modo H_ser_en_implica_concebirse_def
H_modo_en_sustancia_def H_solo_en_si_solo_por_si_def A4_def H_inherencia_es_causalidad_def)

lemma R_P17_cor2_sin_A1:
  "P15_sin_A1 ⟹ A4 ⟹ D1 ⟹ H_todo_tiene_causa ⟹ H_por_si_excluye_por_otro ⟹ libre x ⟷ x = Dios"
  by (metis A4_def D1_def D6_def H_por_si_excluye_por_otro_def H_ser_en_implica_concebirse_def
H_solo_en_si_solo_por_si_def P14[of x] P7[of Dios] libre_def[of x] sustancia_def[of x])

lemma R_P17_cor2_sin_solo_en_si:
  "P15_sin_solo_en_si ⟹ A4 ⟹ D1 ⟹ H_todo_tiene_causa ⟹ H_por_si_excluye_por_otro ⟹ libre x ⟷ x
= Dios"
  by (metis P15 A1_excluyente_def A4_def H_individuacion_por_atributos_def H_sustancia_con_atributo_def
H_ser_en_implica_concebirse_def H_todo_tiene_causa_def D6_def H_por_si_excluye_por_otro_def
H_en_concibe_implica_debiles P5 sustancia_negativa_def libre_def D1_def sustancia_equiv_neg A2_def)

lemma R_P17_cor2_sin_modo_en_sustancia:
  "P15_sin_modo_en_sustancia ⟹ A4 ⟹ D1 ⟹ H_todo_tiene_causa ⟹ H_por_si_excluye_por_otro ⟹ libre
x ⟷ x = Dios"
  by (metis D6_def P14 P17 P24 P7 libre_def todo_sustancia_o_modo)

lemma R_P21e_sin_A4:
  "D6 ⟹ D1 ⟹ H_por_si_excluye_por_otro ⟹ H_modo_de_atributo_es_modo ⟹
```

```
   H_limitar_es_determinar ⟹ H_absoluto_solo_de_Dios ⟹ SigueDeAtributoAbsoluto x a ⟹
   ¬ finita_modo x ∧ eterno_por_atributo x ∧ ¬ eterno x"
  by (metis D1_def D6_def H_absoluto_solo_de_Dios_def H_modo_de_atributo_es_modo_def P21 eterno_def
eterno_por_atributo_def sustancia_no_modo)

lemma R_P21e_sin_por_si_excluye:
  "D6 ⟹ A4 ⟹ D1 ⟹ H_modo_de_atributo_es_modo ⟹
   H_limitar_es_determinar ⟹ H_absoluto_solo_de_Dios ⟹ SigueDeAtributoAbsoluto x a ⟹
   ¬ finita_modo x ∧ eterno_por_atributo x ∧ ¬ eterno x"
  using D1_def D6_def H_absoluto_solo_de_Dios_def H_modo_de_atributo_es_modo_def P21 eterno_def
eterno_por_atributo_def sustancia_no_modo by blast

end
```

Las cinco vías

Las cinco vías de Tomás de Aquino separadas en sus premisas: siete proposiciones se demuestran y dos no se siguen, con contramodelo.

Fichero `Cinco_vias.thy` · 12 resultados demostrados · 220 líneas

```
theory Cinco_vias
  imports Main
begin

(* Cuarto mundo del Laboratorio (22-10-2026, propuesta del profesor): «Las cinco vías» (Tomás de Aquino,
   Suma de teología I, q. 2, a. 3), sobre el núcleo de los mundos anteriores. Debate abierto: se muestra
qué
   premisa hace cada trabajo y dónde está cada objeción, sin veredicto sobre la existencia de Dios.
   Mismos nombres que la mesa web. *)

typedecl cosa

consts
  Mueve :: "cosa ⇒ cosa ⇒ bool"      (* x mueve a y: lo hace pasar de la potencia al acto (motus) *)
  Causa :: "cosa ⇒ cosa ⇒ bool"      (* x es causa eficiente de y *)
  ExisteEn :: "cosa ⇒ nat ⇒ bool"   (* tercera vía: x existe en el instante t *)
  MasPerfecto :: "cosa ⇒ cosa ⇒ bool"
  TiendeAFin :: "cosa ⇒ bool"
  Conoce :: "cosa ⇒ bool"
  Dirige :: "cosa ⇒ cosa ⇒ bool"

definition movido :: "cosa ⇒ bool" where "movido x ≡ ∃y. Mueve y x"
definition causado :: "cosa ⇒ bool" where "causado x ≡ ∃y. Causa y x"
definition contingente :: "cosa ⇒ bool" where "contingente x ≡ ∃t. ¬ ExisteEn x t"
definition necesario :: "cosa ⇒ bool" where "necesario x ≡ ∀t. ExisteEn x t"
definition maximo :: "cosa ⇒ bool" where "maximo x ≡ ∀y. y ≠ x ⟶ MasPerfecto x y"

(* ---------- Primera vía: el movimiento ---------- *)
definition V1_hay_movimiento :: bool where "V1_hay_movimiento ≡ ∃x. movido x"
definition V1_nada_se_mueve_a_si :: bool where "V1_nada_se_mueve_a_si ≡ ∀x. ¬ Mueve x x"
(* «No se puede proceder al infinito» en los motores POR SÍ (que mueven a la vez, como la mano el bastón):
   la relación «y mueve a x» está bien fundada, no hay cadenas descendentes infinitas. No es un principio
del tiempo. *)
definition V1_sin_regreso :: bool where "V1_sin_regreso ≡ wf {(y, x). Mueve y x}"

(* Hay un primer motor que no es movido por nada. *)
theorem via1:
  assumes "V1_hay_movimiento" "V1_sin_regreso"
  shows "∃x. ¬ movido x ∧ (∃z. Mueve x z)"
proof -
  let ?R = "{(y, x). Mueve y x}"
  obtain m where "movido m" using assms(1) unfolding V1_hay_movimiento_def by blast
  then obtain y0 where "Mueve y0 m" unfolding movido_def by blast
  let ?S = "{x. ∃z. Mueve x z}"
  have "y0 ∈ ?S" using ‹Mueve y0 m› by blast
  have "∃z∈?S. ∀y. (y, z) ∈ ?R ⟶ y ∉ ?S"
    using wf_eq_minimal[THEN iffD1, OF assms(2)[unfolded V1_sin_regreso_def], rule_format, OF ‹y0 ∈ ?S›] .
  then obtain x where "x ∈ ?S" and min: "∀y. (y, x) ∈ ?R ⟶ y ∉ ?S" by blast
  have "¬ movido x" using min unfolding movido_def by blast
  then show ?thesis using ‹x ∈ ?S› by blast
qed

(* «Nada se mueve a sí mismo» no hace falta suponerlo: sale de que no hay regreso al infinito. *)
theorem via1_irreflexiva:
  assumes "V1_sin_regreso"
  shows "¬ Mueve x x"
proof
  assume "Mueve x x"
  have w: "wf {(y, x). Mueve y x}" using assms unfolding V1_sin_regreso_def .
  have "(x, x) ∉ {(y, x). Mueve y x}" using w by (rule wf_not_refl)
  then show False using ‹Mueve x x› by simp
```

```
qed

(* Sin la prohibición del regreso al infinito, no se sigue: cada número lo mueve el siguiente. *)
lemma via1_sin_regreso_contramodelo:
  fixes M :: "nat ⇒ nat ⇒ bool"
  defines "M ≡ λy x. y = Suc x"
  shows "(∃x. ∃y. M y x) ∧ (∀x. ¬ M x x) ∧ ¬ (∃x. ¬ (∃y. M y x) ∧ (∃z. M x z))"
  unfolding M_def by simp

(* En un mundo finito, el regreso al infinito queda excluido solo si no hay círculos de motores. *)
lemma finito_sin_circulos_es_sin_regreso:
  assumes "finite (UNIV :: cosa set)" "acyclic {(y, x). Mueve y x}"
  shows "V1_sin_regreso"
proof -
  have "finite (UNIV :: (cosa × cosa) set)" using assms(1) by (simp add: finite_Prod_UNIV)
  then have "finite {(y, x). Mueve y x}" by (rule finite_subset[rotated]) simp
  then show ?thesis unfolding V1_sin_regreso_def using assms(2) by (simp add: finite_acyclic_wf)
qed

(* ---------- Segunda vía: la causa eficiente (la misma estructura) ---------- *)
definition V2_hay_causas :: bool where "V2_hay_causas ≡ ∃x. causado x"
definition V2_sin_regreso :: bool where "V2_sin_regreso ≡ wf {(y, x). Causa y x}"

theorem via2:
  assumes "V2_hay_causas" "V2_sin_regreso"
  shows "∃x. ¬ causado x ∧ (∃z. Causa x z)"
proof -
  obtain m y0 where "Causa y0 m" using assms(1) unfolding V2_hay_causas_def causado_def by blast
  let ?S = "{x. ∃z. Causa x z}"
  have "y0 ∈ ?S" using ‹Causa y0 m› by blast
  have "∃z∈?S. ∀y. (y, z) ∈ {(y, x). Causa y x} ⟶ y ∉ ?S"
    using wf_eq_minimal[THEN iffD1, OF assms(2)[unfolded V2_sin_regreso_def], rule_format, OF ‹y0 ∈ ?S›] .
  then obtain x where "x ∈ ?S" and min: "∀y. (y, x) ∈ {(y, x). Causa y x} ⟶ y ∉ ?S" by blast
  then show ?thesis unfolding causado_def by blast
qed

(* ---------- Tercera vía: lo contingente y lo necesario ---------- *)
definition V3_siempre_algo :: bool where "V3_siempre_algo ≡ ∀t. ∃x. ExisteEn x t"
(* El paso discutido: «si todo puede no ser, en algún momento no fue nada». *)
definition V3_paso_de_Tomas :: bool where "V3_paso_de_Tomas ≡ (∀x. contingente x) ⟶ (∃t. ∀x. ¬ ExisteEn
x t)"

(* Con el paso de Tomás, si siempre hay algo, hay algo necesario. *)
theorem via3:
  assumes "V3_siempre_algo" "V3_paso_de_Tomas"
  shows "∃x. necesario x"
proof (rule ccontr)
  assume "¬ (∃x. necesario x)"
  then have "∀x. contingente x" unfolding necesario_def contingente_def by simp
  then obtain t where vacio: "∀x. ¬ ExisteEn x t" using assms(2) unfolding V3_paso_de_Tomas_def by blast
  obtain x where "ExisteEn x t" using assms(1) unfolding V3_siempre_algo_def by blast
  then show False using vacio by simp
qed

(* Pero el paso es un salto de cuantificadores: que cada cosa deje de existir en algún instante no obliga
a que
   haya un instante en que no existe nada. Dos cosas que se relevan: la primera existe solo en 0, la
segunda desde 1. *)
lemma via3_salto:
  fixes E :: "bool ⇒ nat ⇒ bool"
  defines "E ≡ λx t. if x then t = 0 else t ≥ 1"
  shows "(∀x. ∃t. ¬ E x t) ∧ (∀t. ∃x. E x t) ∧ ¬ (∃x. ∀t. E x t)"
proof (intro conjI allI)
  fix x :: bool
  show "∃t. ¬ E x t"
  proof (cases x)
    case True then show ?thesis unfolding E_def by (intro exI [of _ 1]) simp
  next
```

```
    case False then show ?thesis unfolding E_def by (intro exI [of _ 0]) simp
  qed
next
  fix t :: nat
  show "∃x. E x t"
  proof (cases "t = 0")
    case True then show ?thesis unfolding E_def by (intro exI [of _ True]) simp
  next
    case False then show ?thesis unfolding E_def by (intro exI [of _ False]) simp
  qed
next
  show "¬ (∃x. ∀t. E x t)"
  proof
    assume "∃x. ∀t. E x t"
    then obtain x where h: "∀t. E x t" by blast
    then have "E x 0" and "E x 1" by blast+
    then show False unfolding E_def by (cases x) simp_all
  qed
qed

(* ---------- Cuarta vía: los grados ---------- *)
definition V4_orden :: bool where "V4_orden ≡ (∀x. ¬ MasPerfecto x x) ∧ (∀x y z. MasPerfecto x y ∧
MasPerfecto y z ⟶ MasPerfecto x z)"
definition V4_comparables :: bool where "V4_comparables ≡ ∀x y. x ≠ y ⟶ MasPerfecto x y ∨ MasPerfecto y
x"

(* Si el orden es estricto y todo es comparable, hay como mucho un máximo. *)
theorem via4_unico:
  assumes "V4_orden"
  shows "maximo x ∧ maximo y ⟶ x = y"
proof (intro impI, elim conjE, rule ccontr)
  assume mx: "maximo x" and my: "maximo y" and d: "x ≠ y"
  have "MasPerfecto x y" using mx d unfolding maximo_def by simp
  moreover have "MasPerfecto y x" using my d unfolding maximo_def by simp
  moreover have tr: "∀a b c. MasPerfecto a b ∧ MasPerfecto b c ⟶ MasPerfecto a c" using assms unfolding
V4_orden_def by (rule conjunct2)
  ultimately have "MasPerfecto x x" by blast
  then show False using assms unfolding V4_orden_def by simp
qed

(* En un mundo finito, si todo es comparable, hay un máximo. *)
theorem via4_finito:
  assumes fin: "finite (UNIV :: cosa set)" and "V4_orden" "V4_comparables"
  shows "∃x. maximo x"
proof -
  let ?R = "{(y, x). MasPerfecto y x}"
  have irr: "∀x. ¬ MasPerfecto x x" and tr: "∀x y z. MasPerfecto x y ∧ MasPerfecto y z ⟶ MasPerfecto x
z"
    using assms(2) unfolding V4_orden_def by blast+
  have "acyclic ?R"
  proof -
    have cl: "(a, b) ∈ ?R⇧+ ⟹ MasPerfecto a b" for a b
    proof (induction rule: trancl_induct)
      case (base y) then show ?case by simp
    next
      case (step y z) then show ?case using tr by blast
    qed
    show ?thesis unfolding acyclic_def using cl irr by blast
  qed
  moreover have "finite ?R"
  proof -
    have "finite (UNIV :: (cosa × cosa) set)" using fin by (simp add: finite_Prod_UNIV)
    then show ?thesis by (rule finite_subset[rotated]) simp
  qed
  ultimately have "wf ?R" by (simp add: finite_acyclic_wf)
  then have "∃z∈(UNIV :: cosa set). ∀y. (y, z) ∈ ?R ⟶ y ∉ UNIV"
    by (rule wf_eq_minimal[THEN iffD1, rule_format, OF _ UNIV_I])
  then obtain x where "∀y. (y, x) ∈ ?R ⟶ y ∉ (UNIV :: cosa set)" by blast
```

```
  then have "∀y. ¬ MasPerfecto y x" by simp
  then show ?thesis using assms(3) unfolding V4_comparables_def maximo_def by blast
qed

(* Que exista un máximo depende de que el mundo sea finito (o de otra premisa): con los naturales, siempre
hay uno más. *)
lemma via4_sin_maximo:
  fixes P :: "nat ⇒ nat ⇒ bool"
  defines "P ≡ λx y. y < x"
  shows "(∀x. ¬ P x x) ∧ (∀x y z. P x y ∧ P y z ⟶ P x z) ∧ (∀x y. x ≠ y ⟶ P x y ∨ P y x)
         ∧ ¬ (∃x. ∀y. y ≠ x ⟶ P x y)"
  unfolding P_def by (auto, metis Suc_n_not_le_n le_eq_less_or_eq lessI)

(* ---------- Quinta vía: el gobierno del mundo ---------- *)
definition V5_dirigido :: bool where "V5_dirigido ≡ ∀x. TiendeAFin x ∧ ¬ Conoce x ⟶ (∃y. Conoce y ∧
Dirige y x)"
definition V5_hay_fines :: bool where "V5_hay_fines ≡ ∃x. TiendeAFin x ∧ ¬ Conoce x"

(* Toda la fuerza está en la premisa: con ella, hay alguien que conoce y dirige. *)
theorem via5:
  assumes "V5_hay_fines" "V5_dirigido"
  shows "∃y. Conoce y ∧ (∃x. Dirige y x)"
  using assms unfolding V5_hay_fines_def V5_dirigido_def by blast

(* ---------- «Y a esto llamamos Dios»: ¿es lo mismo en las cinco? ---------- *)
(* Que el primer motor y la primera causa sean la misma cosa no se sigue: puede haber un primer motor y
otra primera causa. *)
lemma identidad_no_se_sigue:
  "V1_hay_movimiento ⟹ V1_nada_se_mueve_a_si ⟹ V2_hay_causas ⟹
   ¬ movido x ⟹ Mueve x z ⟹ ¬ causado y ⟹ Causa y w ⟹ x = y"
  nitpick [expect = genuine, timeout = 120]
  oops

end
```

Las antinomias de Kant: el comienzo y lo simple

La primera y la segunda antinomia: tesis, antítesis, el choque, la salida de Kant (contrarias, falsas las dos) y la otra salida.

Fichero `Antinomias_matematicas.thy` ▪ 15 resultados demostrados ▪ 331 líneas

```
theory Antinomias_matematicas
  imports Main
begin

(* Quinto mundo del Laboratorio (22-10-2026, propuesta del profesor): las antinomias de Kant. Aquí las
matemáticas;
   las dinámicas están en Antinomias_dinamicas.thy. Primero la primera (Crítica de la razón pura,
A426-433 /
   B454-461), solo en su parte del tiempo; la del espacio tiene la misma forma.
   - Tesis: el mundo tiene un comienzo en el tiempo. Kant: si no lo tuviera, hasta cada instante habría
transcurrido una
     serie infinita, y una serie infinita no puede completarse por síntesis sucesiva. Aquí, esa «síntesis
completa» se
     traduce como buena fundación de la serie de los estados pasados del mundo (interpretación; la mesa lo
dirá).
   - Antítesis: el mundo no tiene comienzo. Kant: si lo tuviera, lo precedería un tiempo vacío, y en un
tiempo vacío
     ningún instante se distingue de otro como condición para que algo empiece. Aquí: el tiempo no tiene
primer instante
     y el tiempo vacío es homogéneo.
   - Solución de Kant para las matemáticas (A501-507 / B529-535): tesis y antítesis no son contradictorias
sino
     contrarias, como «todo cuerpo huele bien» y «todo cuerpo huele mal»: las dos son falsas si los
cuerpos no huelen.
     Las dos presuponen que el mundo es un todo dado en el tiempo; con esa presuposición, una de las dos
vale; sin ella,
     ninguna.
   Pruebas paso a paso, sin auto ni presburger sobre aritmética. *)

section ‹Primera antinomia: el comienzo del mundo en el tiempo›

typedecl instante
consts Antes :: "instante ⇒ instante ⇒ bool"   (* Antes s t: el instante s es anterior a t *)
consts Mundo :: "instante ⇒ bool"                (* Mundo t: el mundo (un estado del mundo) existe en t
*)

(* El mundo empieza: hay un instante en que existe y antes del cual no existía. *)
definition comienzo :: bool where "comienzo ≡ ∃t0. Mundo t0 ∧ (∀s. Antes s t0 ⟶ ¬ Mundo s)"
(* El mundo existe y no tiene comienzo: antes de cada estado del mundo hubo otro. *)
definition sin_comienzo :: bool where
  "sin_comienzo ≡ (∃t. Mundo t) ∧ (∀t. Mundo t ⟶ (∃s. Antes s t ∧ Mundo s))"

(* Premisa propia de la tesis: la serie de los estados pasados del mundo está completa (sin regreso al
infinito). *)
definition T1_sintesis_completa :: bool where
  "T1_sintesis_completa ≡ wf {(s, t). Antes s t ∧ Mundo s ∧ Mundo t}"
(* Premisas propias de la antítesis: el tiempo no tiene primer instante, y el tiempo vacío es homogéneo:
si el mundo
   existe en t y antes de t no había nada, nada distingue a t de un instante vacío anterior s, y el mundo
tendría que
   existir también en s. *)
definition A1_tiempo_sin_principio :: bool where "A1_tiempo_sin_principio ≡ ∀t. ∃s. Antes s t"
definition A1_tiempo_vacio_homogeneo :: bool where
  "A1_tiempo_vacio_homogeneo ≡ ∀t s. Mundo t ∧ (∀u. Antes u t ⟶ ¬ Mundo u) ∧ Antes s t ⟶ Mundo s"

(* Tesis: con la síntesis completa, si el mundo existe en algún instante, tiene un comienzo. *)
theorem tesis1:
  assumes "T1_sintesis_completa" "∃t. Mundo t"
  shows "comienzo"
proof -
  let ?r = "{(s, t). Antes s t ∧ Mundo s ∧ Mundo t}" and ?S = "{t. Mundo t}"
```

```
  obtain a where "Mundo a" using assms(2) by blast
  then have "a ∈ ?S" by simp
  have "∃m∈?S. ∀s. (s, m) ∈ ?r ⟶ s ∉ ?S"
    using wf_eq_minimal[THEN iffD1, OF assms(1)[unfolded T1_sintesis_completa_def], rule_format, OF ‹a ∈ ?
S›] .
  then obtain m where m: "m ∈ ?S" and min: "∀s. (s, m) ∈ ?r ⟶ s ∉ ?S" by blast
  have mm: "Mundo m" using m by simp
  have "∀s. Antes s m ⟶ ¬ Mundo s"
  proof (intro allI impI notI)
    fix s assume "Antes s m" "Mundo s"
    then have "(s, m) ∈ ?r" using mm by simp
    then have "s ∉ ?S" using min by blast
    then show False using ‹Mundo s› by simp
  qed
  then show ?thesis unfolding comienzo_def using mm by blast
qed

(* Antítesis: si el tiempo no tiene principio y el tiempo vacío es homogéneo, el mundo no puede empezar.
*)
theorem antitesis1:
  assumes "A1_tiempo_sin_principio" "A1_tiempo_vacio_homogeneo"
  shows "¬ comienzo"
proof
  assume "comienzo"
  then obtain t0 where e: "Mundo t0" and v: "∀s. Antes s t0 ⟶ ¬ Mundo s" unfolding comienzo_def by blast
  obtain s where a: "Antes s t0" using assms(1) unfolding A1_tiempo_sin_principio_def by blast
  have "Mundo s" using assms(2) e v a unfolding A1_tiempo_vacio_homogeneo_def by blast
  moreover have "¬ Mundo s" using v a by blast
  ultimately show False by contradiction
qed

(* Si el mundo existe como un todo en el tiempo, empezar y no empezar son contradictorios: vale uno de los
dos. *)
theorem con_mundo_dado_contradictorias:
  assumes "∃t. Mundo t"
  shows "(comienzo ∨ sin_comienzo) ∧ ¬ (comienzo ∧ sin_comienzo)"
proof (rule conjI)
  show "comienzo ∨ sin_comienzo"
  proof (cases "∃m. Mundo m ∧ (∀s. Antes s m ⟶ ¬ Mundo s)")
    case True then show ?thesis unfolding comienzo_def by blast
  next
    case False
    then have "∀t. Mundo t ⟶ (∃s. Antes s t ∧ Mundo s)" by blast
    then show ?thesis unfolding sin_comienzo_def using assms by blast
  qed
  show "¬ (comienzo ∧ sin_comienzo)" unfolding comienzo_def sin_comienzo_def by blast
qed

(* Por eso, con el mundo dado, la antítesis da la otra mitad: sin comienzo. *)
corollary antitesis1_sin_comienzo:
  assumes "A1_tiempo_sin_principio" "A1_tiempo_vacio_homogeneo" "∃t. Mundo t"
  shows "sin_comienzo"
  using con_mundo_dado_contradictorias[OF assms(3)] antitesis1[OF assms(1) assms(2)] by blast

(* El choque: las premisas de la tesis y las de la antítesis, con el mundo dado, no caben juntas. *)
theorem choque1:
  assumes "T1_sintesis_completa" "A1_tiempo_sin_principio" "A1_tiempo_vacio_homogeneo" "∃t. Mundo t"
  shows False
  using tesis1[OF assms(1) assms(4)] antitesis1[OF assms(2) assms(3)] by blast

(* La solución de Kant: sin la presuposición de que el mundo es un todo dado en el tiempo, tesis y
antítesis son
   contrarias y caen las dos (como «todo cuerpo huele bien» y «todo cuerpo huele mal» si los cuerpos no
huelen). *)
theorem sin_mundo_dado_contrarias:
  assumes "¬ (∃t. Mundo t)"
  shows "¬ comienzo ∧ ¬ sin_comienzo"
  using assms unfolding comienzo_def sin_comienzo_def by blast
```

```
(* Cada lado tiene su modelo, con los enteros como instantes.
   Tesis: el mundo existe desde el 0; la serie de sus estados pasados está bien fundada y empieza en 0.
   (La homogeneidad del tiempo vacío falla: el -1 está vacío y el 0 no.) *)
lemma modelo_tesis1:
  fixes M :: "int ⇒ bool"
  defines "M ≡ λt. 0 ≤ t"
  shows "wf {(s, t). s < t ∧ M s ∧ M t} ∧ (∃t0. M t0 ∧ (∀s. s < t0 ⟶ ¬ M s))"
proof (rule conjI)
  have "{(s, t). s < t ∧ M s ∧ M t} ⊆ inv_image less_than nat"
  proof (rule subsetI)
    fix p assume "p ∈ {(s, t). s < t ∧ M s ∧ M t}"
    then obtain s t where p: "p = (s, t)" and st: "s < t" and s0: "0 ≤ s" unfolding M_def by blast
    have "nat s < nat t" using st s0 by (simp add: nat_less_eq_zless)
    then show "p ∈ inv_image less_than nat" unfolding p by simp
  qed
  then show "wf {(s, t). s < t ∧ M s ∧ M t}" by (rule wf_subset [OF wf_inv_image [OF wf_less_than]])
  show "∃t0. M t0 ∧ (∀s. s < t0 ⟶ ¬ M s)"
  proof (rule exI [of _ 0])
    show "M 0 ∧ (∀s. s < 0 ⟶ ¬ M s)" unfolding M_def by simp
  qed
qed

(* Antítesis: el mundo existe siempre; el tiempo no tiene principio, la homogeneidad se cumple (nunca hay
un instante
   con el mundo y nada antes) y no hay comienzo; la síntesis completa falla: hay regreso sin fin. *)
lemma modelo_antitesis1:
  fixes M :: "int ⇒ bool"
  defines "M ≡ λt. True"
  shows "(∀t::int. ∃s. s < t) ∧ (∀t s. M t ∧ (∀u. u < t ⟶ ¬ M u) ∧ s < t ⟶ M s)
         ∧ ¬ (∃t0. M t0 ∧ (∀s. s < t0 ⟶ ¬ M s)) ∧ ¬ wf {(s, t). s < t ∧ M s ∧ M t}"
proof (intro conjI)
  show "∀t::int. ∃s. s < t"
  proof
    fix t :: int
    show "∃s. s < t" by (rule exI [of _ "t - 1"]) simp
  qed
  show "∀t s. M t ∧ (∀u. u < t ⟶ ¬ M u) ∧ s < t ⟶ M s" unfolding M_def by simp
  show "¬ (∃t0. M t0 ∧ (∀s. s < t0 ⟶ ¬ M s))"
  proof
    assume "∃t0. M t0 ∧ (∀s. s < t0 ⟶ ¬ M s)"
    then obtain t0 where "∀s. s < t0 ⟶ ¬ M s" by blast
    then have "¬ M (t0 - 1)" by simp
    then show False unfolding M_def by simp
  qed
  show "¬ wf {(s, t). s < t ∧ M s ∧ M t}"
  proof
    assume w: "wf {(s, t). s < t ∧ M s ∧ M t}"
    have h: "∀i. (- int (Suc i), - int i) ∈ {(s, t). s < t ∧ M s ∧ M t}" unfolding M_def by simp
    have "∃f. ∀i. (f (Suc i), f i) ∈ {(s, t). s < t ∧ M s ∧ M t}"
    proof (rule exI [of _ "λn. - int n"])
      show "∀i. ((λn. - int n) (Suc i), (λn. - int n) i) ∈ {(s, t). s < t ∧ M s ∧ M t}" using h by simp
    qed
    then show False using w unfolding wf_iff_no_infinite_down_chain by blast
  qed
qed

section ‹Segunda antinomia: lo simple (A434-443 / B462-471)›

(* - Tesis: toda sustancia compuesta consta de partes simples. Kant: la composición es solo una relación
accidental
     entre sustancias; si se suprimiera toda composición, algo tendría que quedar, y eso serían partes
simples. Aquí:
     la división en partes acaba (la relación «ser parte propia de» está bien fundada), más la
transitividad.
   - Antítesis: nada en el mundo es simple. Kant: todo lo compuesto ocupa un espacio, el espacio no consta
de partes
     simples sino de espacios, y lo que ocupa un espacio tiene partes unas fuera de otras. Aquí: lo que
```

```
está en el
     espacio tiene partes que también están en el espacio.
   - Diferencia con la primera: aquí tesis y antítesis no son contradictorias ni siquiera dando por hecho
que hay
     compuestos. Un mundo mixto (algo simple y un compuesto sin partes simples) las hace falsas a las dos:
son
     contrarias, como dice Kant. Su razón es que la división de un fenómeno solo va «al indefinido»: ni se
llega a
     partes simples ni están dadas infinitas partes (A523-527 / B551-555); eso ya no es lógica, y la mesa
lo dirá. *)

typedecl porcion
consts Parte :: "porcion ⇒ porcion ⇒ bool"   (* Parte y x: y es una parte propia de x *)
consts EnEspacio :: "porcion ⇒ bool"           (* x ocupa un espacio: es un fenómeno *)

definition simple :: "porcion ⇒ bool" where "simple x ≡ ¬ (∃y. Parte y x)"
definition compuesto :: "porcion ⇒ bool" where "compuesto x ≡ ∃y. Parte y x"

(* Premisa de todo el mundo: la parte de una parte es parte. *)
definition M_transitiva :: bool where "M_transitiva ≡ ∀x y z. Parte x y ∧ Parte y z ⟶ Parte x z"
(* Premisa propia de la tesis: la división acaba, no hay partes de partes sin fin. *)
definition T2_division_acaba :: bool where "T2_division_acaba ≡ wf {(y, x). Parte y x}"
(* Premisa propia de la antítesis: lo que está en el espacio tiene partes que están en el espacio. *)
definition A2_espacio_divisible :: bool where
  "A2_espacio_divisible ≡ ∀x. EnEspacio x ⟶ (∃y. Parte y x ∧ EnEspacio y)"

(* Tesis: si la división acaba, todo compuesto tiene alguna parte simple. *)
theorem tesis2:
  assumes "T2_division_acaba" "M_transitiva"
  shows "∀x. compuesto x ⟶ (∃s. simple s ∧ Parte s x)"
proof (intro allI impI)
  fix x assume "compuesto x"
  then obtain y0 where "Parte y0 x" unfolding compuesto_def by blast
  let ?r = "{(y, x). Parte y x}" and ?S = "{y. Parte y x}"
  have "y0 ∈ ?S" using ‹Parte y0 x› by simp
  have "∃m∈?S. ∀z. (z, m) ∈ ?r ⟶ z ∉ ?S"
    using wf_eq_minimal[THEN iffD1, OF assms(1)[unfolded T2_division_acaba_def], rule_format, OF ‹y0 ∈ ?
S›] .
  then obtain m where m: "m ∈ ?S" and min: "∀z. (z, m) ∈ ?r ⟶ z ∉ ?S" by blast
  have pm: "Parte m x" using m by simp
  have "simple m" unfolding simple_def
  proof
    assume "∃z. Parte z m"
    then obtain z where zm: "Parte z m" by blast
    then have "Parte z x" using pm assms(2) unfolding M_transitiva_def by blast
    then have "z ∈ ?S" by simp
    moreover have "(z, m) ∈ ?r" using zm by simp
    ultimately show False using min by blast
  qed
  then show "∃s. simple s ∧ Parte s x" using pm by blast
qed

(* Antítesis: si lo que está en el espacio es divisible en el espacio, nada en el espacio es simple. *)
theorem antitesis2:
  assumes "A2_espacio_divisible"
  shows "¬ (∃s. EnEspacio s ∧ simple s)"
  using assms unfolding A2_espacio_divisible_def simple_def by blast

(* El choque: si algo ocupa un espacio, la división que acaba y el espacio divisible no caben juntos. *)
theorem choque2:
  assumes "T2_division_acaba" "A2_espacio_divisible" "∃x. EnEspacio x"
  shows False
proof -
  let ?r = "{(y, x). Parte y x}" and ?S = "{x. EnEspacio x}"
  obtain a where "EnEspacio a" using assms(3) by blast
  then have "a ∈ ?S" by simp
  have "∃m∈?S. ∀z. (z, m) ∈ ?r ⟶ z ∉ ?S"
    using wf_eq_minimal[THEN iffD1, OF assms(1)[unfolded T2_division_acaba_def], rule_format, OF ‹a ∈ ?
```

```
S›] .
  then obtain m where m: "m ∈ ?S" and min: "∀z. (z, m) ∈ ?r ⟶ z ∉ ?S" by blast
  have "EnEspacio m" using m by simp
  then obtain y where "Parte y m" and "EnEspacio y" using assms(2) unfolding A2_espacio_divisible_def by
blast
  then show False using min by simp
qed

(* Modelo de la tesis: los naturales, con «y es parte de x» si y < x. El 0 es simple y es parte de todo
compuesto. *)
lemma modelo_tesis2:
  fixes P :: "nat ⇒ nat ⇒ bool"
  defines "P ≡ λy x. y < x"
  shows "wf {(y, x). P y x} ∧ (∀x y z. P x y ∧ P y z ⟶ P x z)
         ∧ (∀x. (∃y. P y x) ⟶ (∃s. ¬ (∃y. P y s) ∧ P s x))"
proof (intro conjI)
  show "wf {(y, x). P y x}" unfolding P_def by (rule wf_less)
  show "∀x y z. P x y ∧ P y z ⟶ P x z" unfolding P_def by simp
  show "∀x. (∃y. P y x) ⟶ (∃s. ¬ (∃y. P y s) ∧ P s x)"
  proof (intro allI impI)
    fix x assume "∃y. P y x"
    then obtain y where "y < x" unfolding P_def by blast
    then have "0 < x" by simp
    show "∃s. ¬ (∃y. P y s) ∧ P s x"
    proof (rule exI [of _ 0])
      show "¬ (∃y. P y 0) ∧ P 0 x" unfolding P_def using ‹0 < x› by simp
    qed
  qed
qed

(* Modelo de la antítesis: los naturales, con «y es parte de x» si x < y (cada pieza se parte en otra más
pequeña,
   x + 1, sin fin). Todo está en el espacio, nada es simple y la división no acaba. *)
(* Con la premisa de la tesis, la otra salida (todo divisible sin fin) es imposible: si la división acaba,
algo es
   simple. Es lo que soltaría quien, como Schopenhauer, solo da por buenas las pruebas de las antítesis.
*)
lemma division_acaba_hay_simple:
  assumes "T2_division_acaba"
  shows "∃x. simple x"
proof -
  have "∃m∈UNIV. ∀y. (y, m) ∈ {(y, x). Parte y x} ⟶ y ∉ UNIV"
    using wf_eq_minimal[THEN iffD1, OF assms[unfolded T2_division_acaba_def], rule_format, OF UNIV_I] .
  then obtain m where "∀y. (y, m) ∈ {(y, x). Parte y x} ⟶ y ∉ UNIV" by blast
  then have "simple m" unfolding simple_def by simp
  then show ?thesis by blast
qed

lemma modelo_antitesis2:
  fixes P :: "nat ⇒ nat ⇒ bool"
  defines "P ≡ λy x. x < y"
  shows "(∀x y z. P x y ∧ P y z ⟶ P x z) ∧ (∀x. ∃y. P y x) ∧ ¬ wf {(y, x). P y x}"
proof (intro conjI)
  show "∀x y z. P x y ∧ P y z ⟶ P x z" unfolding P_def by simp
  show "∀x. ∃y. P y x"
  proof
    fix x :: nat
    show "∃y. P y x" unfolding P_def by (rule exI [of _ "Suc x"]) simp
  qed
  show "¬ wf {(y, x). P y x}"
  proof
    assume w: "wf {(y, x). P y x}"
    have "∃f. ∀i. (f (Suc i), f i) ∈ {(y, x). P y x}"
    proof (rule exI [of _ "λn. n"])
      show "∀i. ((λn. n) (Suc i), (λn. n) i) ∈ {(y, x). P y x}" unfolding P_def by simp
    qed
    then show False using w unfolding wf_iff_no_infinite_down_chain by blast
  qed
```

```
qed

(* Contrarias: un mundo mixto en que las dos fallan. El 0 es simple (la antítesis falla) y el 1 es
compuesto sin
   ninguna parte simple (la tesis falla): por encima del 0, cada pieza x se parte en x + 1 sin fin. *)
lemma mixto_contrarias:
  fixes P :: "nat ⇒ nat ⇒ bool"
  defines "P ≡ λy x. 0 < x ∧ x < y"
  shows "(∀x y z. P x y ∧ P y z ⟶ P x z) ∧ (∃s. ¬ (∃y. P y s))
         ∧ (∃x. (∃y. P y x) ∧ ¬ (∃s. ¬ (∃y. P y s) ∧ P s x))"
proof (intro conjI)
  show "∀x y z. P x y ∧ P y z ⟶ P x z" unfolding P_def by simp
  show "∃s. ¬ (∃y. P y s)"
  proof (rule exI [of _ 0])
    show "¬ (∃y. P y 0)" unfolding P_def by simp
  qed
  show "∃x. (∃y. P y x) ∧ ¬ (∃s. ¬ (∃y. P y s) ∧ P s x)"
  proof (rule exI [of _ 1], rule conjI)
    show "∃y. P y 1"
    proof (rule exI [of _ 2])
      show "P 2 1" unfolding P_def by simp
    qed
    show "¬ (∃s. ¬ (∃y. P y s) ∧ P s 1)"
    proof
      assume "∃s. ¬ (∃y. P y s) ∧ P s 1"
      then obtain s where ns: "¬ (∃y. P y s)" and p: "P s 1" by blast
      from p have "1 < s" unfolding P_def by simp
      then have "P (Suc s) s" unfolding P_def by simp
      then show False using ns by blast
    qed
  qed
qed

end
```

Las antinomias de Kant: la libertad y el ser necesario

La tercera y la cuarta antinomia: tesis, antítesis, el choque, la salida de Kant (verdaderas las dos, en planos distintos) y la otra salida.

Fichero `Antinomias_dinamicas.thy` ▪ 15 resultados demostrados ▪ 292 líneas

```
theory Antinomias_dinamicas
  imports Main
begin

(* Quinto mundo del Laboratorio (22-10-2026, propuesta del profesor): las antinomias de Kant, empezando
por las dos
   dinámicas (Crítica de la razón pura, A444-461 / B472-489). Esqueleto lógico, no el idealismo
trascendental:
   - cada prueba de Kant es una reducción al absurdo que depende de una premisa propia;
   - las premisas propias de tesis y antítesis chocan si se aplican al mismo dominio;
   - la solución de Kant para las dinámicas (A532-558 / B560-586): la tesis vale de lo inteligible y la
antítesis de los
     fenómenos; con dos dominios y dos relaciones de causalidad, hay un modelo de las dos a la vez.
   Lo que Kant llama «la serie de las causas completa» se traduce aquí como buena fundación (no hay
regreso al infinito
   hacia atrás), como en Cinco_vias.thy. Es una interpretación, y la mesa web lo dirá.
   Pruebas escritas paso a paso, sin auto ni presburger sobre aritmética (en Cinco_vias.thy una búsqueda
no terminaba). *)

section ‹Tercera antinomia: libertad y causalidad natural (A444-451 / B472-479)›

typedecl suceso
consts Causa :: "suceso ⇒ suceso ⇒ bool"   (* Causa y x: el estado y determina a x según una ley natural
*)

(* Libre, en sentido cosmológico: comienza por sí una serie; ningún estado anterior lo determina. *)
definition libre :: "suceso ⇒ bool" where "libre x ≡ ¬ (∃y. Causa y x)"

(* Premisa propia de la tesis: la serie de las causas de lo que sucede está completa (no hay regreso al
infinito). *)
definition T3_serie_completa :: bool where "T3_serie_completa ≡ wf {(y, x). Causa y x}"
(* Premisa propia de la antítesis: la ley natural, todo lo que sucede tiene un estado anterior que lo
determina. *)
definition A3_ley_natural :: bool where "A3_ley_natural ≡ ∀x. ∃y. Causa y x"

(* Tesis: si la serie está completa, hay causalidad por libertad (algún suceso no lo determina ningún
estado anterior). *)
theorem tesis3:
  assumes "T3_serie_completa"
  shows "∃x. libre x"
proof -
  let ?r = "{(y, x). Causa y x}"
  have w: "wf ?r" using assms unfolding T3_serie_completa_def .
  then have "∃z∈(UNIV :: suceso set). ∀y. (y, z) ∈ ?r ⟶ y ∉ UNIV"
    by (rule wf_eq_minimal[THEN iffD1, rule_format, OF _ UNIV_I])
  then obtain x where "∀y. (y, x) ∈ ?r ⟶ y ∉ (UNIV :: suceso set)" by blast
  then have "¬ (∃y. Causa y x)" by simp
  then show ?thesis unfolding libre_def by blast
qed

(* Antítesis: si rige la ley natural, no hay libertad. *)
theorem antitesis3:
  assumes "A3_ley_natural"
  shows "¬ (∃x. libre x)"
  using assms unfolding A3_ley_natural_def libre_def by blast

(* Las dos premisas, sobre el mismo dominio de sucesos, no caben juntas. *)
theorem choque3:
  assumes "T3_serie_completa" "A3_ley_natural"
  shows False
  using tesis3[OF assms(1)] antitesis3[OF assms(2)] by blast
```

```
(* Cada lado, por separado, tiene un modelo: la tesis, con los naturales (el 0 no tiene causa); la
antítesis, con los
   enteros (cada suceso tiene una causa anterior, sin fin hacia atrás). *)
lemma modelo_tesis3:
  fixes C :: "nat ⇒ nat ⇒ bool"
  defines "C ≡ λy x. x = Suc y"
  shows "wf {(y, x). C y x} ∧ ¬ (∀x. ∃y. C y x)"
proof (rule conjI)
  have "{(y, x). C y x} = pred_nat" unfolding C_def pred_nat_def by simp
  then show "wf {(y, x). C y x}" using wf_pred_nat by simp
  show "¬ (∀x. ∃y. C y x)"
  proof
    assume "∀x. ∃y. C y x"
    then obtain y where "C y 0" by blast
    then show False unfolding C_def by simp
  qed
qed

lemma modelo_antitesis3:
  fixes C :: "int ⇒ int ⇒ bool"
  defines "C ≡ λy x. x = y + 1"
  shows "(∀x. ∃y. C y x) ∧ ¬ wf {(y, x). C y x}"
proof (rule conjI)
  show "∀x. ∃y. C y x"
  proof
    fix x :: int
    show "∃y. C y x" unfolding C_def by (rule exI [of _ "x - 1"]) simp
  qed
  show "¬ wf {(y, x). C y x}"
  proof
    assume w: "wf {(y, x). C y x}"
    have h: "∀i. (- int (Suc i), - int i) ∈ {(y, x). C y x}" unfolding C_def by simp
    have "∃f. ∀i. (f (Suc i), f i) ∈ {(y, x). C y x}"
    proof (rule exI [of _ "λn. - int n"])
      show "∀i. ((λn. - int n) (Suc i), (λn. - int n) i) ∈ {(y, x). C y x}" using h by simp
    qed
    then show False using w unfolding wf_iff_no_infinite_down_chain by blast
  qed
qed

(* La solución de Kant (A532-558 / B560-586): un mismo efecto puede tener una causa empírica, entre
fenómenos, y una
   causa inteligible, que no es un fenómeno. Modelo: los fenómenos son Some k (k entero) y lo inteligible
es None.
   Causa natural (C): de Some k a Some (k+1). Causa inteligible (I): de None a cada fenómeno.
   Se cumplen a la vez la antítesis para los fenómenos (todo fenómeno tiene causa natural entre fenómenos)
y la tesis
   para lo inteligible (hay algo sin causa natural que es causa, inteligible, de los fenómenos). *)
lemma solucion3:
  fixes F :: "int option ⇒ bool" and C I :: "int option ⇒ int option ⇒ bool"
  defines "F ≡ λx. x ≠ None"
    and "C ≡ λy x. ∃k. y = Some k ∧ x = Some (k + 1)"
    and "I ≡ λy x. y = None ∧ x ≠ None"
  shows "(∀x. F x ⟶ (∃y. F y ∧ C y x)) ∧ (∃x. ¬ F x ∧ ¬ (∃y. C y x) ∧ (∀z. F z ⟶ I x z))"
proof (rule conjI)
  show "∀x. F x ⟶ (∃y. F y ∧ C y x)"
  proof (intro allI impI)
    fix x :: "int option"
    assume "F x"
    then obtain k where k: "x = Some k" unfolding F_def by (cases x) simp_all
    show "∃y. F y ∧ C y x"
    proof (rule exI [of _ "Some (k - 1)"])
      show "F (Some (k - 1)) ∧ C (Some (k - 1)) x" unfolding F_def C_def k by simp
    qed
  qed
  show "∃x. ¬ F x ∧ ¬ (∃y. C y x) ∧ (∀z. F z ⟶ I x z)"
  proof (rule exI [of _ None])
```

```
    show "¬ F None ∧ ¬ (∃y. C y None) ∧ (∀z. F z ⟶ I None z)" unfolding F_def C_def I_def by simp
  qed
qed

section ‹Cuarta antinomia: el ser necesario (A452-461 / B480-489)›

typedecl ser
consts Cond :: "ser ⇒ ser ⇒ bool"   (* Cond y x: y es condición de la existencia de x *)
consts EnMundo :: "ser ⇒ bool"          (* x pertenece al mundo sensible: está en el tiempo *)

(* Absolutamente necesario: incondicionado, su existencia no depende de ninguna condición. *)
definition incondicionado :: "ser ⇒ bool" where "incondicionado x ≡ ¬ (∃y. Cond y x)"

(* Premisa propia de la tesis: la serie de las condiciones está completa (no hay regreso al infinito). *)
definition T4_serie_completa :: bool where "T4_serie_completa ≡ wf {(y, x). Cond y x}"
(* Premisa que usan las dos partes: lo que condiciona algo que está en el tiempo está también en el
tiempo. *)
definition T4_condicion_en_el_tiempo :: bool where
  "T4_condicion_en_el_tiempo ≡ ∀x z. Cond x z ∧ EnMundo z ⟶ EnMundo x"
(* Premisa propia de la antítesis: la ley dinámica, todo lo que hay en el mundo tiene una condición. *)
definition A4_ley_dinamica :: bool where "A4_ley_dinamica ≡ ∀x. EnMundo x ⟶ (∃y. Cond y x)"

(* Tesis: con la serie completa, si hay mundo, al mundo pertenece algo absolutamente necesario. *)
theorem tesis4:
  assumes "T4_serie_completa" "T4_condicion_en_el_tiempo" "∃z. EnMundo z"
  shows "∃x. EnMundo x ∧ incondicionado x"
proof -
  let ?r = "{(y, x). Cond y x}" and ?S = "{x. EnMundo x}"
  obtain z where "EnMundo z" using assms(3) by blast
  then have "z ∈ ?S" by simp
  have "∃m∈?S. ∀y. (y, m) ∈ ?r ⟶ y ∉ ?S"
    using wf_eq_minimal[THEN iffD1, OF assms(1)[unfolded T4_serie_completa_def], rule_format, OF ‹z ∈ ?
S›] .
  then obtain m where m: "m ∈ ?S" and min: "∀y. (y, m) ∈ ?r ⟶ y ∉ ?S" by blast
  have "incondicionado m" unfolding incondicionado_def
  proof
    assume "∃y. Cond y m"
    then obtain y where c: "Cond y m" by blast
    have "EnMundo m" using m by simp
    then have "EnMundo y" using c assms(2) unfolding T4_condicion_en_el_tiempo_def by blast
    moreover have "(y, m) ∈ ?r" using c by simp
    ultimately show False using min by simp
  qed
  moreover have "EnMundo m" using m by simp
  ultimately show ?thesis by blast
qed

(* Antítesis, primera parte: con la ley dinámica, nada en el mundo es incondicionado. *)
theorem antitesis4_en_el_mundo:
  assumes "A4_ley_dinamica"
  shows "¬ (∃x. EnMundo x ∧ incondicionado x)"
  using assms unfolding A4_ley_dinamica_def incondicionado_def by blast

(* Antítesis, segunda parte: fuera del mundo tampoco, porque lo que condiciona algo del mundo estaría en
el tiempo. *)
theorem antitesis4_fuera:
  assumes "T4_condicion_en_el_tiempo"
  shows "¬ (∃x. ¬ EnMundo x ∧ (∃z. EnMundo z ∧ Cond x z))"
  using assms unfolding T4_condicion_en_el_tiempo_def by blast

theorem choque4:
  assumes "T4_serie_completa" "T4_condicion_en_el_tiempo" "A4_ley_dinamica" "∃z. EnMundo z"
  shows False
  using tesis4[OF assms(1) assms(2) assms(4)] antitesis4_en_el_mundo[OF assms(3)] by blast

(* El paso del argumento cosmológico, que Kant examina aquí: de «cada miembro de la serie está
condicionado» no se
   sigue «la serie entera tiene una condición fuera de ella». Una serie (un conjunto S) está condicionada
```

```
como un todo si
   algo de fuera de S condiciona algo de S. La serie de todo lo que hay no lo está nunca, aunque cada
miembro tenga su
   condición: hace falta una premisa más. Es lógica de orden superior: se habla de conjuntos de seres. *)
definition serie_condicionada :: "ser set ⇒ bool" where
  "serie_condicionada S ≡ ∃y. y ∉ S ∧ (∃x∈S. Cond y x)"

lemma el_todo_no_tiene_condicion_exterior: "¬ serie_condicionada UNIV"
  unfolding serie_condicionada_def by simp

lemma cada_miembro_no_es_el_todo:
  assumes "∀x. ∃y. Cond y x"
  shows "(∀x∈(UNIV :: ser set). ∃y. Cond y x) ∧ ¬ serie_condicionada UNIV"
  using assms el_todo_no_tiene_condicion_exterior by blast

(* Con la serie completa, la otra salida (nada necesario ni en el mundo ni fuera de él) es imposible,
aunque no se
   use la premisa «la condición está en el tiempo»: basta con que nada de fuera condicione el mundo. *)
lemma salida4_con_serie_completa:
  assumes "T4_serie_completa" "∃z. EnMundo z" "∀x. ¬ EnMundo x ⟶ ¬ (∃y. EnMundo y ∧ Cond x y)"
  shows "∃x. EnMundo x ∧ incondicionado x"
proof -
  let ?r = "{(y, x). Cond y x}" and ?S = "{x. EnMundo x}"
  obtain z where "EnMundo z" using assms(2) by blast
  then have "z ∈ ?S" by simp
  have "∃m∈?S. ∀y. (y, m) ∈ ?r ⟶ y ∉ ?S"
    using wf_eq_minimal[THEN iffD1, OF assms(1)[unfolded T4_serie_completa_def], rule_format, OF ‹z ∈ ?
S›] .
  then obtain m where m: "m ∈ ?S" and min: "∀y. (y, m) ∈ ?r ⟶ y ∉ ?S" by blast
  have "incondicionado m" unfolding incondicionado_def
  proof
    assume "∃y. Cond y m"
    then obtain y where c: "Cond y m" by blast
    have "EnMundo m" using m by simp
    then have "EnMundo y" using c assms(3) by blast
    moreover have "(y, m) ∈ ?r" using c by simp
    ultimately show False using min by simp
  qed
  moreover have "EnMundo m" using m by simp
  ultimately show ?thesis by blast
qed

(* Otra salida, no la de Kant: soltar la premisa de la tesis (la serie completa). Con un regreso infinito
de
   condiciones, todo en el mundo está condicionado y nada es necesario, sin contradicción: es la objeción
de Cleantes en los
   Diálogos de Hume (IX), y la que se sigue del infinito actual de Cantor. Modelo: los enteros, cada uno
condicionado por
   el anterior, todos en el mundo. *)
lemma modelo_antitesis4:
  fixes M :: "int ⇒ bool" and C :: "int ⇒ int ⇒ bool"
  defines "M ≡ λx. True"
    and "C ≡ λy x. x = y + 1"
  shows "(∀x z. C x z ∧ M z ⟶ M x) ∧ (∀x. M x ⟶ (∃y. C y x)) ∧ (∃z. M z)
         ∧ ¬ (∃x. M x ∧ ¬ (∃y. C y x)) ∧ ¬ wf {(y, x). C y x}"
proof (intro conjI)
  show "∀x z. C x z ∧ M z ⟶ M x" unfolding M_def by simp
  show "∀x. M x ⟶ (∃y. C y x)"
  proof (intro allI impI)
    fix x :: int
    show "∃y. C y x" unfolding C_def by (rule exI [of _ "x - 1"]) simp
  qed
  show "∃z. M z" unfolding M_def by simp
  show "¬ (∃x. M x ∧ ¬ (∃y. C y x))"
  proof
    assume "∃x. M x ∧ ¬ (∃y. C y x)"
    then obtain x where n: "¬ (∃y. C y x)" by blast
    have "C (x - 1) x" unfolding C_def by simp
```

```
    then show False using n by blast
  qed
  show "¬ wf {(y, x). C y x}"
  proof
    assume w: "wf {(y, x). C y x}"
    have h: "∀i. (- int (Suc i), - int i) ∈ {(y, x). C y x}" unfolding C_def by simp
    have "∃f. ∀i. (f (Suc i), f i) ∈ {(y, x). C y x}"
    proof (rule exI [of _ "λn. - int n"])
      show "∀i. ((λn. - int n) (Suc i), (λn. - int n) i) ∈ {(y, x). C y x}" using h by simp
    qed
    then show False using w unfolding wf_iff_no_infinite_down_chain by blast
  qed
qed

(* La solución de Kant: el ser necesario no es un miembro de la serie empírica sino su fundamento
inteligible, fuera
   del tiempo. Modelo: en el mundo, Some k; fuera, None. Condición empírica (E): de Some k a Some (k+1);
fundamento
   inteligible (G): de None a todo lo del mundo. Para E vale «la condición de algo del mundo está en el
mundo» y la ley
   dinámica; G no es una condición en el tiempo. *)
lemma solucion4:
  fixes M :: "int option ⇒ bool" and E G :: "int option ⇒ int option ⇒ bool"
  defines "M ≡ λx. x ≠ None"
    and "E ≡ λy x. ∃k. y = Some k ∧ x = Some (k + 1)"
    and "G ≡ λy x. y = None ∧ x ≠ None"
  shows "(∀x z. E x z ∧ M z ⟶ M x) ∧ (∀x. M x ⟶ (∃y. E y x))
         ∧ (∃n. ¬ M n ∧ ¬ (∃y. E y n) ∧ (∀z. M z ⟶ G n z))"
proof (intro conjI)
  show "∀x z. E x z ∧ M z ⟶ M x"
  proof (intro allI impI)
    fix x z :: "int option"
    assume "E x z ∧ M z"
    then obtain k where "x = Some k" unfolding E_def by blast
    then show "M x" unfolding M_def by simp
  qed
  show "∀x. M x ⟶ (∃y. E y x)"
  proof (intro allI impI)
    fix x :: "int option"
    assume "M x"
    then obtain k where k: "x = Some k" unfolding M_def by (cases x) simp_all
    show "∃y. E y x"
    proof (rule exI [of _ "Some (k - 1)"])
      show "E (Some (k - 1)) x" unfolding E_def k by simp
    qed
  qed
  show "∃n. ¬ M n ∧ ¬ (∃y. E y n) ∧ (∀z. M z ⟶ G n z)"
  proof (rule exI [of _ None])
    show "¬ M None ∧ ¬ (∃y. E y None) ∧ (∀z. M z ⟶ G None z)" unfolding M_def E_def G_def by simp
  qed
qed

end
```

Del ser al cosmos: Parménides y sus herederos

Lo uno y lo inmóvil salen del mismo salto; los atomistas, Meliso y la dicotomía de Zenón.

Fichero `Parmenides_herederos.thy` · 12 resultados demostrados · 179 líneas

```
(* Parménides y sus herederos (23-10-2026). Sexto mundo de la mesa «Spinoza», a propuesta del profesor.
   Fuentes: Parménides, fragmentos B2, B3, B6, B7, B8 (Diels-Kranz); Meliso, B1-B7; Aristóteles,
Refutaciones
   sofísticas 5, 167b13-20, y Física I 3, 186a10-16 (la falacia de Meliso); Zenón según Aristóteles,
Física VI 9,
   239b11-14 (la dicotomía). Traducciones propias. Debate abierto: la lógica muestra qué premisas hacen
qué. *)
theory Parmenides_herederos
  imports Main "HOL.Real" "HOL.Series"
begin

section ‹Parménides: el ser y el «es» (B2, B3, B6, B8)›

typedecl cosa
typedecl instante
consts Existe :: "cosa ⇒ instante ⇒ bool"   (* x es, en el sentido de «hay x», en t *)

(* B6.1-2: lo que es, es, y el no ser no es. Leído sobre instantes: nada de lo que es deja de ser ni
empieza a ser. *)
definition P_no_ser :: bool where "P_no_ser ≡ ∀x t. Existe x t"

(* El salto que la mesa examina: leer «no ser F» como «no ser» sin más. Es lógica de orden superior: vale
para
   cualquier propiedad P. Con P = «ser blanco en t», dejar de ser blanco sería dejar de ser; con P = «ser
y»,
   ser distinto de y sería no ser. *)
definition P_lectura_existencial :: bool where
  "P_lectura_existencial ≡ ∀(P :: cosa ⇒ bool) x t. ¬ P x ⟶ ¬ Existe x t"

(* Los dos casos del salto que usa la mesa, en primer orden: «ser otro que y» leído como «no ser», y «no
ser
   caliente» leído como «no ser». Los dos se siguen de la lectura existencial. *)
consts Caliente :: "cosa ⇒ instante ⇒ bool"
definition P_salto_otro :: bool where "P_salto_otro ≡ ∀x y t. x ≠ y ⟶ ¬ Existe x t"
definition P_salto_cambio :: bool where "P_salto_cambio ≡ ∀x t. ¬ Caliente x t ⟶ ¬ Existe x t"

lemma salto_casos:
  assumes "P_lectura_existencial"
  shows "P_salto_otro ∧ P_salto_cambio"
proof
  show "P_salto_otro" unfolding P_salto_otro_def
  proof (intro allI impI)
    fix x y :: cosa and t :: instante
    assume "x ≠ y"
    then have "¬ (λz. z = y) x" by simp
    then show "¬ Existe x t" using assms unfolding P_lectura_existencial_def by blast
  qed
  show "P_salto_cambio" unfolding P_salto_cambio_def
  proof (intro allI impI)
    fix x :: cosa and t :: instante
    assume "¬ Caliente x t"
    then have "¬ (λz. Caliente z t) x" by simp
    then show "¬ Existe x t" using assms unfolding P_lectura_existencial_def by blast
  qed
qed

(* En la mesa: con el no ser y el salto, el ser es uno y nada cambia. *)
theorem uno_mesa:
  assumes "P_no_ser" "P_salto_otro"
  shows "∀x y :: cosa. x = y"
  using assms unfolding P_no_ser_def P_salto_otro_def by blast
```

```
theorem inmovil_mesa:
  assumes "P_no_ser" "P_salto_cambio"
  shows "∀x t u. Caliente x t ⟷ Caliente x u"
  using assms unfolding P_no_ser_def P_salto_cambio_def by blast

(* B8.22-25: el ser es uno. Del no ser y del salto se sigue que no hay dos cosas. *)
theorem uno:
  assumes "P_no_ser" "P_lectura_existencial"
  shows "∀x y :: cosa. x = y"
proof (intro allI)
  fix x y :: cosa
  show "x = y"
  proof (rule ccontr)
    assume d: "x ≠ y"
    obtain t :: instante where True by simp
    have "¬ (λz. z = y) x" using d by simp
    then have "¬ Existe x t" using assms(2) unfolding P_lectura_existencial_def by blast
    moreover have "Existe x t" using assms(1) unfolding P_no_ser_def by blast
    ultimately show False by simp
  qed
qed

(* B8.26-41: el ser no cambia. Con F x t, «x es F en t», nada puede ser F en un instante y no serlo en
otro. *)
theorem inmovil:
  fixes F :: "cosa ⇒ instante ⇒ bool"
  assumes "P_no_ser" "P_lectura_existencial"
  shows "∀x t. F x t"
proof (intro allI)
  fix x t
  show "F x t"
  proof (rule ccontr)
    assume n: "¬ F x t"
    have "¬ (λz. F z t) x" using n by simp
    then have "¬ Existe x t" using assms(2) unfolding P_lectura_existencial_def by blast
    moreover have "Existe x t" using assms(1) unfolding P_no_ser_def by blast
    ultimately show False by simp
  qed
qed

(* Sin el salto, el no ser no impide ni la pluralidad ni el cambio: dos cosas (los booleanos), que existen
siempre,
   y una de ellas es F en un instante y no en otro. Es la salida de Platón en el Sofista (241d, el
«parricidio»):
   «no ser» puede querer decir «ser otro», no «no ser en absoluto». *)
lemma sin_el_salto:
  fixes E :: "bool ⇒ nat ⇒ bool" and F :: "bool ⇒ nat ⇒ bool"
  defines "E ≡ λx t. True" and "F ≡ λx t. x ∧ t = 0"
  shows "(∀x t. E x t) ∧ True ≠ False ∧ F True 0 ∧ ¬ F True 1"
  unfolding E_def F_def by simp

section ‹Meliso: lo no generado no tiene principio (B1-B2)›

(* Aristóteles, Refutaciones sofísticas 5, 167b13-20: Meliso razona «lo generado tiene principio; el ser
no es
   generado; luego no tiene principio». Es negar el antecedente: la conversión de la condicional no vale.
*)
consts Generado :: "cosa ⇒ bool"
consts Principio :: "cosa ⇒ bool"
definition M_generado_principio :: bool where "M_generado_principio ≡ ∀x. Generado x ⟶ Principio x"

lemma falacia_de_Meliso:
  fixes G P :: "bool ⇒ bool"
  defines "G ≡ λx. False" and "P ≡ λx. True"
  shows "(∀x. G x ⟶ P x) ∧ ¬ G True ∧ P True"
  unfolding G_def P_def by simp

(* Con la conversa como premisa, el razonamiento sí vale: el fallo está en la premisa que falta. *)
```

```
theorem Meliso_con_la_conversa:
  assumes "∀x. Principio x ⟶ Generado x" "¬ Generado x"
  shows "¬ Principio x"
  using assms by blast

section ‹Zenón: la dicotomía (Aristóteles, Física VI 9, 239b11-14)›

(* Para recorrer un estadio hay que recorrer antes la mitad, y antes la mitad de lo que queda, y así sin
fin.
   La respuesta moderna: infinitas mitades suman una distancia finita. 1/2 + 1/4 + 1/8 + ... = 1. *)
theorem dicotomia: "(λn. (1/2 :: real) ^ Suc n) sums 1"
proof -
  have "(λn. (1/2 :: real) ^ n) sums (1 / (1 - 1/2))" by (rule geometric_sums) simp
  then have "(λn. (1/2 :: real) * (1/2) ^ n) sums ((1/2) * (1 / (1 - 1/2)))" by (rule sums_mult)
  then show ?thesis by simp
qed

section ‹Lo uno según Aristóteles: lo que separaría dos entes sería el no ser›

(* Aristóteles, Sobre la generación y la corrupción I 8, 325a2-6, lee así a los eléatas: dos entes solo se
   distinguen si entre ellos hay no ser, el vacío; y el vacío no es. Los atomistas aceptan la primera
premisa y
   sueltan la segunda. *)
consts Vacio :: "cosa ⇒ bool"
definition P_sin_vacio :: bool where "P_sin_vacio ≡ ∀x. ¬ Vacio x"
definition P_separa :: bool where "P_separa ≡ ∀x y. x ≠ y ∧ ¬ Vacio x ∧ ¬ Vacio y ⟶ (∃z. Vacio z)"

theorem uno_sin_vacio:
  assumes "P_sin_vacio" "P_separa"
  shows "∀x y. ¬ Vacio x ∧ ¬ Vacio y ⟶ x = y"
  using assms unfolding P_sin_vacio_def P_separa_def by blast

(* Los atomistas: dos átomos y el vacío entre ellos. Se cumple «lo que separa es el vacío». *)
lemma muchos_con_vacio:
  fixes V :: "nat ⇒ bool"
  defines "V ≡ λx. x = 2"
  shows "(∀x y. x ≠ y ∧ ¬ V x ∧ ¬ V y ⟶ (∃z. V z)) ∧ ¬ V 0 ∧ ¬ V 1 ∧ V 2"
  unfolding V_def by auto

section ‹Los atomistas: el vacío (Leucipo y Demócrito, según Aristóteles, Metafísica I 4, 985b4-10)›

(* Los atomistas aceptan que nada viene de la nada, pero sueltan «el no ser no es»: el vacío, el no ser,
es tanto
   como el ser. Modelo: los lugares son números; el lleno se mueve de un lugar a otro a través del vacío.
Hay
   pluralidad (dos átomos) y movimiento, y ningún átomo empieza ni deja de ser. *)
lemma atomistas:
  fixes Lleno :: "nat ⇒ nat ⇒ bool" (* el lugar l está lleno en el instante t *)
  defines "Lleno ≡ λl t. l = t ∨ l = t + 5"
  shows "(∃l t. ¬ Lleno l t) ∧ Lleno 0 0 ∧ ¬ Lleno 0 1 ∧ Lleno 1 1 ∧ (∀t. ∃l1 l2. l1 ≠ l2 ∧ Lleno l1 t ∧
Lleno l2 t)"
proof (intro conjI)
  show "∃l t. ¬ Lleno l t" unfolding Lleno_def by (rule exI [of _ 1], rule exI [of _ 0]) simp
  show "Lleno 0 0" unfolding Lleno_def by simp
  show "¬ Lleno 0 1" unfolding Lleno_def by simp
  show "Lleno 1 1" unfolding Lleno_def by simp
  show "∀t. ∃l1 l2. l1 ≠ l2 ∧ Lleno l1 t ∧ Lleno l2 t"
  proof
    fix t :: nat
    show "∃l1 l2. l1 ≠ l2 ∧ Lleno l1 t ∧ Lleno l2 t" unfolding Lleno_def
      by (rule exI [of _ t], rule exI [of _ "t + 5"]) simp
  qed
qed

end
```

Del ser al cosmos: el Demiurgo

El Timeo de Platón: el hacedor, el modelo eterno, un solo cosmos, si empezó o no y el tiempo que nace con él.

Fichero `Timeo_demiurgo.thy` ▪ 14 resultados demostrados ▪ 177 líneas

```
(* El Demiurgo del Timeo (23-10-2026). Sexto mundo de la mesa «Spinoza», a propuesta del profesor.
   Fuentes: Platón, Timeo 27d-29a, 29e-30a, 31a-b, 37d-38b (ed. Burnet); Aristóteles, Sobre el cielo I 10,
   279b17-280a11 (la lectura literal y la «didáctica» de Jenócrates). Traducciones propias. Platón llama a
su relato
   eikὸs mŷthos, relato verosímil: la lógica muestra su estructura, no si es verdad. Debate abierto, sin
veredicto. *)
theory Timeo_demiurgo
  imports Main
begin

typedecl ente
consts Siempre :: "ente ⇒ bool"          (* lo que siempre es y no deviene *)
consts Deviene :: "ente ⇒ bool"          (* lo que deviene y nunca es del todo *)
consts Visible :: "ente ⇒ bool"          (* visible y tangible: tiene cuerpo *)
consts Causa :: "ente ⇒ ente ⇒ bool"    (* Causa d x: d hace x *)
consts Bello :: "ente ⇒ bool"
consts Bueno :: "ente ⇒ bool"
consts Copia :: "ente ⇒ ente ⇒ bool"    (* Copia x m: x está hecho mirando al modelo m *)
consts cosmos :: ente

section ‹Hay un hacedor y un modelo eterno (27d-29a)›

(* 27d-28a: dos planos, lo que siempre es y lo que deviene; nada está en los dos. *)
definition D_dos_planos :: bool where "D_dos_planos ≡ ∀x. ¬ (Siempre x ∧ Deviene x)"
(* 28a: todo lo que deviene deviene por alguna causa. *)
definition D_causa :: bool where "D_causa ≡ ∀x. Deviene x ⟶ (∃d. Causa d x)"
(* 28b-c: lo visible y tangible deviene; y el cosmos es visible y tangible. *)
definition D_visible :: bool where "D_visible ≡ ∀x. Visible x ⟶ Deviene x"
definition D_cosmos_visible :: bool where "D_cosmos_visible ≡ Visible cosmos"
(* La mesa no tiene nombres propios: habla de lo visible, sea lo que sea. Las mismas premisas, para todo
x. *)
definition D_bello :: bool where "D_bello ≡ ∀x. Visible x ⟶ Bello x"
definition D_hacedor_bueno :: bool where "D_hacedor_bueno ≡ ∀d x. Causa d x ∧ Visible x ⟶ Bueno d"
(* 29a: si el cosmos es bello y su hacedor bueno, este miró a un modelo eterno. Son premisas de valor. *)
definition D_bello_cosmos :: bool where "D_bello_cosmos ≡ Bello cosmos"
definition D_hacedor_bueno_cosmos :: bool where "D_hacedor_bueno_cosmos ≡ ∀d. Causa d cosmos ⟶ Bueno d"
definition D_modelo :: bool where
  "D_modelo ≡ ∀d x. Causa d x ∧ Bello x ∧ Bueno d ⟶ (∃m. Siempre m ∧ Copia x m)"

theorem hay_hacedor:
  assumes "D_causa" "D_visible" "D_cosmos_visible"
  shows "∃d. Causa d cosmos"
  using assms unfolding D_causa_def D_visible_def D_cosmos_visible_def by blast

theorem modelo_eterno:
  assumes "D_causa" "D_visible" "D_cosmos_visible" "D_bello_cosmos" "D_hacedor_bueno_cosmos" "D_modelo"
  shows "∃m. Siempre m ∧ Copia cosmos m"
proof -
  obtain d where c: "Causa d cosmos" using hay_hacedor[OF assms(1-3)] by blast
  have "Bueno d" using c assms(5) unfolding D_hacedor_bueno_cosmos_def by blast
  then show ?thesis using c assms(4,6) unfolding D_bello_cosmos_def D_modelo_def by blast
qed

theorem hay_hacedor_mesa:
  assumes "D_causa" "D_visible"
  shows "∀x. Visible x ⟶ (∃d. Causa d x)"
  using assms unfolding D_causa_def D_visible_def by blast

theorem modelo_eterno_mesa:
  assumes "D_causa" "D_visible" "D_bello" "D_hacedor_bueno" "D_modelo"
  shows "∀x. Visible x ⟶ (∃m. Siempre m ∧ Copia x m)"
```

```
proof (intro allI impI)
  fix x assume v: "Visible x"
  obtain d where c: "Causa d x" using v assms(1,2) unfolding D_causa_def D_visible_def by blast
  have "Bueno d" using c v assms(4) unfolding D_hacedor_bueno_def by blast
  moreover have "Bello x" using v assms(3) unfolding D_bello_def by blast
  ultimately show "∃m. Siempre m ∧ Copia x m" using c assms(5) unfolding D_modelo_def by blast
qed

(* El modelo no es una copia de lo que deviene: está en el otro plano. *)
theorem el_modelo_no_deviene:
  assumes "D_dos_planos" "Siempre m"
  shows "¬ Deviene m"
  using assms unfolding D_dos_planos_def by blast

(* Sin la premisa del hacedor bueno, nada obliga a un modelo eterno: Platón dice que pensarlo «no es
lícito ni
   decirlo» (29a), pero eso ya no es lógica. Modelo: el hacedor no es bueno y el cosmos no copia nada. *)
lemma sin_hacedor_bueno:
  fixes S Dv V B Bu :: "bool ⇒ bool" and C Cp :: "bool ⇒ bool ⇒ bool"
  defines "S ≡ λx. False" and "Dv ≡ λx. x" and "V ≡ λx. x" and "B ≡ λx. True" and "Bu ≡ λx. False"
    and "C ≡ λd x. ¬ d ∧ x" and "Cp ≡ λx m. False"
  shows "(∀x. Dv x ⟶ (∃d. C d x)) ∧ (∀x. V x ⟶ Dv x) ∧ V True ∧ B True
         ∧ (∀d x. C d x ∧ B x ∧ Bu d ⟶ (∃m. S m ∧ Cp x m)) ∧ ¬ (∃m. S m ∧ Cp True m)"
  unfolding S_def Dv_def V_def B_def Bu_def C_def Cp_def by auto

section ‹El cosmos es uno (31a-b)›

(* Si hubiera dos cosmos copiados del mismo modelo, haría falta otro modelo que los abarcara a los dos, y
serían
   copia de ese, no del primero. Y un cosmos es copia de un solo modelo, el completo. *)
definition U_dos_copias :: bool where
  "U_dos_copias ≡ ∀c1 c2 m. Copia c1 m ∧ Copia c2 m ∧ c1 ≠ c2 ⟶ (∃m'. Copia c1 m' ∧ m' ≠ m)"
definition U_un_modelo :: bool where "U_un_modelo ≡ ∀c m m'. Copia c m ∧ Copia c m' ⟶ m = m'"

theorem un_solo_cosmos:
  assumes "U_dos_copias" "U_un_modelo" "Copia cosmos m" "Copia c m"
  shows "c = cosmos"
proof (rule ccontr)
  assume "c ≠ cosmos"
  then obtain m' where "Copia c m'" "m' ≠ m" using assms(1,3,4) unfolding U_dos_copias_def by blast
  then show False using assms(2,4) unfolding U_un_modelo_def by blast
qed

theorem un_solo_cosmos_mesa:
  assumes "U_dos_copias" "U_un_modelo"
  shows "∀c1 c2 m. Copia c1 m ∧ Copia c2 m ⟶ c1 = c2"
  using assms unfolding U_dos_copias_def U_un_modelo_def by blast

(* Todo el peso está en U_dos_copias: sin ella, dos cosmos pueden copiar el mismo modelo. *)
lemma sin_dos_copias:
  fixes Cp :: "nat ⇒ nat ⇒ bool"
  defines "Cp ≡ λx m. m = 0 ∧ (x = 1 ∨ x = 2)"
  shows "(∀c m m'. Cp c m ∧ Cp c m' ⟶ m = m') ∧ Cp 1 0 ∧ Cp 2 0"
  unfolding Cp_def by simp

section ‹¿Empezó el cosmos? (28b: gégonen, «ha llegado a ser»)›

typedecl instante
consts Antes :: "instante ⇒ instante ⇒ bool"
consts Existe :: "ente ⇒ instante ⇒ bool"
definition comienza :: "ente ⇒ bool" where
  "comienza x ≡ ∃t. Existe x t ∧ (∀s. Antes s t ⟶ ¬ Existe x s)"

(* La lectura literal, la de Aristóteles (Sobre el cielo I 10): lo hecho empieza a ser. *)
definition L_literal :: bool where "L_literal ≡ ∀d x. Causa d x ⟶ comienza x"

theorem lectura_literal:
  assumes "L_literal" "D_causa" "D_visible" "D_cosmos_visible"
```

```
  shows "comienza cosmos"
  using hay_hacedor[OF assms(2-4)] assms(1) unfolding L_literal_def by blast

theorem lectura_literal_mesa:
  assumes "L_literal" "D_causa" "D_visible"
  shows "∀x. Visible x ⟶ comienza x"
  using assms unfolding L_literal_def D_causa_def D_visible_def by blast

(* Con la lectura literal, nada hecho deja de comenzar: es la premisa misma. *)
lemma literal_comienza:
  assumes "L_literal" "Causa d x"
  shows "comienza x"
  using assms unfolding L_literal_def by blast

(* La lectura de dependencia (Jenócrates y los neoplatónicos): «hecho» quiere decir «causado», no
«empezado».
   Modelo: el cosmos es causado y existe en todos los instantes, que son los enteros, sin primero. *)
lemma lectura_dependencia:
  fixes C :: "nat ⇒ nat ⇒ bool" and E :: "nat ⇒ int ⇒ bool"
  defines "C ≡ λd x. d = 0 ∧ x = 1" and "E ≡ λx t. True"
  shows "C 0 1 ∧ ¬ (∃t. E 1 t ∧ (∀s. s < t ⟶ ¬ E 1 s))"
proof (intro conjI)
  show "C 0 1" unfolding C_def by simp
  show "¬ (∃t. E 1 t ∧ (∀s. s < t ⟶ ¬ E 1 s))"
  proof
    assume "∃t. E 1 t ∧ (∀s. s < t ⟶ ¬ E 1 s)"
    then obtain t :: int where "∀s. s < t ⟶ ¬ E 1 s" by blast
    then have "¬ E 1 (t - 1)" by simp
    then show False unfolding E_def by simp
  qed
qed

section ‹El tiempo nace con el cosmos (37d-38b)›

(* El tiempo es «imagen móvil de la eternidad» y nace con el cielo. Modelo: los instantes son los
naturales y el
   cosmos existe en todos. Tiene comienzo (el 0) y, sin embargo, no hay ningún instante en que no
existiera: no hay
   un «antes» vacío. Es soltar la premisa A1_tiempo_sin_principio de la primera antinomia. *)
lemma tiempo_con_el_cosmos:
  fixes E :: "nat ⇒ bool"
  defines "E ≡ λt. True"
  shows "(∃t0. E t0 ∧ (∀s. s < t0 ⟶ ¬ E s)) ∧ (∀t. E t) ∧ ¬ (∀t::nat. ∃s. s < t)"
proof (intro conjI)
  show "∃t0. E t0 ∧ (∀s. s < t0 ⟶ ¬ E s)" unfolding E_def by (rule exI [of _ 0]) simp
  show "∀t. E t" unfolding E_def by simp
  show "¬ (∀t::nat. ∃s. s < t)"
  proof
    assume "∀t::nat. ∃s. s < t"
    then obtain s :: nat where "s < 0" by blast
    then show False by simp
  qed
qed

end
```